



UMCS

UNIwersytet Marii Curie-Skłodowskiej
w Lublinie

Wydział Matematyki, Fizyki i Informatyki

Kierunek: **Geoinformatyka**

Specjalność:

Krystian Banach

Nr albumu: 307619

Projekt i implementacja oprogramowania do analizy i przetwarzania tras GPX na potrzeby zawodów sportowych

Design and implementation of software for analyzing and
processing GPX routes for sports competitions

Praca magisterska
napisana w Katedrze Oprogramowania Systemów Informatycznych
Instytutu Informatyki
pod kierunkiem dr Michała Klisowskiego

Lublin 2026

Spis treści

Wstęp	5
1 Teoretyczne podstawy analizy tras GPX	7
1.1 Pamięć i wydajność obliczeń	7
1.1.1 Pamięć podręczna i hierarchia pamięci	8
1.1.2 Równoległe wykonywanie operacji	9
1.1.3 Wektoryzacja	9
1.2 Dane GPS i format GPX	10
1.2.1 Charakterystyka danych GPS	10
1.2.2 Struktura pliku GPX	11
1.2.3 Układy współrzędnych w analizie tras	12
1.3 Metody analizy i porównywania tras	13
1.3.1 Obliczanie dystansu i sum podejść i zejść	14
1.3.2 Porównywanie trasy z trasą referencyjną	16
1.3.3 Segmentowe porównanie prędkości	17
1.3.4 Dyskretna odległość Frécheta	20
2 Reprezentacja danych i narzędzia obliczeniowe w Pythonie	23
2.1 Python i reprezentacja danych liczbowych	23
2.1.1 Listy Pythona jako wariant bazowy	24
2.1.2 NumPy i tablice numeryczne	25
2.1.3 Numba i kompilacja JIT	26
3 Projekt i implementacja oprogramowania do analizy tras GPX	29
3.1 Struktura projektu	29
3.2 Implementacja parsera plików GPX	30
3.3 Implementacja operacji analizy tras	33
3.3.1 Obliczanie dystansu i sum podejść i zejść	34
3.3.2 Porównywanie trasy zawodnika z trasą referencyjną	38
3.3.3 Segmentowe porównanie prędkości	47
3.3.4 Obliczanie dyskretnej odległości Frécheta	53

3.4	Implementacja benchmarków	58
4	Metodyka eksperymentów i analiza wyników	63
4.1	Środowisko testowe	63
4.2	Metodyka pomiarów	64
4.3	Wyniki benchmarków	65
4.3.1	Obliczanie dystansu	66
4.3.2	Obliczanie przewyższeń	68
4.3.3	Porównywanie trasy zawodnika z trasą referencyjną	70
4.3.4	Segmentowe porównanie prędkości	72
4.3.5	Dyskretna odległość Frécheta	74
4.4	Wnioski	76
	Podsumowanie	81
	Bibliografia	83
	Spis listingów	87
	Spis rysunków	89
	Spis tabel	91

Wstęp

Rejestrowanie aktywności sportowych za pomocą zegarków, liczników rowerowych i innych odbiorników GPS stało się powszechną praktyką zarówno w treningu amatorskim, jak i w organizacji zawodów sportowych. Efektem takiej rejestracji jest cyfrowy ślad trasy, czyli uporządkowana sekwencja punktów opisujących przebieg aktywności sportowej. Dane te obejmują przede wszystkim kolejne pozycje geograficzne, a często również wysokość, znaczniki czasu oraz dodatkowe informacje związane z aktywnością. Jednym z powszechnie stosowanych formatów wymiany takich danych jest GPX, czyli tekstowy format oparty na XML, przeznaczony do zapisu punktów, tras i śladów geograficznych.

Analiza tras GPX może obejmować zarówno podstawowe operacje wykonywane na pojedynczym śladzie, jak i bardziej złożone metody porównywania śladu zawodnika z trasą referencyjną. Do pierwszej grupy należą między innymi obliczanie długości trasy oraz sumy podejść i zejść. Do drugiej grupy można zaliczyć porównywanie śladu zawodnika z trasą referencyjną, na przykład w celu określenia, czy ślad zawodnika przebiegał w pobliżu trasy referencyjnej. Tego typu operacje mogą mieć znaczenie w analizie aktywności sportowych, planowaniu tras oraz weryfikacji zgodności zarejestrowanego śladu z trasą wzorcową.

Dane GPX mają charakter sekwencyjny i liczbowy, ponieważ ślad trasy składa się z kolejnych punktów opisanych współrzędnymi geograficznymi oraz, w zależności od pliku, dodatkowymi wartościami takimi jak wysokość lub czas pomiaru. Przetwarzanie takich danych wymaga wielokrotnego wykonywania podobnych operacji na kolejnych elementach sekwencji. Wraz ze wzrostem liczby punktów znaczenia nabiera nie tylko dobór samej metody obliczeniowej, lecz także sposób reprezentacji danych oraz sposób wykonania pętli w programie.

Celem pracy jest zaprojektowanie i zaimplementowanie programu do analizy oraz przetwarzania tras zapisanych w formacie GPX, a następnie porównanie wybranych wariantów obliczeniowych pod względem czasu wykonania. W pracy uwzględniono wariant bazowy oparty na listach Pythona, wariant wykorzystujący tablice NumPy oraz wariant wykorzystujący kompilację JIT w bibliotece Numba. Porównanie tych podejść pozwala sprawdzić, w jakich przypadkach operacje tablicowe lub kompilacja pętli numerycznych mogą skrócić czas wykonania, a w jakich struktura algorytmu ogranicza możliwe przyspieszenie.

Zakres pracy obejmuje przygotowanie parsera plików GPX, implementację funkcji

obliczających dystans i przewyższenia, metodę porównywania punktów śladu zawodnika z segmentami trasy referencyjnej, segmentowe porównanie prędkości wykorzystujące znaczniki czasu oraz funkcję wyznaczającą dyskretną odległość Frécheta. Uwzględniono również transformację współrzędnych geograficznych do układu metrycznego w metodach wymagających obliczeń geometrycznych w płaszczyźnie.

Praca ma charakter implementacyjno-eksperymentalny. Oprócz przygotowania programu obejmuje przeprowadzenie benchmarków porównujących wariant listowy, wariant NumPy oraz wariant Numba dla operacji o różnej strukturze obliczeniowej. Dane testowe są generowane syntetycznie, aby możliwe było kontrolowanie liczby punktów i powtarzalne badanie czasu wykonania funkcji dla rosnących rozmiarów danych wejściowych.

Praca została podzielona na cztery główne rozdziały. Pierwszy rozdział przedstawia podstawy analizy tras GPX, strukturę formatu oraz metody obliczania i porównywania tras. Drugi rozdział omawia reprezentację danych i narzędzia obliczeniowe wykorzystane w pracy: listy Pythona, NumPy oraz Numbę. Trzeci rozdział opisuje implementację programu, parser, funkcje analityczne i skrypty benchmarkowe. Czwarty rozdział zawiera metodykę pomiarów, wyniki benchmarków oraz ich interpretację.

Rozdział 1

Teoretyczne podstawy analizy tras GPX

Rozdział przedstawia podstawy teoretyczne potrzebne do omówienia projektu oraz implementacji programu analizującego trasy zapisane w formacie GPX. Punktem wyjścia jest charakterystyka danych wejściowych, czyli śladów GPS złożonych z kolejnych punktów opisujących położenie użytkownika. Następnie omówiono strukturę formatu GPX, znaczenie układów współrzędnych oraz metody wykorzystywane do obliczania i porównywania tras.

W rozdziale uwzględniono również wybrane zagadnienia związane z wydajnością obliczeń. Przetwarzanie tras GPX wymaga wielokrotnego wykonywania operacji na sekwencjach wartości liczbowych, dlatego znaczenie mają nie tylko same metody analityczne, lecz także sposób reprezentacji danych i możliwość zapisu części obliczeń jako operacji tablicowych.

1.1 Pamięć i wydajność obliczeń

Wydajność programów przetwarzających duże zbiory danych zależy nie tylko od liczby wykonywanych operacji arytmetycznych, lecz także od sposobu organizacji dostępu do pamięci. W wielu zastosowaniach numerycznych czas wykonania może być ograniczany nie przez samą liczbę operacji procesora, ale przez koszt dostarczenia danych z pamięci operacyjnej. Z tego powodu współczesne architektury komputerowe wykorzystują hierarchię pamięci obejmującą między innymi rejestry, pamięć podręczną oraz pamięć główną [4, 5]. Efektywność wykorzystania tej hierarchii zależy między innymi od lokalności czasowej i przestrzennej odwołań do danych.[5]

Trasy zapisane w formacie GPX mogą być przetwarzane jako uporządkowane sekwencje punktów śladu `trkpt`. [3] Operacje takie jak obliczanie dystansu, sumowanie przewyższeń lub porównywanie śladu z trasą referencyjną wymagają wielokrotnego przechodzenia

po kolejnych wartościach liczbowych. Z tego powodu sposób reprezentacji danych może wpływać zarówno na koszt dostępu do pamięci, jak i na możliwość zapisu obliczeń w postaci operacji tablicowych.

1.1.1 Pamięć podręczna i hierarchia pamięci

Pamięć podręczna procesora pełni funkcję szybkiej warstwy pośredniczącej pomiędzy jednostką centralną a pamięcią operacyjną. Jej zastosowanie wynika z różnicy prędkości pomiędzy procesorem a pamięcią główną: dane używane niedawno lub potencjalnie potrzebne w najbliższym czasie mogą zostać umieszczone bliżej procesora.[4] Dzięki temu część odwołań do danych może zostać obsłużona bez każdorazowego pobierania ich z pamięci głównej.

Działanie pamięci podręcznej jest związane z pojęciami trafienia oraz chybienia. Trafienie cache występuje wtedy, gdy poszukiwane dane znajdują się już w pamięci podręcznej i mogą zostać z niej odczytane. Chybiecie oznacza natomiast konieczność pobrania danych z niższego poziomu hierarchii pamięci i umieszczenia ich w cache.[5]

Podstawą efektywnego wykorzystania pamięci podręcznej jest lokalność odwołań. Lokalność czasowa oznacza możliwość ponownego użycia danych wykorzystywanych niedawno, natomiast lokalność przestrzenna dotyczy korzystania z danych położonych blisko siebie w przestrzeni adresowej.[5] W praktyce czas wykonania programu może być ograniczany zarówno przez koszt operacji arytmetycznych, jak i przez koszt dostępu do danych. W niniejszej pracy zagadnienie to stanowi jedynie kontekst dla porównania list Pythona z jednorodnymi tablicami numerycznymi.

Istotną cechą pamięci podręcznej jest blokowy sposób przenoszenia danych między pamięcią główną a cache. Pamięć główna jest traktowana jako zbiór bloków adresowalnych komórek, natomiast pamięć podręczna składa się z wierszy odpowiadających takim blokom.[5] Odczyt jednej wartości może więc spowodować przeniesienie do pamięci podręcznej również wartości sąsiednich, co sprzyja programom przetwarzającym dane ułożone kolejno w pamięci.

Z tej perspektywy wybór struktury danych ma znaczenie nie tylko organizacyjne, lecz także wydajnościowe. Dane przechowywane w sposób ciągły, na przykład w jednorodnych tablicach liczbowych, mogą sprzyjać sekwencyjnemu odczytowi i lepiej odpowiadać sposobowi działania pamięci podręcznej [5]. Nie oznacza to jednak, że każda zmiana reprezentacji danych automatycznie prowadzi do skrócenia czasu wykonania. Efekt zależy od konkretnego algorytmu, sposobu dostępu do danych oraz narzutu związanego z używanym mechanizmem wykonania. Mniej korzystny z punktu widzenia lokalności pamięci może być model, w którym dane są reprezentowane jako wiele odrębnych obiektów powiązanych odwołaniami, ponieważ kolejne logiczne elementy przetwarzanej sekwencji nie muszą znajdować się obok siebie w pamięci. W języku Python dane programu są

reprezentowane jako obiekty posiadające tożsamość, typ i wartość.[6] Taki model jest istotny przy porównaniu list Pythona z jednorodnymi tablicami numerycznymi.

1.1.2 Równoległe wykonywanie operacji

Wydajność obliczeń numerycznych może być zwiększana nie tylko przez zmianę algorytmu, lecz także przez wykorzystanie równoległości dostępnej na poziomie sprzętu i sposobu wykonania programu. Jednym z klasycznych sposobów opisu architektur obliczeniowych jest taksonomia Flynna, dzieląca systemy według liczby strumieni instrukcji i danych. W kontekście obliczeń numerycznych szczególne znaczenie ma model SIMD, czyli *Single Instruction, Multiple Data*, w którym jedna instrukcja jest wykonywana dla wielu elementów danych.[10]

Model SIMD jest istotny wtedy, gdy ta sama operacja może zostać zastosowana do wielu wartości liczbowych. W analizie tras GPX taki charakter mają między innymi operacje na współrzędnych, wysokościach oraz różnicach między sąsiednimi punktami. Jednorodna reprezentacja danych i uporządkowany układ wartości w pamięci mogą sprzyjać przetwarzaniu wielu elementów według tego samego schematu obliczeń.

W taksonomii Flynna wyróżnia się również model MIMD, czyli *Multiple Instruction, Multiple Data*, w którym równoległość dotyczy zarówno strumieni instrukcji, jak i strumieni danych.[10] Taki model można odnieść na przykład do niezależnego przetwarzania wielu tras lub wielu plików GPX. W praktyce jednak nie każda część programu może być wykonywana równoległe. Możliwe przyspieszenie jest ograniczone przez fragmenty obliczeń, które muszą pozostać sekwencyjne, co opisuje prawo Amdahla [12].

1.1.3 Wektoryzacja

Wektoryzację można rozumieć jako sposób zapisu lub wykonania obliczeń, w którym ta sama operacja jest stosowana do wielu elementów danych zamiast do pojedynczych wartości przetwarzanych osobno. Pojęcie to może odnosić się zarówno do poziomu sprzętowego, związanego z instrukcjami SIMD, jak i do poziomu programistycznego, w którym operacje zapisywane są jako działania na całych tablicach danych.

W analizie danych GPX zapis tablicowy może być użyteczny dla operacji wykonywanych na sekwencjach wartości liczbowych, takich jak obliczanie różnic wysokości, wykonywanie funkcji trygonometrycznych dla wielu punktów lub sumowanie odległości między kolejnymi próbkami trasy. Tego typu obliczenia można często zapisać jako działania na tablicach numerycznych, ograniczając udział jawnych pętli wykonywanych bezpośrednio w kodzie Pythona.

Istotnym czynnikiem wpływającym na możliwość efektywnego wykorzystania takiego podejścia jest reprezentacja danych. Dane jednorodne, liczbowe i ułożone w sposób

sprzyjający lokalności przestrzennej łatwiej zapisać jako operacje tablicowe niż dane rozproszone jako wiele odrębnych obiektów [5]. W języku Python dane programu są reprezentowane jako obiekty posiadające tożsamość, typ i wartość [6], dlatego porównanie list Pythona z jednorodnymi tablicami numerycznymi wymaga uwzględnienia nie tylko samego algorytmu, lecz także sposobu reprezentacji danych.

Przykładem jawnego wskazania możliwości wykonania pętli w sposób wektorowy jest konstrukcja `simd` dostępna w standardzie OpenMP. Służy ona do oznaczenia pętli, której iteracje mogą zostać wykonane z użyciem instrukcji wektorowych, jeżeli spełnione są warunki poprawnościowe dotyczące między innymi zależności między iteracjami [11]. Przykład ten pokazuje, że efektywna wektoryzacja wymaga odpowiedniej struktury obliczeń, a nie tylko samego faktu pracy na danych liczbowych.

Nie każda operacja zapisana jako działanie na tablicach prowadzi automatycznie do skrócenia czasu wykonania. Efektywność takiego podejścia zależy od charakteru algorytmu, układu danych w pamięci oraz sposobu wykonania operacji przez użyte narzędzia obliczeniowe.

1.2 Dane GPS i format GPX

Dane wykorzystywane w analizie tras sportowych można traktować jako uporządkowany zapis kolejnych położenia użytkownika. W formacie GPX ślad trasy jest reprezentowany przez element `trk`, który może zawierać segmenty `trkseg`. Segmenty te składają się z kolejnych punktów śladu `trkpt`, opisujących przebieg trasy [3]. Pojedynczy punkt śladu zawiera współrzędne geograficzne zapisane w atrybutach `lat` i `lon`, a dodatkowo może zawierać wysokość `ele` oraz czas pomiaru `time` [3].

Kolejność punktów jest istotna, ponieważ pozwala obliczać odległości między sąsiednimi pozycjami, wyznaczać zmiany wysokości oraz porównywać przebieg śladu z trasą referencyjną. W dalszych częściach sekcji omówiono charakter danych GPS, strukturę pliku GPX oraz znaczenie układów współrzędnych w analizie tras.

1.2.1 Charakterystyka danych GPS

Dane GPS rejestrowane podczas aktywności sportowej mają charakter sekwencyjny. Kolejne punkty zapisu odpowiadają następującym po sobie pomiarom położenia użytkownika, dlatego ślad trasy można analizować jako uporządkowany ciąg próbek. Taka reprezentacja jest podstawą operacji polegających na porównywaniu sąsiednich punktów, między innymi przy obliczaniu dystansu oraz zmian wysokości.

Dane GPS/GNSS mogą być obarczone błędami pomiarowymi. Jednym z typowych źródeł błędów jest wielotorowość sygnału, czyli sytuacja, w której sygnał satelitarny dociera do odbiornika różnymi drogami wskutek odbić od przeszkód terenowych lub

obiektów znajdujących się w otoczeniu [13]. Z tego powodu wyniki obliczeń wykonywanych na podstawie śladów GPS należy interpretować z uwzględnieniem ograniczonej dokładności danych wejściowych.

1.2.2 Struktura pliku GPX

GPX jest formatem zapisu danych geograficznych opartym na XML. Dokumentacja GPX 1.1 opisuje plik jako hierarchię elementów XML, w której elementem głównym jest `gpx`. [3] W analizie śladów sportowych szczególne znaczenie ma część struktury związana z zapisem trasy jako śladu, obejmująca elementy `trk`, `trkseg` oraz `trkpt`.

Wykorzystywana w dalszym przetwarzaniu hierarchia elementów ma następującą postać:

- `gpx` – element główny dokumentu,
- `metadata` – metadane pliku,
- `trk` – ślad zarejestrowanej aktywności,
- `trkseg` – segment śladu,
- `trkpt` – pojedynczy punkt śladu.

Punkt śladu `trkpt` zawiera atrybuty `lat` i `lon`, określające odpowiednio szerokość i długość geograficzną punktu. Dodatkowo punkt może zawierać element `ele`, przechowujący wysokość, oraz element `time`, określający czas rejestracji próbki [3]. Brak wysokości lub czasu nie musi oznaczać braku punktu geometrycznego, ponieważ położenie punktu wynika ze współrzędnych `lat` i `lon`. W niniejszej pracy przyjęto jednak założenie pracy na kompletnych punktach wejściowych, ponieważ dane wykorzystane w benchmarkach są generowane syntetycznie i zawierają wartości `lat`, `lon`, `ele` oraz `time`. Ograniczenie to dotyczy zakresu przygotowanej implementacji, a nie samego standardu GPX.

Listing 1.1 przedstawia uproszczony fragment pliku GPX obejmujący elementy istotne dla dalszego przetwarzania: `metadata`, `trk`, `trkseg`, `trkpt`, atrybuty `lat` i `lon` oraz elementy `ele` i `time`.

Listing 1.1: Fragment przykładowego pliku GPX

```
1 <gpx creator="Garmin Connect" version="1.1"
2   xmlns="http://www.topografix.com/GPX/1/1">
3   <metadata>
4     <time>2026-03-12T18:26:20.000Z</time>
5   </metadata>
6   <trk>
7     <type>Przykładowy plik GPX</type>
```

```
8      <trkseg>
9          <trkpt lat="51.231467" lon="22.50682">
10              <ele>124.7</ele>
11              <time>2026-03-12T18:26:27.000Z</time>
12          </trkpt>
13          <trkpt lat="51.231454" lon="22.506844">
14              <ele>124</ele>
15              <time>2026-03-12T18:26:28.000Z</time>
16          </trkpt>
17          <trkpt lat="51.231439" lon="22.506866">
18              <ele>124</ele>
19              <time>2026-03-12T18:26:29.000Z</time>
20          </trkpt>
21      </trkseg>
22  </trk>
23 </gpx>
```

Przedstawiony fragment pokazuje sposób zapisu pojedynczego punktu śladu. Atrybuty `lat` i `lon` określają położenie punktu, element `ele` przechowuje wysokość, natomiast `time` zawiera czas rejestracji próbek. Na podstawie kolejnych elementów `trkpt` można utworzyć sekwencję punktów trasy stanowiącą dane wejściowe dla operacji analitycznych.

W plikach GPX mogą występować również rozszerzenia oraz dodatkowe informacje wykraczające poza podstawowe elementy punktów śladu.[3] W obliczeniach wykonywanych w przygotowanym programie wartości takie jak dystans czy suma przewyższeń nie są jednak odczytywane jako gotowe metryki z pliku, lecz wyznaczane ponownie na podstawie kolejnych punktów trasy. Pozwala to porównywać warianty implementacyjne wykonujące te same operacje na tych samych danych wejściowych.

Format GPX, jako format tekstowy oparty na XML, wymaga wydobycia danych ze struktury dokumentu przed rozpoczęciem właściwych obliczeń.[3] Dlatego w implementacji konieczne jest oddzielenie etapu parsowania pliku od etapu obliczeń wykonywanych na przygotowanej reprezentacji danych.

1.2.3 Układy współrzędnych w analizie tras

Zapis pozycji w danych GPS oraz w formacie GPX opiera się na globalnym układzie odniesienia WGS84. W tym układzie pozycja jest opisywana za pomocą współrzędnych geograficznych: szerokości i długości geograficznej. Dokumentacja GPX 1.1 wskazuje, że współrzędne zapisane w pliku GPX odnoszą się do układu WGS84, natomiast wartości takie jak wysokość są wyrażane w jednostkach metrycznych [14, 3].

Konsekwencją takiego sposobu zapisu jest konieczność rozróżnienia obliczeń wykonywanych na współrzędnych geograficznych i obliczeń wymagających układu metrycznego. Szerokość i długość geograficzna są wyrażane w stopniach, dlatego nie powinny być

traktowane bezpośrednio jako współrzędne kartezjańskie w metrach. Przy obliczaniu odległości między punktami na powierzchni Ziemi można zastosować wzór uwzględniający geograficzny charakter współrzędnych, natomiast operacje geometryczne wykonywane w płaszczyźnie wymagają reprezentacji, w której jednostką odległości jest metr.

Przekształcenie współrzędnych geograficznych na płaszczyznę wymaga zastosowania odwzorowania kartograficznego. Ponieważ każde odwzorowanie wiąże się z określonym rodzajem i rozkładem zniekształceń, dobór układu płaskiego powinien być związany z obszarem opracowania oraz charakterem wykonywanych analiz.[15] Nie istnieje więc jeden uniwersalny układ płaski, który byłby jednakowo odpowiedni dla wszystkich obszarów i wszystkich zastosowań.

W implementacji przyjęto, że analizowane trasy znajdują się na obszarze Polski. Dla operacji geometrycznych wykonywanych w płaszczyźnie, takich jak porównywanie punktu śladu z segmentem trasy referencyjnej, zastosowano układ współrzędnych płaskich PL-1992. W bazie EPSG układ ten występuje jako ETRS89 / PL-1992 z kodem EPSG:2180, typem *Projected CRS*, zakresem stosowania obejmującym Polskę oraz dwuwymiarowym kartezjańskim układem współrzędnych wyrażonym w metrach [16]. Dobór tego układu wynika z ograniczenia analiz do obszaru Polski oraz z potrzeby wykonywania lokalnych obliczeń geometrycznych w jednostkach metrycznych.

1.3 Metody analizy i porównywania tras

Analiza tras sportowych obejmuje zarówno wyznaczanie podstawowych parametrów pojedynczego śladu, jak i ocenę relacji między śladem zawodnika a trasą referencyjną. Podobne funkcje występują także w praktycznych aplikacjach do planowania i analizy aktywności, takich jak Ride with GPS, gdzie użytkownik może analizować między innymi przebieg trasy, profil wysokościowy, podjazdy, zjazdy oraz informacje związane z planowaniem przejazdu [1, 2]. Przykład ten nie stanowi podstawy definicji metod obliczeniowych użytych w pracy, lecz pokazuje, że automatyczne wyznaczanie metryk trasy ma zastosowanie w istniejących narzędziach użytkowych.

W przypadku zawodów sportowych istotne może być jednak nie tylko opisanie jednej trasy, lecz także sprawdzenie zgodności rzeczywistego śladu zawodnika z przebiegiem trasy referencyjnej. Takie porównanie pozwala ocenić, czy zawodnik poruszał się w pobliżu wyznaczonej trasy, gdzie wystąpiły największe odchylenia oraz jaki udział punktów śladu mieścił się w przyjętych progach tolerancji. Wymaga to zastosowania innych operacji niż samo obliczenie dystansu lub przewyższeń, ponieważ konieczne jest analizowanie relacji pomiędzy punktami śladu a geometrią trasy wzorcowej.

Metody wykorzystywane do analizy tras GPX można więc podzielić według rodzaju danych, na których operują, oraz według kosztu obliczeniowego. Najprostsze operacje

dotyczą pojedynczego śladu i polegają na przejściu po kolejnych punktach trasy. Do tej grupy należą obliczanie dystansu oraz wyznaczanie sumy podejść i zejść. Ich wynik zależy od relacji między sąsiednimi próbkami, dlatego naturalnym modelem obliczeń jest sekwencyjne przetwarzanie uporządkowanego ciągu punktów.

Bardziej złożone są metody porównujące ślad zawodnika z trasą referencyjną. Wymagają one analizy relacji między dwoma przebiegami trasy, na przykład przez wyznaczanie odległości punktów śladu od segmentów linii referencyjnej albo przez porównanie dwóch sekwencji punktów jako całości. W tym drugim przypadku wykorzystana może być dyskretna odległość Frécheta, która uwzględnia nie tylko położenie punktów, lecz także ich kolejność w obu porównywanych sekwencjach.[22]

Podział metod ma znaczenie również dla sposobu implementacji. Operacje wykonywane na jednej sekwencji danych, takie jak obliczanie różnic wysokości albo odległości między sąsiednimi punktami, łatwiej zapisać jako działania na tablicach numerycznych. Wynika to z faktu, że kolejne wartości mogą być przetwarzane według tego samego schematu obliczeń. Metody porównujące dwa przebiegi trasy, zwłaszcza oparte na relacjach punkt–segment lub programowaniu dynamicznym, mają bardziej złożoną strukturę obliczeń i nie zawsze poddają się prostej wektoryzacji.

1.3.1 Obliczanie dystansu i sum podejść i zejść

Obliczanie dystansu i sum przewyższeń służy do podstawowego opisu pojedynczej trasy lub śladu zawodnika. Wynikiem tej grupy operacji są trzy metryki: długość trasy, suma podejść oraz suma zejść. W kontekście aktywności sportowej wartości te pozwalają określić długość biegu lub przejazdu, profil wysokościowy oraz trudność wysiłkową trasy. Mogą być wykorzystywane zarówno przy opisie trasy referencyjnej, jak i przy analizie śladu zarejestrowanego przez zawodnika.

Z technicznego punktu widzenia ślad GPX jest traktowany jako uporządkowana sekwencja punktów. Dystans można więc wyznaczyć jako sumę odległości między kolejnymi punktami, natomiast sumy przewyższeń jako sumy dodatnich i ujemnych różnic wysokości między sąsiednimi próbkami. W badaniach dotyczących tras trailowych i górskich odległość między sąsiednimi punktami pliku GPX obliczano na podstawie współrzędnych geograficznych, traktując ją jako odległość poziomą, natomiast pionowe przemieszczenie między punktami analizowano oddzielnie jako składnik sumy podejść lub zejść [17].

Odległość między kolejnymi punktami może być obliczana z użyciem wzoru Haversine’a, który pozwala wyznaczyć przybliżoną odległość po wielkim kole między dwoma punktami opisanymi szerokością i długością geograficzną [18]. Wzór ten zakłada model sferyczny Ziemi, dlatego wynik należy traktować jako przybliżenie wystarczające dla przyjętego zakresu analiz tras GPX. Dla punktów o szerokościach geograficznych ϕ_1 , ϕ_2 oraz długościach geograficznych λ_1 , λ_2 , przyjmuje się:

$$\Delta\phi = \phi_2 - \phi_1,$$

$$\Delta\lambda = \lambda_2 - \lambda_1.$$

Następnie obliczana jest wartość pomocnicza:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos\phi_1 \cos\phi_2 \sin^2\left(\frac{\Delta\lambda}{2}\right).$$

Odległość między punktami jest wyznaczana jako:

$$d = 2R \operatorname{atan2}\left(\sqrt{a}, \sqrt{1-a}\right),$$

gdzie R oznacza przyjęty promień Ziemi, a d oznacza odległość między punktami po powierzchni sfery. Przed podstawieniem do wzoru wartości szerokości i długości geograficznej muszą zostać wyrażone w radianach.[18]

Długość całej trasy obliczana jest jako suma odległości dla wszystkich par sąsiednich punktów śladu:

$$D = \sum_{i=1}^{n-1} d(p_i, p_{i+1}),$$

gdzie n oznacza liczbę punktów trasy, a $d(p_i, p_{i+1})$ oznacza odległość między punktem p_i oraz punktem p_{i+1} .

Sumy podejść i zejść są wyznaczane na podstawie różnic wysokości między kolejnymi punktami. Dla dwóch sąsiednich punktów różnicę wysokości można zapisać jako:

$$\Delta h_i = h_{i+1} - h_i.$$

Suma podejść obejmuje dodatnie różnice wysokości:

$$H^+ = \sum_{i=1}^{n-1} \max(\Delta h_i, 0),$$

natomiast suma zejść obejmuje wartości bezwzględne różnic ujemnych:

$$H^- = \sum_{i=1}^{n-1} \max(-\Delta h_i, 0).$$

Wyznaczone wartości dystansu oraz przewyższeń zależą od jakości i rozdzielczości danych wejściowych. W badaniach nad trasami trailowymi i górskimi wskazano, że rozdzielczość przestrzenna zapisu trasy może wpływać na obliczany dystans oraz skumulowane przewyższenia [17]. Dodatkowo błąd pomiaru GPS może prowadzić do zniekształcenia długości trajektorii, w tym do jej systematycznego zawyżania w określonych warunkach.[19]

Z punktu widzenia kosztu obliczeniowego obliczanie dystansu, sumy podejść i sumy zejść wymaga jednokrotnego przejścia po sekwencji punktów. Każdy punkt, poza pierwszym,

jest porównywany z punktem poprzednim, dlatego liczba podstawowych operacji rośnie liniowo wraz z liczbą punktów trasy. Oznacza to, że przy podobnym koszcie przetworzenia jednej pary sąsiednich punktów wzrost liczby punktów powinien prowadzić do zbliżonego liniowego wzrostu czasu wykonania funkcji. W praktyce dokładny czas zależy jednak również od sposobu reprezentacji danych, kosztu wywołań funkcji matematycznych oraz mechanizmu wykonania kodu.

Struktura tych obliczeń sprzyja zapisowi w postaci operacji tablicowych. Różnice wysokości oraz różnice współrzędnych między sąsiednimi punktami mogą być wyznaczane dla wielu elementów sekwencji według tego samego schematu. Nie oznacza to automatycznie skrócenia czasu wykonania w każdym przypadku, ale sprawia, że obliczanie dystansu oraz sum przewyższeń jest naturalnym przykładem operacji podatnych na wektoryzację.

1.3.2 Porównywanie trasy z trasą referencyjną

Porównywanie trasy z trasą referencyjną służy do oceny, jak blisko ślad zawodnika przebiega względem wcześniej przyjętej trasy wzorcowej. W kontekście zawodów sportowych taka metoda może być używana jako narzędzie pomocnicze do kontroli zgodności śladu z trasą wyznaczoną przez organizatora. Wynikiem nie jest pojedyncza decyzja o poprawności lub niepoprawności przejazdu albo biegu, lecz zestaw wartości opisujących lokalne odchylenia od trasy referencyjnej.

W tej metodzie dla każdego punktu śladu zawodnika wyznaczana jest minimalna odległość do trasy referencyjnej. Na podstawie zbioru takich odległości można obliczyć średnie odchylenie, największe odchylenie oraz udział punktów mieszczących się w przyjętych progach tolerancji. Takie wartości mogą wskazywać fragmenty wymagające dodatkowej analizy, na przykład z powodu oddalenia się zawodnika od przebiegu wzorcowego.

Technicznie metoda wymaga przyjęcia geometrycznej reprezentacji trasy referencyjnej. W standardzie OGC Simple Feature Access geometria typu `LineString` jest opisywana jako krzywa z interpolacją liniową między punktami, a każda kolejna para punktów definiuje segment linii [20]. W pracy przyjęto analogiczny model: trasa referencyjna jest traktowana jako linia łamana utworzona z uporządkowanej sekwencji punktów.

Sąsiednie punkty trasy referencyjnej tworzą segmenty, względem których oceniane jest położenie punktów śladu zawodnika. Lokalne porównanie przebiegu tras można wówczas sprowadzić do obliczenia odległości punktu od segmentu trasy referencyjnej.

Dla odcinka AB oraz punktu C najbliższy punkt na odcinku można wyznaczyć przez rzutowanie punktu C na prostą przechodzącą przez A i B , a następnie sprawdzenie, czy punkt rzutu znajduje się wewnątrz odcinka. Jeżeli rzut znajduje się na odcinku, to on jest punktem najbliższym względem C . Jeżeli rzut wypada poza odcinkiem, najbliższym punktem jest jeden z końców odcinka [21].

Niech A , B oraz C oznaczają punkty w układzie płaskim. Punkt na prostej przecho-

dzącej przez końce odcinka można zapisać parametrycznie jako:

$$P(t) = A + t(B - A).$$

Wartość parametru t można obliczyć z wykorzystaniem iloczynu skalarnego:

$$t = \frac{(C - A) \cdot (B - A)}{\|B - A\|^2}.$$

Parametr t określa położenie rzutu punktu C względem odcinka AB . Jeżeli mianownik $\|B - A\|^2$ jest równy zero, segment ma zerową długość i odległość można sprowadzić do odległości punktu C od punktu A . W przeciwnym przypadku wyróżnia się trzy sytuacje:

- dla $t < 0$ najbliższym punktem odcinka jest punkt A ,
- dla $0 \leq t \leq 1$ najbliższym punktem jest rzut $P(t)$,
- dla $t > 1$ najbliższym punktem odcinka jest punkt B .

Odległość punktu śladu od segmentu może zostać obliczona jako odległość euklidesowa między punktem C a wyznaczonym punktem najbliższym. Jeżeli punkt śladu jest porównywany z wieloma segmentami trasy referencyjnej, minimalna z otrzymanych odległości może być traktowana jako odległość od przebiegu trasy wzorcowej.

Przy prostym podejściu obliczeniowym każdy punkt śladu zawodnika jest porównywany ze wszystkimi segmentami trasy referencyjnej. Dla m punktów śladu zawodnika i s segmentów trasy referencyjnej konieczne jest sprawdzenie $m \cdot s$ relacji punkt-segment, dlatego złożoność czasowa metody wynosi $O(m \cdot s)$. Jeżeli liczba punktów w obu trasach rośnie w podobnym tempie, a liczba segmentów trasy referencyjnej jest zbliżona do liczby jej punktów, czas wykonania może rosnać w przybliżeniu kwadratowo względem liczby punktów. Z tego powodu metoda ta jest znacznie bardziej kosztowna od operacji liniowych wykonywanych na pojedynczym śladzie.

1.3.3 Segmentowe porównanie prędkości

Segmentowe porównanie prędkości służy do porównania tempa pokonywania kolejnych fragmentów trasy referencyjnej i śladu zawodnika. W odróżnieniu od metod opartych wyłącznie na geometrii, metoda ta wykorzystuje również znaczniki czasu zapisane w pliku GPX. Dzięki temu można sprawdzić, czy na odpowiadających sobie odcinkach dystansowych tempo śladu zawodnika było wyższe, niższe lub zbliżone do tempa referencyjnego.

W kontekście aktywności sportowych taka informacja może uzupełniać analizę zgodności geometrycznej. Sama bliskość śladu do trasy referencyjnej nie mówi jeszcze, jak zmieniało się tempo na kolejnych fragmentach. Segmentowe porównanie prędkości może więc wskazać odcinki, na których tempo wyraźnie odbiega od tempa referencyjnego. Wynik

tej metody nie powinien być jednak traktowany jako samodzielne kryterium oceny poprawności biegu lub przejazdu, ponieważ zależy od jakości danych czasowych, charakteru trasy referencyjnej oraz sposobu rejestracji śladu.

W analizie trajektorii ruchu obiektu może być traktowany jako uporządkowana sekwencja punktów, w której każdy punkt zawiera informację przestrzenną oraz czasową [29]. Metoda zastosowana w pracy polega na podziale trasy referencyjnej i śladu zawodnika na segmenty dystansowe. Segmentacja trajektorii jest jedną z technik przetwarzania danych ruchu, ponieważ pozwala analizować wybrane fragmenty przebiegu, a nie tylko całą trajektorię jako jedną sekwencję [29].

W pracy przyjęto segment o długości 500 m. Wartość tę potraktowano jako kompromis między szczegółowością analizy a stabilnością wyniku. Krótsze segmenty zwiększałyby wrażliwość wyniku na lokalne zakłócenia pomiaru i nierównomierne próbkowanie punktów, natomiast dłuższe słabiej pokazywałyby lokalne zmiany tempa.

W pierwszym kroku dla każdej z porównywanych tras wyznaczany jest dystans narastający. Oznacza to, że każdemu punktowi śladu można przypisać odległość od początku trasy. Następnie tworzony jest zbiór granic segmentów:

$$0, 500, 1000, 1500, \dots, D,$$

gdzie D oznacza wspólny analizowany dystans obu tras. W implementacji jako D przyjęto mniejszą z długości trasy referencyjnej oraz śladu zawodnika. Dzięki temu porównywane są tylko te fragmenty, które występują w obu trasach. Należy podkreślić, że segmenty są definiowane według dystansu narastającego od początku każdego śladu. Metoda nie wykonuje przestrzennego dopasowania punktów zawodnika do geometrii trasy referencyjnej. Oznacza to, że porównywane są odpowiadające sobie przedziały dystansowe, na przykład pierwszy i drugi odcinek 500 m, a niekoniecznie dokładnie te same fragmenty trasy w sensie geometrycznym. Przy danych syntetycznych z niewielkimi odchyleniami jest to wystarczające do porównania wariantów implementacyjnych, natomiast dla rzeczywistych śladów z dużymi odstępstwami wynik należałoby interpretować ostrożnie.

Uwzględniany jest również ostatni fragment trasy, nawet jeśli jest krótszy niż standardowy segment 500 m. Aby krótszy końcowy odcinek nie miał takiego samego wpływu na wynik jak pełny segment, wynik końcowy jest liczony jako średnia ważona długością segmentów.

Granica segmentu nie musi przypadać dokładnie w miejscu zarejestrowanego punktu GPX. Dane opisujące ruch są rejestrowane jako dyskretne próbki położenia w czasie, dlatego odtworzenie wartości pomiędzy kolejnymi próbkami wymaga przyjęcia modelu interpolacji. Jednym z podstawowych podejść stosowanych w analizie trajektorii obiektów ruchomych jest interpolacja liniowa między sąsiednimi próbkami [30]. Jeżeli szukana odległość znajduje się pomiędzy dwoma sąsiednimi punktami śladu, czas osiągnięcia tej odległości można oszacować właśnie przez interpolację liniową. Dla odległości d , znajdującej

się między dystansami narastającymi s_{i-1} oraz s_i , czas można zapisać jako:

$$t(d) = t_{i-1} + \frac{d - s_{i-1}}{s_i - s_{i-1}} (t_i - t_{i-1}).$$

Wzór ten zakłada liniową zmianę położenia między sąsiednimi próbkami śladu. W praktyce pozwala to wyznaczyć czas osiągnięcia granicy segmentu nawet wtedy, gdy granica ta nie pokrywa się z żadnym punktem zapisanym w pliku GPX.

Dla segmentu ograniczonego odległościami d_{j-1} oraz d_j czas jego pokonania wynosi:

$$\Delta t_j = t(d_j) - t(d_{j-1}).$$

Prędkość jest informacją pochodną względem zarejestrowanej trajektorii, ponieważ wynika z relacji między przebyтым dystansem a czasem ruchu [30]. Dla pojedynczego segmentu średnią prędkość można zapisać jako:

$$v_j = \frac{L_j}{\Delta t_j},$$

gdzie $L_j = d_j - d_{j-1}$ oznacza długość segmentu. Dla większości segmentów wartość L_j wynosi 500 m, natomiast ostatni segment może być krótszy.

W celu porównania śladu zawodnika z trasą referencyjną dla każdego segmentu obliczany jest stosunek średniej prędkości wyznaczonej dla śladu zawodnika do średniej prędkości wyznaczonej dla śladu referencyjnego:

$$r_j = \frac{v_{zaw,j}}{v_{ref,j}}.$$

Ponieważ w danym segmencie porównywana jest ta sama długość odcinka dla referencji i zawodnika, zależność tę można uprościć do ilorazu czasów:

$$r_j = \frac{L_j / \Delta t_{zaw,j}}{L_j / \Delta t_{ref,j}} = \frac{\Delta t_{ref,j}}{\Delta t_{zaw,j}}.$$

Wartość $r_j > 1$ oznacza, że zawodnik pokonał dany segment szybciej niż ślad referencyjny. Wartość $r_j < 1$ oznacza wolniejsze tempo względem referencji.

Wynik końcowy metody jest liczony jako średni stosunek prędkości ważony długością segmentów:

$$R = \frac{\sum_{j=1}^k r_j L_j}{\sum_{j=1}^k L_j},$$

gdzie k oznacza liczbę analizowanych segmentów. Ważenie długością segmentu jest potrzebne ze względu na możliwość wystąpienia krótszego segmentu końcowego. Dzięki temu ostatni fragment trasy jest uwzględniany, ale jego wpływ na wynik jest proporcjonalny do rzeczywistej długości odcinka.

Koszt obliczeniowy tej metody jest zbliżony do liniowego względem liczby punktów obu tras. Dla każdej trasy konieczne jest przejście po kolejnych punktach w celu wyznaczenia dystansu narastającego oraz czasów odpowiadających granicom segmentów.

Liczba segmentów zależy od długości trasy i przyjętej długości segmentu, ale dla ustalonej struktury danych nie wymaga porównywania każdego punktu jednej trasy z każdym punktem drugiej. Z tego powodu wzrost liczby punktów powinien prowadzić do znacznie łagodniejszego wzrostu czasu wykonania niż w metodach o charakterze kwadratowym.

W badaniach nad wykorzystaniem GPS w sporcie podkreśla się, że na jakość i interpretację danych wpływają między innymi częstotliwość próbkowania, sposób umieszczenia urządzenia, warunki odbioru sygnału, filtracja danych oraz sposób raportowania wyników [31]. Dodatkowo błędy pomiaru GPS mogą wpływać na obliczane długości trajektorii i metryki pochodne, dlatego wyniki analizy prędkości należy interpretować z uwzględnieniem jakości danych wejściowych [19]. W niniejszej pracy segmentowe porównanie prędkości jest więc traktowane jako pomocniczy wskaźnik różnic tempa, a nie jako mechanizm automatycznego wykrywania nieprawidłowości.

1.3.4 Dyskretna odległość Fréchet

Dyskretna odległość Fréchet służy do porównania dwóch tras jako uporządkowanych sekwencji punktów. W odróżnieniu od lokalnego sprawdzania odległości pojedynczych punktów od segmentów, metoda ta ocenia podobieństwo przebiegu dwóch krzywych jako całości. Ma to znaczenie w analizie śladów GPX, ponieważ trasa jest zapisem kolejnych położeń, a nie nieuporządkowanym zbiorem punktów.

W kontekście zawodów sportowych dyskretna odległość Fréchet może być dodatkową miarą podobieństwa całego śladu do trasy referencyjnej, ponieważ uwzględnia kolejność punktów. Ze względu na większy koszt obliczeniowy metoda ta lepiej nadaje się do krótszych lub przefiltrowanych tras.

Formalnie dyskretna odległość Fréchet jest miarą podobieństwa krzywych łamanych reprezentowanych przez skończone sekwencje punktów. Uwzględnia ona uporządkowanie punktów na obu porównywanych krzywych [22].

Eiter i Mannila definiują dyskretną odległość Fréchet dla dwóch krzywych łamanych P oraz Q , reprezentowanych przez sekwencje punktów:

$$\sigma(P) = (u_1, \dots, u_p),$$

$$\sigma(Q) = (v_1, \dots, v_q).$$

Podstawą definicji jest uporządkowane parowanie punktów obu sekwencji, określane jako *coupling* [22]. Parowanie rozpoczyna się od pierwszych punktów obu sekwencji i kończy na ich ostatnich punktach. W kolejnych krokach można przejść do następnego punktu pierwszej krzywej, do następnego punktu drugiej krzywej albo do następnych punktów obu krzywych jednocześnie. Dzięki temu zachowana zostaje kolejność punktów, a jednocześnie możliwe jest porównywanie sekwencji o różnej liczbie próbek.

Długość danego parowania jest określana jako największa odległość pomiędzy punktami występującymi w sparowanych parach. Dyskretna odległość Frécheta odpowiada najmniejszej możliwej wartości tej największej odległości, rozpatrywanej po wszystkich dopuszczalnych uporządkowanych parowaniach punktów obu krzywych [22]. Wynik tej miary nie zależy więc od pojedynczego lokalnego dopasowania, lecz od najlepszego możliwego przejścia po obu sekwencjach z zachowaniem ich kolejności.

Obliczenie dyskretnej odległości Frécheta można przeprowadzić metodą programowania dynamicznego. Dla każdej pary punktów u_i oraz v_j wyznaczana jest wartość opisująca najlepsze dopasowanie początkowych fragmentów obu sekwencji kończących się odpowiednio w tych punktach. Wartość ta zależy od wcześniej obliczonych sąsiednich stanów oraz od odległości $d(u_i, v_j)$ między aktualnie porównywanymi punktami [22].

Przy indeksowaniu od 1 warunki brzegowe można zapisać następująco:

$$\begin{aligned} c(1, 1) &= d(u_1, v_1), \\ c(i, 1) &= \max\{c(i-1, 1), d(u_i, v_1)\}, \\ c(1, j) &= \max\{c(1, j-1), d(u_1, v_j)\}. \end{aligned}$$

Dla przypadku $i > 1$ oraz $j > 1$ zależność rekurencyjną można zapisać następująco:

$$c(i, j) = \max(\min\{c(i-1, j), c(i-1, j-1), c(i, j-1)\}, d(u_i, v_j)).$$

Wartość $c(i, j)$ oznacza najmniejszą możliwą wartość maksymalnej odległości potrzebnej do dopasowania początkowych fragmentów obu sekwencji kończących się w punktach u_i oraz v_j . Końcową wartością dyskretnej odległości Frécheta jest $c(p, q)$, czyli wartość odpowiadająca dopasowaniu pełnych sekwencji punktów.

Dla sekwencji o długościach p oraz q algorytm wymaga rozpatrzenia par punktów obu krzywych, dlatego jego złożoność czasowa wynosi $O(pq)$ [22]. Jeżeli obie sekwencje mają zbliżoną długość n , koszt czasowy rośnie w przybliżeniu kwadratowo, czyli jak $O(n^2)$. Oznacza to, że dwukrotne zwiększenie liczby punktów może prowadzić do około czterokrotnego wzrostu liczby aktualizowanych stanów. Jeżeli implementacja przechowuje pełną macierz programowania dynamicznego, złożoność pamięciowa również wynosi $O(pq)$. Ma to znaczenie praktyczne przy porównywaniu długich tras GPX, ponieważ liczba komórek macierzy rośnie wraz z iloczynem liczby punktów obu śladów.

W analizie śladów GPX dyskretna odległość Frécheta może służyć do porównania trasy zawodnika z trasą referencyjną na poziomie całego przebiegu trasy. Jej koszt obliczeniowy jest jednak większy niż w przypadku operacji liniowych, ponieważ wymaga rozpatrzenia relacji między punktami obu porównywanych sekwencji.

Rozdział 2

Reprezentacja danych i narzędzia obliczeniowe w Pythonie

Rozdział przedstawia zagadnienia związane z reprezentacją danych liczbowych oraz sposobem wykonywania obliczeń w języku Python. Są one istotne z punktu widzenia implementacji metod analizy tras GPX, ponieważ te same metryki mogą zostać obliczone przy użyciu różnych struktur danych i różnych mechanizmów wykonania kodu. Różnice między wariantami implementacyjnymi nie dotyczą definicji obliczanych miar, lecz sposobu przechowywania danych, organizacji pętli oraz wykorzystania bibliotek numerycznych.

W dalszej części rozdziału omówiono trzy podejścia zastosowane w projekcie: wariant oparty na wbudowanych strukturach języka Python, wariant wykorzystujący tablice NumPy oraz wariant kompilowany z użyciem biblioteki Numba. Wariant listowy pełni rolę punktu odniesienia, ponieważ pozwala zapisać algorytmy w sposób bezpośredni i czytelny. NumPy umożliwia przechowywanie danych w jednorodnych tablicach numerycznych oraz zapis części operacji w postaci działań tablicowych. Numba pozwala natomiast kompilować wybrane funkcje numeryczne, o ile ich struktura oraz użyte typy danych są zgodne z ograniczeniami kompilatora JIT. Ma to znaczenie zwłaszcza dla metod, w których zachowanie jawnych pętli jest prostsze lub bardziej naturalne niż próba pełnej wektoryzacji.

2.1 Python i reprezentacja danych liczbowych

Po odczytaniu pliku GPX dane trasy nie są już przetwarzane jako dokument XML, lecz jako sekwencje wartości liczbowych wykorzystywane w dalszych operacjach analitycznych. Do najważniejszych wartości używanych w programie należą szerokości geograficzne, długości geograficzne, wysokości oraz znaczniki czasu. Współrzędne i wysokości są wykorzystywane w większości funkcji analitycznych, natomiast czas pomiaru jest używany w segmentowym porównaniu prędkości. W przygotowanej implementacji dane te są po par-

sowaniu przechowywane jako osobne sekwencje składowych punktu, a nie jako pojedyncza lista złożonych obiektów punktów.

Język Python umożliwia zapis algorytmów w sposób czytelny i bezpośredni, jednak jego ogólny model danych różni się od modelu jednorodnych tablic liczbowych wykorzystywanych w obliczeniach numerycznych. Zgodnie z dokumentacją języka Python wszystkie dane w programie są reprezentowane przez obiekty lub relacje między obiektami, a każdy obiekt posiada tożsamość, typ i wartość [6]. Ma to znaczenie przy przetwarzaniu dużych sekwencji liczb, ponieważ operacje wykonywane na obiektach Pythona mogą wiązać się z innym narzutem wykonania niż operacje wykonywane na jednorodnych tablicach numerycznych [23].

W analizie tras GPX szczególnie istotne są operacje powtarzane dla wielu kolejnych punktów śladu. Obliczanie dystansu wymaga przechodzenia po parach sąsiednich punktów, suma podejść i zejść wynika z różnic wysokości między kolejnymi próbkami, natomiast porównywanie śladu zawodnika z trasą referencyjną wymaga analizy relacji między punktami jednej trasy a segmentami drugiej trasy. Są to operacje o różnej strukturze obliczeniowej, dlatego nie wszystkie w takim samym stopniu nadają się do zapisu w postaci działań tablicowych.

Z tego powodu w projekcie zastosowano trzy warianty implementacyjne. Wariant bazowy wykorzystuje listy Pythona i stanowi punkt odniesienia dla pozostałych rozwiązań. Wariant NumPy przechowuje dane w jednorodnych tablicach numerycznych i pozwala zapisać część obliczeń jako działania na całych sekwencjach wartości. Wariant Numba korzysta z tablic NumPy jako danych wejściowych, ale kluczowe pętle numeryczne są wykonywane w funkcjach kompilowanych z użyciem mechanizmu JIT. Porównanie tych podejść pozwala ocenić wpływ reprezentacji danych i mechanizmu wykonania kodu na czas działania tych samych metod analizy tras GPX.

2.1.1 Listy Pythona jako wariant bazowy

Jednym z podstawowych sposobów reprezentowania sekwencji danych w języku Python jest lista. W kontekście analizy tras GPX jest to rozwiązanie naturalne z punktu widzenia czytelności kodu, ponieważ ślad trasy również ma postać uporządkowanej sekwencji pomiarów. W przygotowanej implementacji dane odczytane z pliku GPX nie są jednak przechowywane jako jedna lista złożonych obiektów punktów, lecz jako osobne listy odpowiadające poszczególnym składowym śladu: szerokościom geograficznym, długościom geograficznym oraz wysokościami. Taki układ ułatwia bezpośrednie przechodzenie po wybranych wartościach potrzebnych w danej operacji.

Wariant listowy należy rozpatrywać w kontekście obiektowego modelu danych Pythona. Zgodnie z dokumentacją języka każdy obiekt posiada tożsamość, typ oraz wartość [6]. W praktyce oznacza to, że wartości liczbowe znajdujące się na liście są obsługiwane jako

obiekty Pythona, a nie jako elementy jednorodnego bufora liczbowego porównywalnego z tablicą typu `float64`.

W standardowej implementacji CPython lista jest strukturą zmiennej długości opartą na tablicy referencji do obiektów, a nie na tablicy przechowującej bezpośrednio wartości liczbowe [7]. Dzięki temu lista jest elastyczna i pozwala łatwo dodawać kolejne elementy, na przykład podczas parsowania punktów śladu GPX. Jednocześnie taka reprezentacja różni się od jednorodnych tablic numerycznych, w których elementy mają ustalony typ i są przeznaczone bezpośrednio do obliczeń na danych liczbowych.

Wariant listowy pełni rolę bazową, ponieważ zapisuje algorytmy wprost, bez użycia operacji tablicowych ani kompilacji JIT. Obliczanie dystansu można w takim wariantcie zrealizować przez przechodzenie po parach sąsiednich wartości szerokości i długości geograficznej. Suma podejść i zejść może być wyznaczana przez porównywanie kolejnych wartości wysokości, natomiast porównywanie śladu z trasą referencyjną przez jawne sprawdzanie relacji między punktami i segmentami.

W części geometrycznej programu, po przekształceniu współrzędnych do układu metrycznego, wariant listowy nadal korzysta z wbudowanych struktur Pythona. Istotne jest tutaj przede wszystkim zachowanie bezpośredniego, sekwencyjnego sposobu zapisu algorytmu. Wariant listowy różni się więc od wariantu NumPy nie definicją wykonywanych obliczeń, lecz sposobem organizacji danych i realizacji pętli.

Wariant listowy jest więc czytelny i elastyczny, ale nie tworzy jednorodnego bufora wartości liczbowych porównywalnego z tablicą NumPy typu `float64`. Dzięki temu może pełnić rolę punktu odniesienia dla wariantów wykorzystujących tablice NumPy oraz kompilację JIT.

2.1.2 NumPy i tablice numeryczne

NumPy jest jedną z podstawowych bibliotek wykorzystywanych w języku Python do obliczeń numerycznych i programowania tablicowego. Jej centralną strukturą danych jest `ndarray`, czyli wielowymiarowa tablica elementów tego samego typu i rozmiaru. Typ elementów przechowywanych w tablicy jest określany przez obiekt `dtype` związany z daną tablicą [25]. Taki model różni się od wbudowanych list Pythona, które przechowują odwołania do obiektów i mogą zawierać elementy różnych typów.

Znaczenie NumPy w obliczeniach naukowych wynika z możliwości organizowania danych w postaci tablic oraz wykonywania wielu operacji matematycznych bez konieczności zapisywania ich jako jawnych pętli bezpośrednio w kodzie Pythona. Harris i in. opisują NumPy jako podstawową bibliotekę programowania tablicowego w ekosystemie naukowego Pythona [24]. W kontekście niniejszej pracy ma to znaczenie, ponieważ dane odczytane z plików GPX mają charakter liczbowy i sekwencyjny: obejmują między innymi szerokości geograficzne, długości geograficzne oraz wysokości punktów śladu.

W przygotowanej implementacji wariant NumPy wykorzystuje osobne tablice dla wybranych składowych śladu, takich jak współrzędne i wysokości. Taki układ danych pozwala wykonywać operacje na całych sekwencjach jednej składowej, na przykład na wszystkich wysokościach albo na wszystkich szerokościach geograficznych.

W wariacie NumPy wykorzystano między innymi funkcje uniwersalne, określane jako **ufuncs**. Są to funkcje działające element po elemencie na tablicach **ndarray** i wspierające mechanizmy takie jak broadcasting [26]. W kontekście analizy tras pozwala to zapisać wybrane działania, na przykład funkcje trygonometryczne albo odejmowanie sąsiednich wartości, jako operacje na całych tablicach.

Podobnie obliczanie sumy podejść i zejść można zapisać jako operację na przesuniętych fragmentach tablicy wysokości. Różnice wysokości są obliczane dla całej sekwencji, a następnie filtrowane przy użyciu masek logicznych. Dodatkowo różnice składają się na sumę podejść, natomiast wartości ujemne, po zmianie znaku lub użyciu wartości bezwzględnej, tworzą sumę zejść. Ten typ operacji dobrze odpowiada modelowi tablicowemu, ponieważ ta sama operacja jest stosowana do wielu kolejnych elementów danych.

Należy jednak odróżnić wykorzystanie tablic NumPy od pełnej wektoryzacji algorytmu. Część metod, takich jak obliczanie przewyższeń, można zapisać niemal bezpośrednio jako operacje na tablicach. Inne, na przykład porównywanie punktów z segmentami trasy referencyjnej lub dyskretna odległość Frécheta, nadal wymagają pętli albo zależności między wcześniej obliczonymi stanami. W takich przypadkach NumPy służy głównie do reprezentacji danych i przyspieszenia wybranych fragmentów obliczeń, a nie do całkowitego usunięcia iteracji.

Mechanizm broadcastingu pozwala wykonywać operacje na tablicach o różnych, lecz zgodnych kształtach. Dokumentacja NumPy wskazuje, że w takim przypadku pętle mogą być wykonywane po stronie kodu niskopoziomowego zamiast bezpośrednio w Pythonie, bez niepotrzebnego kopiowania danych [27]. W tej pracy większe znaczenie mają jednak proste operacje na tablicach jednowymiarowych, wycinki tablic oraz maski logiczne.

Wariant NumPy pełni więc w pracy rolę reprezentacji tablicowej i narzędzia do zapisu tych obliczeń, które naturalnie można przedstawić jako operacje na jednorodnych sekwencjach wartości liczbowych. Porównanie tego wariantu z podejściem listowym i z wariantem Numba pozwala sprawdzić, jak zapis tablicowy wpływa na czas wykonania metod o różnej strukturze obliczeniowej.

2.1.3 Numba i kompilacja JIT

Trzecim wariantem implementacyjnym jest podejście oparte na kompilacji JIT (*just-in-time compilation*). W tym modelu wybrane funkcje są kompilowane w czasie działania programu dla konkretnych typów argumentów, co pozwala ograniczyć narzut wykonywania pętli przez interpreter Pythona.

W projekcie zastosowano bibliotekę Numba, która została zaprojektowana jako kompilator JIT dla Pythona ukierunkowany na obliczenia naukowe oraz tablicowe [28]. Numba może być szczególnie użyteczna w sytuacjach, w których algorytm ma charakter numeryczny, operuje na typach liczbowych lub tablicach NumPy, a jego logika jest naturalnie zapisana za pomocą jawnych pętli. Nie oznacza to jednak, że każda funkcja Pythona może zostać automatycznie przyspieszona. Możliwość skutecznej kompilacji zależy od użytych typów danych, instrukcji oraz bibliotek zewnętrznych.

W dokumentacji Numby wyróżniane są dwa podstawowe tryby kompilacji: *nopython mode* oraz *object mode*. Tryb *nopython* zakłada wykonanie skompilowanej funkcji bez operowania na ogólnych obiektach Pythona w głównej części obliczeń, natomiast tryb *object mode* zachowuje obsługę obiektów Pythona i zwykle ogranicza potencjalne korzyści wydajnościowe [32]. Z tego powodu w obliczeniach numerycznych istotne jest takie zapisanie funkcji, aby mogła zostać skompilowana w trybie *nopython*. W implementacji programu wykorzystano w tym celu dekorator `@jit`, będący wygodną formą wymuszenia kompilacji w tym trybie.

Wariant Numba zastosowano przede wszystkim dla metod, których struktura obliczeń utrudnia pełne przekształcenie do postaci operacji tablicowych NumPy. Dotyczy to między innymi porównywania punktów śladu zawodnika z segmentami trasy referencyjnej oraz obliczania dyskretnej odległości Frécheta. W obu przypadkach występują jawne pętle i zależności między kolejnymi etapami obliczeń. Wariant Numba pozwala zachować czytelną strukturę algorytmu opartą na pętlach, a jednocześnie może ograniczyć narzut ich wykonywania przez interpreter Pythona.

Kompilacja JIT nie musi obejmować całej funkcji wysokiego poziomu. W metodach wymagających przekształcenia współrzędnych do układu metrycznego część przygotowawcza, taka jak transformacja współrzędnych z użyciem zewnętrznej biblioteki, może pozostać poza zakresem kompilacji. Kompilowany jest wówczas przede wszystkim numeryczny rdzeń algorytmu, na przykład pętla obliczająca odległości punktów od segmentów albo pętla wypełniająca macierz programowania dynamicznego.

Przy pomiarze czasu wykonania funkcji kompilowanych JIT należy uwzględnić koszt pierwszego wywołania funkcji. Pierwsze uruchomienie może obejmować kompilację dla konkretnych typów argumentów, dlatego nie powinno być bezpośrednio porównywane z kolejnymi wywołaniami funkcji już skompilowanej. Z tego powodu w eksperymentach wydajnościowych konieczne jest rozdzielenie etapu rozgrzewki lub wymuszenia kompilacji od właściwego pomiaru czasu wykonania.

Wariant Numba stanowi trzeci sposób wykonania tych samych metod analitycznych. Jego uwzględnienie pozwala porównać zapis tablicowy NumPy z podejściem opartym na kompilacji pętli numerycznych.

Rozdział 3

Projekt i implementacja oprogramowania do analizy tras GPX

Celem rozdziału jest przedstawienie sposobu realizacji programu służącego do analizy tras zapisanych w formacie GPX. Opis obejmuje strukturę projektu, parser plików GPX, warianty funkcji analitycznych oraz skrypty benchmarkowe. W projekcie przygotowano trzy warianty obliczeniowe: wariant bazowy oparty na listach Pythona, wariant wykorzystujący tablice NumPy oraz wariant kompilowany z użyciem biblioteki Numba.

Zakres implementacji obejmuje odczyt punktów śladu GPX, obliczanie dystansu, wyznaczanie sumy podejść i zejść, porównywanie śladu zawodnika z trasą referencyjną, segmentowe porównanie prędkości oraz obliczanie dyskretnej odległości Frécheta. Dodatkowo przygotowano skrypt generujący syntetyczne pliki GPX o kontrolowanej liczbie punktów, wykorzystywane później w benchmarkach.

3.1 Struktura projektu

Projekt został podzielony na katalogi odpowiadające kolejnym etapom pracy z danymi: przygotowaniu plików wejściowych, implementacji funkcji analitycznych oraz zapisowi wyników benchmarków. Taki podział pozwala oddzielić dane wejściowe od kodu programu oraz od wyników pomiarów czasu wykonania.

Listing 3.1: Ogólna struktura katalogów projektu

```
gpx_project/  
|-- data/  
|-- results/  
|   |-- benchmark_distance.json  
|   |-- benchmark_elevation.json  
|   |-- benchmark_route_comparison.json  
|   |-- benchmark_segment_speed.json  
|   |-- benchmark_frechet_distance.json
```

```
|-- scripts/
|   |-- generate_gpx_data.py
|   |-- gpx_parser.py
|   |-- gpx_list.py
|   |-- gpx_numpy.py
|   |-- gpx_numba.py
|   |-- gpx_benchmark_distance.py
|   |-- gpx_benchmark_elevation.py
|   |-- gpx_benchmark_route_comparison.py
|   |-- gpx_benchmark_segment_speed.py
|   |-- gpx_benchmark_frechet_distance.py
|-- requirements.txt
```

Katalog `data` zawiera syntetycznie wygenerowane pliki GPX wykorzystywane jako dane wejściowe. Dla każdej liczby punktów tworzona jest para plików: `trasa_N.gpx` oraz `trasa_N_zawodnik.gpx`. Pierwszy plik pełni rolę trasy referencyjnej, natomiast drugi reprezentuje ślad zawodnika z niewielkimi odchyleniami współrzędnych. Ze względu na rozmiar danych katalog ten może zostać odtworzony przez ponowne uruchomienie skryptu `generate_gpx_data.py`.

Katalog `scripts` zawiera moduły odpowiedzialne za generowanie danych, parsowanie plików GPX, implementację wariantów listowego, NumPy i Numba oraz uruchamianie benchmarków. Katalog `results` przechowuje pliki JSON z wynikami pomiarów czasu wykonania, osobno dla każdej grupy operacji. Plik `requirements.txt` zawiera listę bibliotek wymaganych do uruchomienia projektu, w tym NumPy, Numbę oraz pyproj.

Przepływ danych obejmuje wygenerowanie plików GPX, odczyt punktów przez parser, wykonanie funkcji analitycznych oraz zapis wyników benchmarków.

3.2 Implementacja parsera plików GPX

Pierwszym etapem przetwarzania danych jest odczyt pliku GPX i przekształcenie jego struktury XML do postaci struktur danych wykorzystywanych przez funkcje analityczne. W projekcie zaimplementowano dwie funkcje parsujące: wariant oparty na listach Pythona oraz wariant wykorzystujący tablice NumPy. Obie funkcje realizują tę samą logikę odczytu punktów śladu, lecz różnią się sposobem przechowywania wyników.

Parser został przygotowany dla plików GPX 1.1, w których punkty śladu są zapisane jako elementy `trkpt` w przestrzeni nazw `http://www.topografix.com/GPX/1/1`. Z każdego punktu odczytywane są atrybuty `lat` i `lon` oraz elementy `ele` i `time`. Współrzędne geograficzne i wysokość są konwertowane do typu zmiennoprzecinkowego, natomiast czas pomiaru jest przekształcany do wartości typu `timestamp`.

Znaczniki czasu nie są potrzebne w większości funkcji analitycznych, dlatego przy

obliczaniu dystansu, przewyższeń, porównania geometrycznego oraz odległości Frécheta nie są przekazywane do właściwych funkcji obliczeniowych. Są natomiast wykorzystywane w segmentowym porównaniu prędkości, gdzie służą do wyznaczania czasu pokonania kolejnych odcinków trasy.

Do odczytu dokumentu XML wykorzystano moduł `xml.etree.ElementTree` z biblioteki standardowej Pythona [8]. Dodatkowo zastosowano funkcję pomocniczą `parse_gpx_time`, która zamienia tekstowy zapis czasu z pliku GPX na wartość liczbową. W listingach pokazano wyłącznie fragmenty istotne dla logiki parsera, dlatego pominięto elementy techniczne niewpływające bezpośrednio na sposób odczytu danych GPX.

Listing 3.2: Parser GPX w wariancie listowym

```
1 def parse_gpx_time(value):
2     return datetime.datetime.fromisoformat(
3         value.replace("Z", "+00:00")
4     ).timestamp()
5
6 def parser_py(file):
7     ns = {"gpx": "http://www.topografix.com/GPX/1/1"}
8     tree = ET.parse(file)
9     root = tree.getroot()
10    trackpoints = root.findall("./gpx:trkpt", ns)
11    latitudes = []
12    longitudes = []
13    elevations = []
14    times = []
15    for trkpt in trackpoints:
16        lat_str = trkpt.get("lat")
17        lon_str = trkpt.get("lon")
18        ele_elem = trkpt.find("gpx:ele", ns)
19        time_elem = trkpt.find("gpx:time", ns)
20        if (
21            lat_str is None
22            or lon_str is None
23            or ele_elem is None
24            or not ele_elem.text
25            or time_elem is None
26            or not time_elem.text
27        ):
28            continue
29        latitudes.append(float(lat_str))
30        longitudes.append(float(lon_str))
31        elevations.append(float(ele_elem.text))
32        times.append(parse_gpx_time(time_elem.text))
33    return latitudes, longitudes, elevations, times
```

Listing 3.2 przedstawia parser GPX w wariancie opartym na listach Pythona. W funkcji `parser_py` najpierw definiowana jest przestrzeń nazw GPX, a następnie wyszukiwane są wszystkie elementy `trkpt` (listing 3.2, linie 6–10). Uwzględnienie przestrzeni nazw jest konieczne, ponieważ punkty śladu w pliku GPX należą do domyślnej przestrzeni nazw dokumentu XML.

Zasadnicza część funkcji polega na przejściu po znalezionych punktach śladu. W tej pętli odczytywane są atrybuty `lat` i `lon` oraz elementy `ele` i `time`. Warunek sprawdzający kompletność wartości `lat`, `lon`, `ele` oraz `time` decyduje o pominięciu punktu niespełniającego założeń implementacji (listing 3.2, linie 20–28). Pominięcie punktu bez wysokości lub czasu jest decyzją implementacyjną przyjętą dla potrzeb benchmarków, w których wykorzystywane są kompletne dane syntetyczne. Nie oznacza to, że taki punkt nie ma znaczenia geometrycznego w standardzie GPX. W bardziej ogólnej aplikacji parser mógłby zachowywać punkty zawierające wyłącznie współrzędne, a brakujące wartości wysokości lub czasu obsługiwać osobno.

Poprawnie odczytane wartości są konwertowane i dopisywane do czterech osobnych list: szerokości geograficznych, długości geograficznych, wysokości oraz znaczników czasu. Funkcja zwraca cztery sekwencje: szerokości, długości, wysokości oraz czasy.

Listing 3.3: Parser GPX w wariancie NumPy

```
1 def parser_np(file):
2     ns = {"gpx": "http://www.topografix.com/GPX/1/1"}
3     tree = ET.parse(file)
4     root = tree.getroot()
5     trackpoints = root.findall("./gpx:trkpt", ns)
6     n = len(trackpoints)
7     latitudes = np.empty(n, dtype=np.float64)
8     longitudes = np.empty(n, dtype=np.float64)
9     elevations = np.empty(n, dtype=np.float64)
10    times = np.empty(n, dtype=np.float64)
11    i = 0
12    for trkpt in trackpoints:
13        lat_str = trkpt.get("lat")
14        lon_str = trkpt.get("lon")
15        ele_elem = trkpt.find("gpx:ele", ns)
16        time_elem = trkpt.find("gpx:time", ns)
17        if (
18            lat_str is None
19            or lon_str is None
20            or ele_elem is None
21            or not ele_elem.text
22            or time_elem is None
23            or not time_elem.text
24        ):
```



```
25         continue
26         latitudes[i] = float(lat_str)
27         longitudes[i] = float(lon_str)
28         elevations[i] = float(ele_elem.text)
29         times[i] = parse_gpx_time(time_elem.text)
30         i += 1
31     return latitudes[:i], longitudes[:i], elevations[:i], times[:i]
```

Listing 3.3 przedstawia parser przygotowany dla wariantu NumPy. Zakres odczytanych danych pozostaje taki sam jak w wariancie listowym: funkcja wyszukuje punkty `trkpt`, odczytuje wartości `lat`, `lon`, `ele` oraz `time`, a punkty niespełniające tych założeń są pomijane.

Różnica dotyczy przede wszystkim sposobu organizacji pamięci dla wyników. Po ustaleniu liczby znalezionych punktów tworzone są tablice typu `float64` dla współrzędnych, wysokości oraz czasu (listing 3.3, linie 6–10). Wartości nie są więc dopisywane do list metodą `append`, lecz wpisywane do wcześniej zaalokowanych tablic.

Zmienna `i` pełni rolę licznika punktów faktycznie zapisanych w tablicach. Ma to znaczenie, ponieważ część punktów może zostać pominięta przez warunek sprawdzający kompletność danych. Z tego powodu funkcja zwraca wycinki tablic do indeksu `i`, a nie całe początkowo zaalokowane tablice (listing 3.3, linia 31).

Parser odczytuje punkty `trkpt` jako jedną uporządkowaną sekwencję. W benchmarkach wykorzystano pliki syntetyczne zawierające jeden segment `trkseg`, dlatego nie występuje problem łączenia punktów pochodzących z wielu segmentów śladu.

Parser nie odczytuje gotowych metryk zapisanych ewentualnie w rozszerzeniach pliku GPX. Dystans, suma podejść, suma zejść oraz miary porównania tras są wyznaczane ponownie na podstawie kolejnych punktów śladu.

3.3 Implementacja operacji analizy tras

Po odczytaniu pliku GPX dane wejściowe są przekazywane do funkcji analitycznych. W projekcie zaimplementowano pięć grup operacji: obliczanie dystansu, wyznaczanie sumy podejść i zejść, porównywanie trasy zawodnika z trasą referencyjną, segmentowe porównanie prędkości oraz obliczanie dyskretnej odległości Frécheta.

Każda z tych operacji została przygotowana w trzech wariantach: listowym, NumPy oraz Numba. Warianty te zwracają ten sam typ wyniku dla danej operacji, ale różnią się organizacją danych i sposobem wykonania obliczeń.

3.3.1 Obliczanie dystansu i sum podejść i zejść

Pierwszą grupę operacji stanowią metody wykonywane na pojedynczej trasie. Należą do nich obliczanie całkowitego dystansu oraz wyznaczanie sumy podejść i zejść. Obie operacje opierają się na relacji między sąsiednimi punktami śladu. Dystans jest obliczany jako suma odległości między kolejnymi parami punktów, natomiast przewyższenia wynikają z różnic wysokości między następującymi po sobie próbkami.

We wszystkich wariantach obliczania dystansu wykorzystano wzór Haversine’a jako podstawę wyznaczania odległości między kolejnymi punktami śladu. Dane wejściowe stanowią szerokości i długości geograficzne zapisane w stopniach, które przed wykonaniem obliczeń są przeliczane na radiany. W implementacji przyjęto promień Ziemi równy 6371000.0 metrów.

Wariant listowy zestawiono w listingu 3.4. Zawiera on funkcję obliczającą odległość między dwoma punktami, funkcję sumującą dystans dla całej trasy oraz funkcję wyznaczającą sumę podejść i zejść. Wariant ten wykorzystuje jawne pętle Pythona i stanowi bezpośredni zapis algorytmów opisanych w części teoretycznej.

Listing 3.4: Obliczanie dystansu i przewyższeń w wariancie listowym

```
1 R = 6371000.0
2
3 def haversine_distance(lat1, lon1, lat2, lon2):
4     lat1_rad = math.radians(lat1)
5     lon1_rad = math.radians(lon1)
6     lat2_rad = math.radians(lat2)
7     lon2_rad = math.radians(lon2)
8     delta_lat = lat2_rad - lat1_rad
9     delta_lon = lon2_rad - lon1_rad
10    a = (
11        math.sin(delta_lat / 2) ** 2
12        + math.cos(lat1_rad)
13        * math.cos(lat2_rad)
14        * math.sin(delta_lon / 2) ** 2
15    )
16    a = min(1.0, max(0.0, a))
17    c = 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))
18    return R * c
19
20 def haversine_total_distance(latitudes, longitudes):
21     total_distance = 0.0
22     for i in range(1, len(latitudes)):
23         distance = haversine_distance(
24             latitudes[i - 1],
25             longitudes[i - 1],
26             latitudes[i],
```

```
27         longitudes[i],
28     )
29     total_distance += distance
30     return total_distance
31
32 def elevation_gain_loss(elevations):
33     elevation_gain = 0.0
34     elevation_loss = 0.0
35     for i in range(1, len(elevations)):
36         difference = elevations[i] - elevations[i - 1]
37         if difference > 0:
38             elevation_gain += difference
39         elif difference < 0:
40             elevation_loss += abs(difference)
41     return elevation_gain, elevation_loss
```

W funkcji `haversine_distance` najważniejszy fragment stanowi przeliczenie współrzędnych na radiany oraz podstawienie ich do wzoru Haversine’a (listing 3.4, linie 3–18). Wartość pomocnicza `a` jest dodatkowo ograniczana do przedziału od 0 do 1 (listing 3.4, linia 16), co zabezpiecza dalsze obliczenia przed skutkami błędów zaokrągleń.

Całkowity dystans trasy jest wyznaczany w funkcji `haversine_total_distance`. Kluczowa jest tu pętla przechodząca po kolejnych parach sąsiednich punktów (listing 3.4, linie 20–30). Dla każdej pary wywoływana jest funkcja `haversine_distance`, a uzyskana odległość jest dodawana do sumy całkowitej.

Funkcja `elevation_gain_loss` wykorzystuje analogiczny schemat przejścia po sąsiednich próbkach, ale operuje na wysokościach zamiast na współrzędnych geograficznych. Fragment z liniami 35–40 odpowiada za obliczenie różnicy wysokości i rozdzielenie jej na sumę podejść oraz sumę zejść.

Wariant NumPy realizuje te same obliczenia, lecz zmienia sposób przechodzenia po danych. Zamiast wywoływać funkcję odległości osobno dla każdej pary punktów, program przekazuje do funkcji Haversine’a przesunięte względem siebie fragmenty tablic. Dzięki temu odległości cząstkowe dla kolejnych par punktów mogą zostać obliczone jako operacja tablicowa.

Listing 3.5: Obliczanie dystansu i przewyższeń w wariantcie NumPy

```
1 R = 6371000.0
2
3 def haversine_distance_np(lat1, lon1, lat2, lon2):
4     lat1_rad = np.radians(lat1)
5     lon1_rad = np.radians(lon1)
6     lat2_rad = np.radians(lat2)
7     lon2_rad = np.radians(lon2)
8     delta_lat = lat2_rad - lat1_rad
```

```

9     delta_lon = lon2_rad - lon1_rad
10    a = (
11        np.sin(delta_lat / 2.0) ** 2
12        + np.cos(lat1_rad)
13        * np.cos(lat2_rad)
14        * np.sin(delta_lon / 2.0) ** 2
15    )
16    a = np.clip(a, 0.0, 1.0)
17    c = 2.0 * np.arctan2(np.sqrt(a), np.sqrt(1.0 - a))
18    return R * c
19
20 def haversine_total_distance_np(latitudes, longitudes):
21     distances = haversine_distance_np(
22         latitudes[:-1],
23         longitudes[:-1],
24         latitudes[1:],
25         longitudes[1:],
26     )
27     return float(np.sum(distances))
28
29 def elevation_gain_loss_np(elevations):
30     differences = elevations[1:] - elevations[:-1]
31     elevation_gain = np.sum(differences[differences > 0.0])
32     elevation_loss = np.sum(np.abs(differences[differences < 0.0]))
33     return float(elevation_gain), float(elevation_loss)

```

Wariant tablicowy tych samych operacji pokazano w listingu 3.5. Wzór Haversine’a pozostaje taki sam jak w wariancie listowym, ale funkcje matematyczne pochodzą z biblioteki NumPy i mogą działać na całych tablicach wartości (listing 3.5, linie 3–18).

Najważniejsza różnica pojawia się w funkcji `haversine_total_distance_np`. Wycinki `latitudes[:-1]` i `latitudes[1:]` tworzą dwie przesunięte względem siebie sekwencje punktów (listing 3.5, linie 21–25). Analogiczne wycinki są wykonywane dla długości geograficznych. W efekcie funkcja Haversine’a otrzymuje wszystkie pary sąsiednich punktów jednocześnie, a wynikowa tablica odległości jest sumowana funkcją `np.sum`.

Podobnie zrealizowano obliczanie przewyższeń. Wyrażenie `elevations[1:] - elevations[:-1]` tworzy tablicę różnic wysokości między sąsiednimi punktami (listing 3.5, linia 30). Następnie maski logiczne wybierają wartości dodatnie i ujemne, które są sumowane jako suma podejść i suma zejść (listing 3.5, linie 31–32). Ten fragment jest przykładem operacji, która dobrze pasuje do jednowymiarowej reprezentacji tablicowej.

Trzeci wariant wykorzystuje bibliotekę Numba. W przeciwieństwie do wariantu NumPy nie próbuje on zapisać całego obliczenia jako operacji tablicowej. Struktura funkcji pozostaje zbliżona do wariantu listowego, ale najważniejsze pętle są umieszczone w funkcjach oznaczonych dekoratorem `@njit`. Dane wejściowe dla tego wariantu są przekazywane jako

tablice NumPy.

Listing 3.6: Obliczanie dystansu i przewyższeń w wariancie Numba

```
1 R = 6371000.0
2
3 @njit
4 def haversine_distance_numba(lat1, lon1, lat2, lon2):
5     lat1_rad = math.radians(lat1)
6     lon1_rad = math.radians(lon1)
7     lat2_rad = math.radians(lat2)
8     lon2_rad = math.radians(lon2)
9     delta_lat = lat2_rad - lat1_rad
10    delta_lon = lon2_rad - lon1_rad
11    a = (
12        math.sin(delta_lat / 2.0) ** 2
13        + math.cos(lat1_rad)
14        * math.cos(lat2_rad)
15        * math.sin(delta_lon / 2.0) ** 2
16    )
17    if a < 0.0:
18        a = 0.0
19    elif a > 1.0:
20        a = 1.0
21    c = 2.0 * math.atan2(math.sqrt(a), math.sqrt(1.0 - a))
22    return R * c
23
24 @njit
25 def haversine_total_distance_numba(latitudes, longitudes):
26     total_distance = 0.0
27     for i in range(1, len(latitudes)):
28         distance = haversine_distance_numba(
29             latitudes[i - 1],
30             longitudes[i - 1],
31             latitudes[i],
32             longitudes[i],
33         )
34         total_distance += distance
35     return total_distance
36
37 @njit
38 def elevation_gain_loss_numba(elevations):
39     elevation_gain = 0.0
40     elevation_loss = 0.0
41     for i in range(1, len(elevations)):
42         difference = elevations[i] - elevations[i - 1]
43         if difference > 0.0:
```

```
44         elevation_gain += difference
45     elif difference < 0.0:
46         elevation_loss += abs(difference)
47     return elevation_gain, elevation_loss
```

Listing 3.6 przedstawia wariant wykorzystujący bibliotekę Numba. Dekorator `@njit` wskazuje funkcje, które mają zostać skompilowane (listing 3.6, linie 3, 24 i 37). Sama struktura obliczeń pozostaje zbliżona do wariantu listowego: całkowity dystans jest wyznaczany przez pętlę po kolejnych parach punktów, a przewyższenia przez pętlę po kolejnych wysokościach.

Najważniejsza różnica względem wariantu listowego dotyczy sposobu wykonania tych pętli. W funkcji `haversine_total_distance_numba` pętla po punktach znajduje się w części kompilowanej przez Numbę (listing 3.6, linie 27–35). Analogicznie funkcja `elevation_gain_loss_numba` zachowuje sekwencyjne porównywanie sąsiednich wysokości, ale wykonuje je w funkcji oznaczonej dekoratorem `@njit` (listing 3.6, linie 41–47).

Wszystkie trzy warianty zwracają ten sam typ wyniku: dla dystansu pojedynczą wartość liczbową wyrażoną w metrach, a dla przewyższeń parę wartości oznaczających sumę podejść i sumę zejść. Różnice między wariantami dotyczą więc sposobu wykonania obliczeń, a nie definicji metryki. Z punktu widzenia zawodów sportowych wynik tych funkcji może służyć do przygotowania podstawowego opisu trasy lub śladu zawodnika: długości, sumy podejść oraz sumy zejść.

W implementacji przewyższeń nie zastosowano dodatkowego wygładzania profilu wysokościowego ani progów odrzucających bardzo małe zmiany wysokości. Funkcja realizuje więc bezpośrednią definicję sumowania dodatnich i ujemnych różnic wysokości między sąsiednimi punktami. Dzięki temu wszystkie warianty obliczeniowe wykonują tę samą operację, co ułatwia późniejsze porównanie czasu wykonania.

3.3.2 Porównywanie trasy zawodnika z trasą referencyjną

Kolejną grupę operacji stanowią metody porównujące dwie trasy: trasę referencyjną oraz ślad zawodnika. W odróżnieniu od obliczania dystansu i przewyższeń, które dotyczą pojedynczej sekwencji punktów, porównywanie tras wymaga analizy relacji między dwoma zbiorami danych. W przygotowanej implementacji przyjęto podejście polegające na obliczaniu odległości punktów śladu zawodnika od segmentów trasy referencyjnej.

Ponieważ współrzędne zapisane w plikach GPX są współrzędnymi geograficznymi wyrażonymi w stopniach, przed wykonaniem obliczeń geometrycznych są one przekształcane do układu metrycznego EPSG:2180. Dopiero w tym układzie możliwe jest bezpośrednie stosowanie odległości euklidesowej oraz interpretowanie wyników w metrach. Transformacja jest wykonywana z użyciem obiektu `Transformer` z biblioteki `pyproj` [9]. W kodzie zastosowano parametr `always_xy=True`, który wymusza kolejność argumentów zgodną

z układem x, y . W przypadku współrzędnych geograficznych oznacza to przekazywanie najpierw długości geograficznej, a następnie szerokości geograficznej. Jest to istotne, ponieważ zamiana kolejności tych wartości prowadziłaby do błędnej transformacji współrzędnych i w konsekwencji do niepoprawnych odległości w układzie metrycznym.

Trasa referencyjna po transformacji jest traktowana jako linia łamana. Każda para sąsiednich punktów tworzy segment, względem którego można obliczać odległość punktu śladu zawodnika. Dla każdego punktu zawodnika wyznaczana jest minimalna odległość do wszystkich segmentów trasy referencyjnej. Następnie na podstawie uzyskanych odległości obliczana jest średnia odległość od trasy, największe odchylenie oraz odsetek punktów mieszczących się w przyjętych progach tolerancji.

Listing 3.7: Funkcje pomocnicze porównywania tras w wariancie listowym

```

1 def route_to_euclidean(latitudes, longitudes):
2     xs = []
3     ys = []
4     for lat, lon in zip(latitudes, longitudes):
5         x, y = TRANSFORMER_2180.transform(lon, lat)
6         xs.append(x)
7         ys.append(y)
8     return xs, ys
9
10 def build_segments(xs, ys):
11     segments = []
12     for i in range(1, len(xs)):
13         ax = xs[i - 1]
14         ay = ys[i - 1]
15         bx = xs[i]
16         by = ys[i]
17         abx = bx - ax
18         aby = by - ay
19         ab_len2 = abx * abx + aby * aby
20         segments.append((ax, ay, bx, by, abx, aby, ab_len2))
21     return segments
22
23 def distance_to_segments(px, py, segments):
24     min_dist2 = None
25     for ax, ay, bx, by, abx, aby, ab_len2 in segments:
26         apx = px - ax
27         apy = py - ay
28         if ab_len2 == 0.0:
29             dx = px - ax
30             dy = py - ay
31         else:
32             t = (apx * abx + apy * aby) / ab_len2

```

```

33         if t < 0.0:
34             dx = px - ax
35             dy = py - ay
36         elif t > 1.0:
37             dx = px - bx
38             dy = py - by
39         else:
40             closest_x = ax + t * abx
41             closest_y = ay + t * aby
42             dx = px - closest_x
43             dy = py - closest_y
44         dist2 = dx * dx + dy * dy
45         if min_dist2 is None or dist2 < min_dist2:
46             min_dist2 = dist2
47     return math.sqrt(min_dist2)

```

Funkcje pomocnicze wariantu listowego zestawiono w listingu 3.7. Pierwszym etapem jest transformacja współrzędnych geograficznych do układu EPSG:2180. Funkcja `route_to_euclidean` zapisuje wynik transformacji w dwóch osobnych listach `xs` oraz `ys` (listing 3.7, linie 1–8). Taki układ jest zgodny z ogólną reprezentacją składowych punktu przyjętą w projekcie.

Kolejnym krokiem jest budowa segmentów trasy referencyjnej. Funkcja `build_segments` przechodzi po sąsiednich punktach trasy i dla każdej pary zapisuje współrzędne początku i końca segmentu, składowe wektora oraz kwadrat długości segmentu (listing 3.7, linie 10–21). Kwadrat długości jest później wykorzystywany przy rzutowaniu punktu zawodnika na segment.

Najważniejszy fragment obliczeniowy znajduje się w funkcji `distance_to_segments`. Dla każdego segmentu wyznaczany jest parametr `t`, który określa położenie rzutu punktu względem segmentu (listing 3.7, linia 32). Następnie obsługiwane są trzy przypadki: rzut przed początkiem segmentu, rzut za końcem segmentu oraz rzut znajdujący się wewnątrz segmentu (listing 3.7, linie 33–43). Funkcja porównuje kwadraty odległości i dopiero na końcu zwraca pierwiastek z najmniejszej wartości.

Listing 3.8: Agregacja wyników porównania tras w wariacie listowym

```

1 def compare_routes(
2     reference_latitudes,
3     reference_longitudes,
4     runner_latitudes,
5     runner_longitudes,
6     thresholds_m=(5.0, 10.0, 20.0),
7 ):
8     reference_xs, reference_ys = route_to_euclidean(
9         reference_latitudes,

```



```
10     reference_longitudes,
11 )
12 runner_xs, runner_ys = route_to_euclidean(
13     runner_latitudes,
14     runner_longitudes,
15 )
16 segments = build_segments(reference_xs, reference_ys)
17 total_distance_from_route = 0.0
18 max_distance = 0.0
19 threshold_counts = {}
20 for threshold in thresholds_m:
21     threshold_counts[threshold] = 0
22 for i in range(len(runner_xs)):
23     distance = distance_to_segments(
24         runner_xs[i],
25         runner_ys[i],
26         segments,
27     )
28     total_distance_from_route += distance
29     if distance > max_distance:
30         max_distance = distance
31     for threshold in thresholds_m:
32         if distance <= threshold:
33             threshold_counts[threshold] += 1
34 points_count = len(runner_xs)
35 result = {
36     "mean_distance_m": total_distance_from_route / points_count,
37     "max_distance_m": max_distance,
38 }
39 for threshold in thresholds_m:
40     key = f"within_{int(threshold)}m_percent"
41     result[key] = threshold_counts[threshold] / points_count * 100.0
42 return result
```

Główną funkcję wariantu listowego pokazano w listingu 3.8. Na początku obie trasy są transformowane do układu metrycznego, a dla trasy referencyjnej budowana jest lista segmentów (listing 3.8, linie 8–16). Dopiero po tym etapie możliwe jest właściwe porównanie śladu zawodnika z trasą referencyjną.

Główna pętla funkcji przechodzi po punktach zawodnika i dla każdego z nich wyznacza minimalną odległość od segmentów trasy referencyjnej (listing 3.8, linie 22–33). W tej samej pętli aktualizowana jest suma odległości, największe odchylenie oraz liczniki punktów mieszczących się w zadanych progach tolerancji. Wynikiem funkcji jest słownik zawierający średnią odległość, maksymalną odległość oraz procent punktów mieszczących się w progach 5, 10 i 20 metrów.

Wariant NumPy wykorzystuje tę samą definicję odległości punktu od trasy referencyjnej, ale inaczej organizuje dane po transformacji współrzędnych. Współrzędne punktów oraz parametry segmentów są przechowywane w osobnych tablicach, co pozwala obliczać odległość jednego punktu zawodnika od wszystkich segmentów trasy referencyjnej jako operację tablicową.

Listing 3.9: Budowa segmentów i odległość punkt-segment w wariancie NumPy

```
1 def route_to_euclidean_np(latitudes, longitudes):
2     xs, ys = TRANSFORMER_2180.transform(longitudes, latitudes)
3     return xs, ys
4
5 def build_segments_np(xs, ys):
6     ax = xs[:-1]
7     ay = ys[:-1]
8     bx = xs[1:]
9     by = ys[1:]
10    abx = bx - ax
11    aby = by - ay
12    ab_len2 = abx * abx + aby * aby
13    return ax, ay, bx, by, abx, aby, ab_len2
14
15 def distance_to_segments_np(px, py, segments):
16     ax, ay, bx, by, abx, aby, ab_len2 = segments
17     apx = px - ax
18     apy = py - ay
19     t = np.zeros_like(ab_len2)
20     valid = ab_len2 != 0.0
21     t[valid] = (
22         apx[valid] * abx[valid]
23         + apy[valid] * aby[valid]
24     ) / ab_len2[valid]
25     t = np.clip(t, 0.0, 1.0)
26     closest_x = ax + t * abx
27     closest_y = ay + t * aby
28     dx = px - closest_x
29     dy = py - closest_y
30     dist2 = dx * dx + dy * dy
31     min_dist2 = np.min(dist2)
32     return float(np.sqrt(min_dist2))
```

Listing 3.9 pokazuje tablicową organizację obliczeń dla segmentów trasy referencyjnej. Funkcja `build_segments_np` nie tworzy listy krotek, lecz zestaw tablic opisujących początki segmentów, końce segmentów, wektory segmentów oraz kwadraty ich długości (listing 3.9, linie 5–13).

Kluczowy fragment częściowej wektoryzacji znajduje się w funkcji `distance_to_segments_np`. Dla jednego punktu zawodnika parametr `t` jest obliczany jednocześnie dla wszystkich segmentów referencyjnych (listing 3.9, linie 19–25). Ograniczenie wartości `t` do przedziału od 0 do 1 pozwala obsłużyć rzutowanie na segment bez osobnej pętli po segmentach. Następnie tablicowo wyznaczane są najbliższe punkty na segmentach oraz kwadraty odległości, z których wybierana jest wartość minimalna (listing 3.9, linie 26–32).

Listing 3.10: Porównywanie tras w wariancie NumPy

```
1 def compare_routes_np(  
2     reference_latitudes,  
3     reference_longitudes,  
4     runner_latitudes,  
5     runner_longitudes,  
6     thresholds_m=(5.0, 10.0, 20.0),  
7 ):  
8     reference_xs, reference_ys = route_to_euclidean_np(  
9         reference_latitudes,  
10        reference_longitudes,  
11    )  
12    runner_xs, runner_ys = route_to_euclidean_np(  
13        runner_latitudes,  
14        runner_longitudes,  
15    )  
16    segments = build_segments_np(reference_xs, reference_ys)  
17    distances = np.empty(len(runner_xs), dtype=np.float64)  
18    for i in range(len(runner_xs)):  
19        distances[i] = distance_to_segments_np(  
20            runner_xs[i],  
21            runner_ys[i],  
22            segments,  
23        )  
24    result = {  
25        "mean_distance_m": float(np.mean(distances)),  
26        "max_distance_m": float(np.max(distances)),  
27    }  
28    for threshold in thresholds_m:  
29        key = f"within_{int(threshold)}m_percent"  
30        result[key] = float(  
31            np.sum(distances <= threshold) / len(distances) * 100.0  
32        )  
33    return result
```

Listing 3.10 przedstawia funkcję główną wariantu NumPy. W odróżnieniu od operacji dystansu i przewyższeń metoda ta nie została w pełni zapisana jako jedna operacja tablicowa. Funkcja nadal przechodzi po punktach zawodnika w pętli (listing 3.10, linie

18–23), ponieważ dla każdego punktu trzeba wyznaczyć minimalną odległość od całej trasy referencyjnej.

Częściowa wektoryzacja dotyczy więc obliczenia odległości jednego punktu od wszystkich segmentów, a nie całego porównania obu tras jednocześnie. Po wypełnieniu tablicy `distances` średnia i maksymalna odległość są obliczane za pomocą funkcji `np.mean` i `np.max`, natomiast udział punktów mieszczących się w progach tolerancji jest wyznaczany z użyciem maski logicznej `distances <= threshold` (listing 3.10, linie 24–33).

Wariant Numba zachowuje strukturę obliczeń opartą na pętlach, ale przenosi główną część porównywania tras do funkcji kompilowanej. Transformacja współrzędnych z użyciem biblioteki `pyproj` pozostaje poza funkcją oznaczoną dekoratorem `@jit`, natomiast rdzeń numeryczny otrzymuje już tablice współrzędnych oraz tablice opisujące segmenty trasy referencyjnej.

Listing 3.11: Rdzeń porównywania tras w wariacie Numba

```

1 @jit
2 def compare_routes_core_numba(
3     runner_xs,
4     runner_ys,
5     ax,
6     ay,
7     bx,
8     by,
9     abx,
10    aby,
11    ab_len2,
12    thresholds,
13 ):
14     total_distance_from_route = 0.0
15     max_distance = 0.0
16     threshold_counts = np.zeros(len(thresholds), dtype=np.int64)
17     for i in range(len(runner_xs)):
18         distance = distance_to_segments_numba(
19             runner_xs[i],
20             runner_ys[i],
21             ax,
22             ay,
23             bx,
24             by,
25             abx,
26             aby,
27             ab_len2,
28         )
29         total_distance_from_route += distance
30         if distance > max_distance:

```

```

31         max_distance = distance
32     for j in range(len(thresholds)):
33         if distance <= thresholds[j]:
34             threshold_counts[j] += 1
35     mean_distance = total_distance_from_route / len(runner_xs)
36     return mean_distance, max_distance, threshold_counts

```

Listing 3.11 przedstawia skompilowany rdzeń wariantu Numba. Najważniejszy fragment znajduje się w pętli po punktach zawodnika (listing 3.11, linie 17–34). Dla każdego punktu wywoływana jest funkcja obliczająca minimalną odległość od segmentów referencyjnych, a następnie aktualizowane są suma odległości, maksymalne odchylenie i liczniki progów.

Warto zwrócić uwagę, że funkcja rdzeniowa nie buduje końcowego słownika wynikowego. Zwraca wyłącznie wartości liczbowe oraz tablicę liczników (listing 3.11, linie 35–36). Dzięki temu część objęta kompilacją JIT pozostaje numeryczna i nie wymaga operowania na złożonych strukturach Pythona.

Listing 3.12: Funkcja zewnętrzna porównywania tras w wariacie Numba

```

1 def compare_routes_numba(
2     reference_latitudes,
3     reference_longitudes,
4     runner_latitudes,
5     runner_longitudes,
6     thresholds_m=(5.0, 10.0, 20.0),
7 ):
8     reference_xs, reference_ys = route_to_euclidean_numba(
9         reference_latitudes,
10        reference_longitudes,
11    )
12    runner_xs, runner_ys = route_to_euclidean_numba(
13        runner_latitudes,
14        runner_longitudes,
15    )
16    ax, ay, bx, by, abx, aby, ab_len2 = build_segments_numba(
17        reference_xs,
18        reference_ys,
19    )
20    thresholds = np.asarray(thresholds_m, dtype=np.float64)
21    mean_distance, max_distance, threshold_counts = compare_routes_core_numba(
22        runner_xs,
23        runner_ys,
24        ax,
25        ay,
26        bx,
27        by,
28        abx,

```

```
29     aby,  
30     ab_len2,  
31     thresholds,  
32 )  
33 result = {  
34     "mean_distance_m": float(mean_distance),  
35     "max_distance_m": float(max_distance),  
36 }  
37 for i in range(len(thresholds)):  
38     key = f"within_{int(thresholds[i])}_m_percent"  
39     result[key] = float(threshold_counts[i] / len(runner_xs) * 100.0)  
40 return result
```

Listing 3.12 przedstawia funkcję zewnętrzną wariantu Numba. Jej zadaniem jest przygotowanie danych dla funkcji skompilowanej: transformacja obu tras do układu EPSG:2180, budowa segmentów trasy referencyjnej oraz konwersja progów tolerancji do tablicy NumPy (listing 3.12, linie 8–20).

Właściwe porównanie tras jest wykonywane przez funkcję `compare_routes_core_numba` (listing 3.12, linie 21–33). Dopiero po jej zakończeniu funkcja zewnętrzna tworzy słownik wynikowy w takim samym układzie jak wariant listowy i NumPy. Takie rozdzielanie pokazuje, że w wariancie Numba kompilowany jest przede wszystkim rdzeń numeryczny, a nie cały proces od danych wejściowych do końcowej struktury wyniku.

Wszystkie trzy warianty implementacyjne zwracają wynik w tej samej postaci. Słownik wynikowy zawiera średnią odległość punktów śladu zawodnika od trasy referencyjnej, maksymalne odchylenie oraz procent punktów mieszczących się w progach tolerancji 5, 10 i 20 metrów. Dzięki temu wyniki wariantów listowego, NumPy i Numba mogą być porównywane zarówno pod względem wartości obliczonych metryk, jak i czasu wykonania. Tak przygotowany wynik może być wykorzystany w systemie obsługi zawodów do wstępnego oznaczenia śladów, które znacząco odbiegają od trasy referencyjnej albo wymagają dodatkowej kontroli przez organizatora.

Z punktu widzenia złożoności obliczeniowej metoda wymaga porównania każdego punktu śladu zawodnika z segmentami trasy referencyjnej. Jeżeli trasa zawodnika zawiera m punktów, a trasa referencyjna s segmentów, liczba podstawowych relacji punkt–segment wynosi $m \cdot s$. W przypadku par tras o tej samej liczbie punktów n trasa referencyjna zawiera $n - 1$ segmentów, więc liczba sprawdzeń wynosi $n(n - 1)$. Oznacza to wzrost zbliżony do kwadratowego względem liczby punktów.

Wariant NumPy należy traktować jako częściowo zwektoryzowany: operacje względem segmentów są wykonywane tablicowo dla jednego punktu zawodnika, lecz funkcja nadal iteruje po kolejnych punktach śladu. Wariant Numba zachowuje pętle, ale przenosi ich wykonanie do funkcji kompilowanej.

3.3.3 Segmentowe porównanie prędkości

Segmentowe porównanie prędkości jest funkcją wykorzystującą znaczniki czasu odczytane z pliku GPX. W odróżnieniu od metod opartych wyłącznie na geometrii śladu, funkcja ta pozwala porównać tempo pokonywania kolejnych fragmentów śladu referencyjnego i śladu zawodnika. W aktualnej implementacji parser zwraca cztery sekwencje danych: szerokości geograficzne, długości geograficzne, wysokości oraz znaczniki czasu. Pierwsze trzy wartości są używane także przez wcześniejsze funkcje analityczne, natomiast sekwencja czasów jest wykorzystywana właśnie w segmentowym porównaniu prędkości.

Metoda działa na dwóch trasach: referencyjnej oraz trasie zawodnika. Dla każdej z nich obliczany jest dystans narastający, a następnie wyznaczane są granice segmentów dystansowych. W benchmarkach przyjęto segment o długości 500 m. Ostatni fragment trasy również jest uwzględniany, nawet jeżeli jest krótszy od pełnego segmentu. Aby taki końcowy odcinek nie miał zbyt dużego wpływu na wynik, końcowy stosunek prędkości jest liczony jako średnia ważona długościami segmentów.

Wariant listowy przedstawiono w listingu 3.13. Funkcja `segment_durations` odpowiada za wyznaczenie czasów pokonania kolejnych segmentów, natomiast `compare_segment_speeds` porównuje czasy segmentów trasy referencyjnej i trasy zawodnika.

Listing 3.13: Segmentowe porównanie prędkości w wariancie listowym

```
1 def segment_durations(latitudes, longitudes, times, boundaries):
2     durations = []
3     cumulative_distance = 0.0
4     boundary_index = 1
5     segment_start_time = times[0]
6     for i in range(1, len(latitudes)):
7         previous_distance = cumulative_distance
8         distance = haversine_distance(
9             latitudes[i - 1],
10            longitudes[i - 1],
11            latitudes[i],
12            longitudes[i],
13        )
14        cumulative_distance += distance
15        while (
16            boundary_index < len(boundaries)
17            and boundaries[boundary_index] <= cumulative_distance
18        ):
19            target_distance = boundaries[boundary_index]
20            ratio = (target_distance - previous_distance) / distance
21            boundary_time = (
22                times[i - 1]
```

```
23         + ratio * (times[i] - times[i - 1])
24     )
25     durations.append(boundary_time - segment_start_time)
26     segment_start_time = boundary_time
27     boundary_index += 1
28     return durations
29
30 def compare_segment_speeds(
31     reference_latitudes,
32     reference_longitudes,
33     reference_times,
34     runner_latitudes,
35     runner_longitudes,
36     runner_times,
37     segment_length_m=500.0,
38 ):
39     reference_distance = haversine_total_distance(
40         reference_latitudes,
41         reference_longitudes,
42     )
43     runner_distance = haversine_total_distance(
44         runner_latitudes,
45         runner_longitudes,
46     )
47     common_distance = min(reference_distance, runner_distance)
48     boundaries = [0.0]
49     next_boundary = segment_length_m
50     while next_boundary < common_distance:
51         boundaries.append(next_boundary)
52         next_boundary += segment_length_m
53     boundaries.append(common_distance)
54     reference_durations = segment_durations(
55         reference_latitudes,
56         reference_longitudes,
57         reference_times,
58         boundaries,
59     )
60     runner_durations = segment_durations(
61         runner_latitudes,
62         runner_longitudes,
63         runner_times,
64         boundaries,
65     )
66     segments_count = min(len(reference_durations), len(runner_durations))
67     total_weighted_speed_ratio = 0.0
68     total_distance = 0.0
```



```

69     for i in range(segments_count):
70         segment_distance = boundaries[i + 1] - boundaries[i]
71         speed_ratio = reference_durations[i] / runner_durations[i]
72         total_weighted_speed_ratio += speed_ratio * segment_distance
73         total_distance += segment_distance
74     return total_weighted_speed_ratio / total_distance

```

W funkcji `segment_durations` kluczowe znaczenie ma pętla przechodząca po kolejnych punktach śladu oraz aktualizująca dystans narastający (listing 3.13, linie 6–14). Jeżeli w danym kroku zostaje osiągnięta kolejna granica segmentu, funkcja wyznacza jej położenie między sąsiednimi punktami za pomocą współczynnika `ratio`, a następnie oblicza interpolowany czas osiągnięcia tej granicy (listing 3.13, linie 15–26).

Funkcja `compare_segment_speeds` buduje wspólny zestaw granic segmentów dla trasy referencyjnej i śladu zawodnika. Ostatnią granicą jest `common_distance`, czyli krótszy z dystansów obu tras, co pozwala uwzględnić końcowy fragment wspólny dla obu śladów (listing 3.13, linie 39–53). Wynik końcowy jest obliczany jako średni ważony stosunek prędkości, gdzie wagą jest długość segmentu (listing 3.13, linie 66–74).

Wariant NumPy tej samej metody pokazano w listingu 3.14. Zachowuje on tę samą strukturę obliczeń, ale część operacji wykonywana jest tablicowo. Odległości między kolejnymi punktami są obliczane dla całych wycinków tablic, dystans narastający tworzony jest funkcją `np.cumsum`, natomiast czasy odpowiadające granicom segmentów wyznaczone są funkcją `np.interp`.

Listing 3.14: Segmentowe porównanie prędkości w wariancie NumPy

```

1  def segment_durations_np(latitudes, longitudes, times, boundaries):
2      distances = haversine_distance_np(
3          latitudes[:-1],
4          longitudes[:-1],
5          latitudes[1:],
6          longitudes[1:],
7      )
8      cumulative = np.empty(len(latitudes), dtype=np.float64)
9      cumulative[0] = 0.0
10     cumulative[1:] = np.cumsum(distances)
11     boundary_times = np.interp(boundaries, cumulative, times)
12     return boundary_times[1:] - boundary_times[:-1]
13
14 def compare_segment_speeds_np(
15     reference_latitudes,
16     reference_longitudes,
17     reference_times,
18     runner_latitudes,
19     runner_longitudes,

```

```
20     runner_times,
21     segment_length_m=500.0,
22 ):
23     reference_distance = haversine_total_distance_np(
24         reference_latitudes,
25         reference_longitudes,
26     )
27     runner_distance = haversine_total_distance_np(
28         runner_latitudes,
29         runner_longitudes,
30     )
31     common_distance = min(reference_distance, runner_distance)
32     boundaries = np.arange(
33         0.0,
34         common_distance,
35         segment_length_m,
36         dtype=np.float64,
37     )
38     boundaries = np.append(boundaries, common_distance)
39     reference_durations = segment_durations_np(
40         reference_latitudes,
41         reference_longitudes,
42         reference_times,
43         boundaries,
44     )
45     runner_durations = segment_durations_np(
46         runner_latitudes,
47         runner_longitudes,
48         runner_times,
49         boundaries,
50     )
51     segments_count = min(len(reference_durations), len(runner_durations))
52     segment_distances = (
53         boundaries[1:segments_count + 1] - boundaries[:segments_count]
54     )
55     speed_ratios = (
56         reference_durations[:segments_count]
57         / runner_durations[:segments_count]
58     )
59     return float(
60         np.sum(speed_ratios * segment_distances)
61         / np.sum(segment_distances)
62     )
```

W wariancie NumPy funkcja `segment_durations_np` wykonuje obliczanie odległości między kolejnymi punktami na tablicach wejściowych (listing 3.14, linie 2–7). Dystans

narastający jest następnie wyznaczany funkcją `np.cumsum` (listing 3.14, linie 8–10). Kluczowym etapem jest użycie funkcji `np.interp`, która dla wszystkich granic segmentów wyznacza odpowiadające im czasy (listing 3.14, linia 11). W funkcji głównej granice segmentów tworzone są za pomocą `np.arange`, a końcowy wspólny dystans jest dopisywany do tablicy granic przez `np.append` (listing 3.14, linie 31–38). Końcowe obliczenie wyniku odbywa się tablicowo przez przemnożenie stosunków prędkości przez długości segmentów i zsumowanie tych wartości (listing 3.14, linie 51–62).

Wariant Numba pokazano w listingu 3.15. Podobnie jak w innych funkcjach kompilowanych, wariant ten korzysta z tablic NumPy, ale główna część obliczeń jest oznaczona dekoratorem `@njit`. W porównaniu z wariantem NumPy ręcznie tworzona jest tablica granic segmentów, ponieważ kod wykonywany w funkcji kompilowanej ma bardziej jawny charakter.

Listing 3.15: Segmentowe porównanie prędkości w wariancie Numba

```
1 @njit
2 def segment_durations_numba(latitudes, longitudes, times, boundaries):
3     distances = np.empty(len(latitudes) - 1, dtype=np.float64)
4     for i in range(1, len(latitudes)):
5         distances[i - 1] = haversine_distance_numba(
6             latitudes[i - 1],
7             longitudes[i - 1],
8             latitudes[i],
9             longitudes[i],
10        )
11    cumulative = np.empty(len(latitudes), dtype=np.float64)
12    cumulative[0] = 0.0
13    cumulative[1:] = np.cumsum(distances)
14    boundary_times = np.interp(boundaries, cumulative, times)
15    return boundary_times[1:] - boundary_times[:-1]
16
17 @njit
18 def compare_segment_speeds_numba(
19     reference_latitudes,
20     reference_longitudes,
21     reference_times,
22     runner_latitudes,
23     runner_longitudes,
24     runner_times,
25     segment_length_m=500.0,
26 ):
27     reference_distance = haversine_total_distance_numba(
28         reference_latitudes,
29         reference_longitudes,
30     )
```

```

31 runner_distance = haversine_total_distance_numba(
32     runner_latitudes,
33     runner_longitudes,
34 )
35 common_distance = min(reference_distance, runner_distance)
36 full_segments = int(common_distance // segment_length_m)
37 last_boundary = full_segments * segment_length_m
38 boundaries_count = full_segments + 1
39 if last_boundary < common_distance:
40     boundaries_count += 1
41 boundaries = np.empty(boundaries_count, dtype=np.float64)
42 for i in range(full_segments + 1):
43     boundaries[i] = i * segment_length_m
44 if last_boundary < common_distance:
45     boundaries[boundaries_count - 1] = common_distance
46 reference_durations = segment_durations_numba(
47     reference_latitudes,
48     reference_longitudes,
49     reference_times,
50     boundaries,
51 )
52 runner_durations = segment_durations_numba(
53     runner_latitudes,
54     runner_longitudes,
55     runner_times,
56     boundaries,
57 )
58 segments_count = min(len(reference_durations), len(runner_durations))
59 total_weighted_speed_ratio = 0.0
60 total_distance = 0.0
61 for i in range(segments_count):
62     segment_distance = boundaries[i + 1] - boundaries[i]
63     speed_ratio = reference_durations[i] / runner_durations[i]
64     total_weighted_speed_ratio += speed_ratio * segment_distance
65     total_distance += segment_distance
66 return total_weighted_speed_ratio / total_distance

```

W funkcji `segment_durations_numba` obliczane są odległości między sąsiednimi punktami, a następnie tworzony jest dystans narastający (listing 3.15, linie 3–13). Podobnie jak w wariancie NumPy, do wyznaczenia czasu na granicach segmentów wykorzystano interpolację (listing 3.15, linia 14). W funkcji głównej najpierw obliczana jest wspólna długość analizowanego fragmentu obu tras (listing 3.15, linie 27–35). Następnie tworzona jest tablica granic segmentów, przy czym ostatnia granica odpowiada końcowemu wspólnemu dystansowi (listing 3.15, linie 36–45). Po obliczeniu czasów segmentów dla obu tras wynik jest wyznaczany jako średni ważony stosunek prędkości (listing 3.15, linie 58–66).

Wszystkie trzy warianty zwracają jedną wartość liczbową: średni ważony stosunek prędkości śladu zawodnika do prędkości wyznaczonej dla trasy referencyjnej. Wartość większa od 1 oznacza, że analizowany ślad był średnio szybszy na porównywanych segmentach, natomiast wartość mniejsza od 1 oznacza tempo niższe względem referencji. Wynik nie jest interpretowany jako samodzielna ocena poprawności śladu. Jest to metryka pomocnicza, której znaczenie zależy od sposobu przygotowania trasy referencyjnej, jakości znaczników czasu oraz charakteru danych wejściowych.

3.3.4 Obliczanie dyskretnej odległości Fréchet

Ostatnią zaimplementowaną metodą porównywania tras jest dyskretna odległość Fréchet. W przeciwieństwie do metody punkt-segment, która dla każdego punktu śladu zawodnika wyznacza minimalną odległość od trasy referencyjnej, dyskretna odległość Fréchet porównuje dwie sekwencje punktów jako całość. Metoda ta uwzględnia kolejność punktów w obu trasach, dlatego opisuje podobieństwo przebiegu dwóch linii łamanych w sposób zależny od uporządkowania punktów.

Podobnie jak w przypadku porównywania punktów z segmentami, przed wykonaniem obliczeń współrzędne geograficzne są przekształcane do układu EPSG:2180. Dzięki temu odległość między punktami może być obliczana jako odległość euklidesowa w metrach. Po transformacji obie trasy są traktowane jako sekwencje punktów płaskich. Następnie tworzone jest macierz programowania dynamicznego, w której komórka odpowiada porównaniu początkowych fragmentów obu sekwencji kończących się w danej parze punktów.

W implementacji zastosowano pełną macierz programowania dynamicznego o wymiarach n na m , gdzie n oznacza liczbę punktów trasy referencyjnej, a m liczbę punktów trasy zawodnika. Oznacza to, że metoda ma zarówno koszt czasowy, jak i pamięciowy zależny od iloczynu liczby punktów obu tras.

Listing 3.16: Dyskretna odległość Fréchet w wariantcie listowym

```
1 def frechet_distance(  
2     reference_latitudes,  
3     reference_longitudes,  
4     runner_latitudes,  
5     runner_longitudes,  
6 ):  
7     reference_xs, reference_ys = route_to_euclidean(  
8         reference_latitudes,  
9         reference_longitudes,  
10    )  
11    runner_xs, runner_ys = route_to_euclidean(  
12        runner_latitudes,  
13        runner_longitudes,  
14    )
```

```

15     n = len(reference_xs)
16     m = len(runner_xs)
17     ca = [[0.0 for _ in range(m)] for _ in range(n)]
18     for i in range(n):
19         for j in range(m):
20             dx = reference_xs[i] - runner_xs[j]
21             dy = reference_ys[i] - runner_ys[j]
22             dist2 = dx * dx + dy * dy
23             if i == 0 and j == 0:
24                 ca[i][j] = dist2
25             elif i == 0:
26                 ca[i][j] = max(ca[i][j - 1], dist2)
27             elif j == 0:
28                 ca[i][j] = max(ca[i - 1][j], dist2)
29             else:
30                 ca[i][j] = max(
31                     min(
32                         ca[i - 1][j],
33                         ca[i - 1][j - 1],
34                         ca[i][j - 1],
35                     ),
36                     dist2,
37                 )
38     return math.sqrt(ca[n - 1][m - 1])

```

Listing 3.16 przedstawia wariant listowy obliczania dyskretnej odległości Fréchet’a. Pierwszym etapem jest transformacja obu tras do układu metrycznego (listing 3.16, linie 7–14). Funkcja nie operuje więc bezpośrednio na szerokościach i długościach geograficznych, lecz na dwóch listach współrzędnych `xs` oraz `ys` dla każdej z tras.

Kluczowa struktura danych tworzona jest w linii 17. Jest to dwuwymiarowa lista `ca`, pełniąca rolę macierzy programowania dynamicznego. Jej rozmiar zależy od liczby punktów obu porównywanych tras. Zasadnicze obliczenia wykonywane są w dwóch zagnieżdżonych pętlach (listing 3.16, linie 18–37), które przechodzą po wszystkich parach punktów trasy referencyjnej i trasy zawodnika.

W liniach 20–22 obliczany jest kwadrat odległości euklidesowej między aktualną parą punktów. W macierzy zapisywana jest wartość `dist2`, a nie pierwiastek z tej wartości. Jest to możliwe, ponieważ porównywanie odległości w rekurencji odbywa się przez operacje minimum i maksimum, a pierwiastek kwadratowy jest funkcją monotoniczną dla wartości nieujemnych. Końcowy pierwiastek jest obliczany dopiero przy zwracaniu wyniku funkcji.

Listing 3.17: Dyskretna odległość Fréchet’a w wariantcie NumPy

```

1 def frechet_distance_np(
2     reference_latitudes,

```

```

3     reference_longitudes,
4     runner_latitudes,
5     runner_longitudes,
6 ):
7     reference_xs, reference_ys = route_to_euclidean_np(
8         reference_latitudes,
9         reference_longitudes,
10    )
11    runner_xs, runner_ys = route_to_euclidean_np(
12        runner_latitudes,
13        runner_longitudes,
14    )
15    n = len(reference_xs)
16    m = len(runner_xs)
17    ca = np.zeros((n, m), dtype=np.float64)
18    for i in range(n):
19        for j in range(m):
20            dx = reference_xs[i] - runner_xs[j]
21            dy = reference_ys[i] - runner_ys[j]
22            dist2 = dx * dx + dy * dy
23            if i == 0 and j == 0:
24                ca[i, j] = dist2
25            elif i == 0:
26                ca[i, j] = max(ca[i, j - 1], dist2)
27            elif j == 0:
28                ca[i, j] = max(ca[i - 1, j], dist2)
29            else:
30                ca[i, j] = max(
31                    min(
32                        ca[i - 1, j],
33                        ca[i - 1, j - 1],
34                        ca[i, j - 1],
35                    ),
36                    dist2,
37                )
38    return float(np.sqrt(ca[n - 1, m - 1]))

```

Wariant NumPy tej samej metody zaprezentowano w listingu 3.17. Zakres obliczeń pozostaje taki sam jak w wariancie listowym: obie trasy są transformowane do układu EPSG:2180, a następnie porównywane za pomocą macierzy programowania dynamicznego. Różnica dotyczy reprezentacji tej macierzy. W wariancie NumPy macierz `ca` jest tworzona jako tablica `ndarray` typu `float64` (listing 3.17, linia 17), a nie jako lista `list`.

Warto podkreślić, że zastosowanie NumPy w tym miejscu nie oznacza pełnej wektoryzacji algorytmu. Macierz nadal jest wypełniana iteracyjnie w dwóch zagnieżdżonych pętlach (listing 3.17, linie 18–37). Wynika to z zależności rekurencyjnej: wartość bieżącej komórki

zależy od wcześniej obliczonych komórek po lewej, powyżej oraz po przekątnej. NumPy pełni tutaj przede wszystkim rolę jednorodnej reprezentacji macierzy numerycznej.

Listing 3.18: Rdzeń obliczania dyskretniej odległości Fréchet’a w wariancie Numba

```

1 @njit
2 def frechet_distance_xy_numba(reference_xs, reference_ys, runner_xs, runner_ys):
3     n = len(reference_xs)
4     m = len(runner_xs)
5     ca = np.zeros((n, m), dtype=np.float64)
6     for i in range(n):
7         for j in range(m):
8             dx = reference_xs[i] - runner_xs[j]
9             dy = reference_ys[i] - runner_ys[j]
10            dist2 = dx * dx + dy * dy
11            if i == 0 and j == 0:
12                ca[i, j] = dist2
13            elif i == 0:
14                ca[i, j] = max(ca[i, j - 1], dist2)
15            elif j == 0:
16                ca[i, j] = max(ca[i - 1, j], dist2)
17            else:
18                ca[i, j] = max(
19                    min(
20                        ca[i - 1, j],
21                        ca[i - 1, j - 1],
22                        ca[i, j - 1],
23                    ),
24                    dist2,
25                )
26     return math.sqrt(ca[n - 1, m - 1])

```

Listing 3.18 przedstawia rdzeń obliczeniowy wariantu Numba. Funkcja otrzymuje już współrzędne w układzie metrycznym, dlatego nie obejmuje etapu transformacji z wykorzystaniem biblioteki `pyproj`. Kluczowe znaczenie ma dekorator `@njit` (listing 3.18, linia 1), który wskazuje, że funkcja ma zostać skompilowana przez Numbę.

Sama struktura algorytmu pozostaje taka sama jak w wariancie NumPy: tworzona jest macierz `ca`, a następnie dwie zagnieżdżone pętle przechodzą po wszystkich parach punktów (listing 3.18, linie 5–25). Różnica polega na sposobie wykonania tych pętli. Wariant Numba nie usuwa zależności rekurencyjnej, lecz przenosi jej wykonanie do funkcji kompilowanej.

Listing 3.19: Funkcja zewnętrzna obliczania odległości Fréchet’a w wariancie Numba

```

1 def frechet_distance_numba(
2     reference_latitudes,
3     reference_longitudes,

```



```

4     runner_latitudes,
5     runner_longitudes,
6 ):
7     reference_xs, reference_ys = route_to_euclidean_numba(
8         reference_latitudes,
9         reference_longitudes,
10    )
11    runner_xs, runner_ys = route_to_euclidean_numba(
12        runner_latitudes,
13        runner_longitudes,
14    )
15    return float(
16        frechet_distance_xy_numba(
17            reference_xs,
18            reference_ys,
19            runner_xs,
20            runner_ys,
21        )
22    )

```

Listing 3.19 przedstawia funkcję zewnętrzną wariantu Numba. Jej zadaniem jest wykonanie transformacji obu tras do układu EPSG:2180 (listing 3.19, linie 7–14), a następnie przekazanie przygotowanych tablic współrzędnych do funkcji rdzeniowej (listing 3.19, linie 15–22). Takie rozdzielanie jest istotne, ponieważ kompilacją JIT objęty jest numeryczny rdzeń algorytmu, a nie cały proces przygotowania danych.

Wszystkie trzy warianty implementują tę samą zależność programowania dynamicznego i zwracają pojedynczą wartość odległości w metrach. Różnice między nimi dotyczą sposobu reprezentacji danych oraz wykonania pętli. Wariant listowy używa osobnych list współrzędnych oraz listy list jako macierzy DP. Wariant NumPy wykorzystuje tablicę `ndarray`, ale nadal wypełnia macierz iteracyjnie. Wariant Numba zachowuje strukturę iteracyjną, lecz wykonuje główną pętlę w funkcji kompilowanej. W praktycznym zastosowaniu taka wartość może pełnić rolę dodatkowej miary podobieństwa całego śladu zawodnika do trasy referencyjnej, szczególnie wtedy, gdy organizator chce porównać ogólny przebieg dwóch sekwencji punktów, a nie tylko lokalne odległości od segmentów.

Ze względu na konieczność utworzenia macierzy o wymiarach n na m , metoda ta jest znacznie bardziej kosztowna pamięciowo niż obliczanie dystansu, przewyższeń lub lokalnych odległości punkt–segment. Jeżeli obie porównywane trasy mają po n punktów, liczba komórek macierzy wynosi n^2 , co oznacza kwadratowy wzrost zapotrzebowania na pamięć oraz liczby aktualizowanych stanów.

Dla dwóch tras zawierających po 10 000 punktów macierz programowania dynamicznego zawiera 100 000 000 komórek. W przypadku tablicy typu `float64` oznacza to około 800 MB danych wyłącznie na samą macierz, bez uwzględnienia pozostałych struktur pomocniczych.

Wariant listowy nie jest pod tym względem bezpośrednio porównywalny z tablicą `float64`, ponieważ lista list w Pythonie przechowuje referencje do obiektów, a nie jednorodny bufor wartości liczbowych.

3.4 Implementacja benchmarków

Ostatnim elementem implementacji są skrypty benchmarkowe służące do pomiaru czasu wykonania poszczególnych operacji analitycznych. W projekcie przygotowano osobne skrypty dla każdej grupy metod: obliczania dystansu, obliczania przewyższeń, porównywania trasy zawodnika z trasą referencyjną, segmentowego porównania prędkości oraz obliczania dyskretnej odległości Fréchet’a. Taki podział pozwala analizować funkcje o różnej strukturze obliczeniowej niezależnie od siebie.

Benchmarki nie mierzą czasu parsowania pliku GPX. Każdy skrypt najpierw odczytuje dane wejściowe do struktur odpowiednich dla danego wariantu implementacyjnego, a dopiero następnie mierzy czas wykonania właściwej funkcji analitycznej. Oznacza to, że wyniki benchmarków odnoszą się do kosztu obliczeń wykonywanych na przygotowanych danych, a nie do całkowitego czasu przetwarzania pliku od momentu odczytu dokumentu XML. Takie rozdzielenie jest istotne, ponieważ celem pracy jest porównanie wariantów obliczeniowych, a nie porównanie wydajności parsera XML.

Do pomiaru czasu wykorzystano funkcję `time.perf_counter`. W każdym skrypcie pomiarowym zastosowano funkcję pomocniczą `measure`, która wykonuje przebiegi rozgrzewkowe, serię pomiarów właściwych oraz zwraca średni czas wykonania i listę czasów pojedynczych przebiegów. W tabelach wynikowych przedstawiono średnie czasy wykonania, natomiast pełne listy czasów pojedynczych przebiegów zapisano w plikach JSON. Pozwala to w razie potrzeby sprawdzić rozrzut pomiarów bez rozbudowywania tabel w głównej części pracy. W interpretacji wyników większy nacisk położono na ogólne trendy wraz ze wzrostem liczby punktów niż na pojedyncze różnice między sąsiednimi pomiarami.

Listing 3.20: Funkcja pomocnicza do pomiaru czasu wykonania

```
1 def measure(function, *args):
2     for _ in range(WARMUP_RUNS):
3         function(*args)
4     times = []
5     last_result = None
6     for _ in range(MEASURED_RUNS):
7         start = time.perf_counter()
8         last_result = function(*args)
9         elapsed_time = time.perf_counter() - start
10        times.append(elapsed_time)
11    return {
12        "czas": sum(times) / len(times),
```

```
13     "czasy": times,  
14     "wynik": last_result,  
15 }
```

Listing 3.20 przedstawia wspólny schemat pomiaru czasu. Przebiegi rozgrzewkowe wykonywane są przed właściwą serią pomiarów (listing 3.20, linie 2–3). Ich celem jest ograniczenie wpływu jednorazowych kosztów przygotowania funkcji lub danych na wynik końcowy. Właściwe pomiary wykonywane są w pętli z liniami 6–10, gdzie dla każdego przebiegu zapisywany jest czas rozpoczęcia, wywoływana jest badana funkcja, a następnie obliczany jest czas trwania wywołania.

Funkcja zwraca średni czas wykonania, listę czasów cząstkowych oraz wynik ostatniego wywołania (listing 3.20, linie 11–15). Pole **wynik** może być wykorzystane pomocniczo do sprawdzenia, czy wariant listowy, NumPy i Numba zwracają zgodne wartości dla tych samych danych wejściowych. Nie musi ono jednak być zapisywane w pliku JSON z wynikami benchmarku.

Dla operacji wykonywanych na pojedynczej trasie, takich jak obliczanie dystansu oraz sumowanie przewyższeń, zastosowano większe zbiory danych. W tych przypadkach koszt obliczeniowy rośnie liniowo wraz z liczbą punktów, dlatego benchmarki obejmują pliki od `trasa_10000.gpx` do `trasa_1000000.gpx`. Dla tych operacji wykonano trzy przebiegi rozgrzewkowe oraz dziesięć przebiegów pomiarowych.

Listing 3.21: Zakres danych i liczba pomiarów dla operacji liniowych

```
1 FILES = [  
2     "data/trasa_10000.gpx",  
3     "data/trasa_25000.gpx",  
4     "data/trasa_50000.gpx",  
5     "data/trasa_100000.gpx",  
6     "data/trasa_250000.gpx",  
7     "data/trasa_500000.gpx",  
8     "data/trasa_1000000.gpx",  
9 ]  
10 WARMUP_RUNS = 3  
11 MEASURED_RUNS = 10
```

Listing 3.21 pokazuje zakres danych wykorzystywany dla operacji liniowych. Pliki wejściowe zapisane w liniach 2–8 różnią się liczbą punktów śladu, natomiast linie 10–11 określają liczbę przebiegów rozgrzewkowych i pomiarowych. Taki zakres jest uzasadniony dla dystansu i przewyższeń, ponieważ obie operacje wymagają jedynie przejścia po sekwencji punktów.

Inaczej potraktowano metody porównujące dwie trasy. Zarówno porównywanie punktów śladu zawodnika z segmentami trasy referencyjnej, jak i obliczanie dyskretnej odległości

Fréchet są znacznie bardziej kosztowne niż operacje liniowe. W pierwszej metodzie liczba sprawdzeń rośnie wraz z iloczynem liczby punktów zawodnika i liczby segmentów trasy referencyjnej. W drugiej metodzie tworzona jest macierz programowania dynamicznego, której liczba komórek jest iloczynem liczby punktów obu tras.

Dla benchmarku porównywania tras oraz dla benchmarku dyskretnej odległości Fréchet przyjęto te same pary plików wejściowych. W każdym przypadku plik `trasa_N.gpx` jest porównywany z plikiem `trasa_N_zawodnik.gpx`, gdzie `N` przyjmuje wartości od 1000 do 10000 co 1000 punktów. Takie ustawienie ułatwia późniejsze porównanie trendów czasu wykonania dla obu metod działających na dwóch trasach.

Listing 3.22: Zakres danych dla benchmarków metod porównujących dwie trasy

```
1 ROUTE_PAIRS = [  
2     ("data/trasa_1000.gpx", "data/trasa_1000_zawodnik.gpx"),  
3     ("data/trasa_2000.gpx", "data/trasa_2000_zawodnik.gpx"),  
4     ("data/trasa_3000.gpx", "data/trasa_3000_zawodnik.gpx"),  
5     ("data/trasa_4000.gpx", "data/trasa_4000_zawodnik.gpx"),  
6     ("data/trasa_5000.gpx", "data/trasa_5000_zawodnik.gpx"),  
7     ("data/trasa_6000.gpx", "data/trasa_6000_zawodnik.gpx"),  
8     ("data/trasa_7000.gpx", "data/trasa_7000_zawodnik.gpx"),  
9     ("data/trasa_8000.gpx", "data/trasa_8000_zawodnik.gpx"),  
10    ("data/trasa_9000.gpx", "data/trasa_9000_zawodnik.gpx"),  
11    ("data/trasa_10000.gpx", "data/trasa_10000_zawodnik.gpx"),  
12 ]  
13 WARMUP_RUNS = 2  
14 MEASURED_RUNS = 5
```

Listing 3.22 przedstawia zakres danych stosowany dla metod porównujących dwie trasy. W liniach 2–11 zapisano pary plików wejściowych, przy czym każda para zawiera trasę referencyjną oraz odpowiadającą jej trasę zawodnika o tej samej liczbie punktów. Linie 13–14 określają liczbę przebiegów rozgrzewkowych i pomiarowych. Takie same ustawienia zastosowano w benchmarku porównywania punktów z segmentami oraz w benchmarku dyskretnej odległości Fréchet.

Należy jednak zaznaczyć, że mimo tego samego zakresu danych obie metody mają inną strukturę obliczeniową. Porównywanie punktów z segmentami wymaga sprawdzenia relacji punkt–segment, natomiast dyskretna odległość Fréchet wymaga utworzenia macierzy programowania dynamicznego. Z tego powodu wyniki czasowe tych metod należy interpretować oddzielnie.

Osobny zakres danych zastosowano dla segmentowego porównania prędkości. Metoda ta również operuje na parze tras referencyjna–zawodnik, ale jej główny koszt ma charakter zbliżony do liniowego względem liczby punktów. Z tego powodu benchmark tej funkcji obejmuje większe pary plików, od `trasa_10000.gpx` do `trasa_1000000.gpx`. W bench-

marku przyjęto długość segmentu równą 500 m, trzy przebiegi rozgrzewkowe oraz dziesięć przebiegów pomiarowych.

W przypadku funkcji kompilowanych z użyciem Numby istotne jest oddzielenie kosztu kompilacji JIT od właściwego czasu wykonania funkcji. Oprócz przebiegów rozgrzewkowych na danych z danego benchmarku wykonywane jest również krótkie wywołanie wariantu Numba na fragmencie danych. Celem tego wywołania jest wymuszenie kompilacji funkcji oznaczonych dekoratorem `@njit`, a nie uzyskanie wyniku pomiarowego.

Listing 3.23: Przykład krótkiego wywołania wymuszającego kompilację Numba

```
1 frechet_distance_numba(  
2     ref_lat_np[:10],  
3     ref_lon_np[:10],  
4     run_lat_np[:10],  
5     run_lon_np[:10],  
6 )
```

Listing 3.23 pokazuje przykład wywołania kompilacyjnego dla wariantu Numba. Do funkcji przekazywany jest tylko krótki fragment danych wejściowych, ograniczony do pierwszych dziesięciu punktów każdej trasy (listing 3.23, linie 2–5). Wynik tego wywołania nie jest zapisywany jako wynik benchmarku. Właściwy pomiar czasu rozpoczyna się dopiero po zakończeniu takiego wywołania przygotowawczego.

Każdy skrypt benchmarkowy zapisuje wyniki do osobnego pliku JSON w katalogu `results`. Oddzielne pliki wynikowe utworzono dla dystansu, przewyższeń, porównywania tras, segmentowego porównania prędkości oraz dyskretnej odległości Frécheta. Dzięki temu wyniki metod o różnych kosztach obliczeniowych nie są mieszane w jednym zbiorze danych.

Przykładowy rekord wynikowy zawiera nazwę pliku wejściowego, liczbę punktów, nazwę operacji, liczbę przebiegów rozgrzewkowych, liczbę przebiegów pomiarowych, średni czas wykonania dla każdego wariantu oraz listy czasów pojedynczych pomiarów. W przypadku metod porównujących dwie trasy zapisywane są dodatkowo nazwy obu plików wejściowych oraz liczba relacji punkt–segment albo liczba komórek macierzy programowania dynamicznego.

Listing 3.24: Przykładowa struktura rekordu wynikowego benchmarku

```
1 record = {  
2     "plik": file,  
3     "liczba_punktow": len(lat),  
4     "operacja": "dystans",  
5     "liczba_warmupow": WARMUP_RUNS,  
6     "liczba_pomiarow": MEASURED_RUNS,  
7     "sredni_czas_lista": list_result["czas"],  
8     "sredni_czas_numpy": numpy_result["czas"],
```

```
9     "sredni_czas_numba": numba_result["czas"],  
10     "czasy_lista": list_result["czasy"],  
11     "czasy_numpy": numpy_result["czasy"],  
12     "czasy_numba": numba_result["czasy"],  
13 }
```

Listing 3.24 przedstawia przykładową strukturę rekordu wynikowego dla operacji wykonywanej na pojedynczej trasie. Linie 2–6 opisują dane identyfikujące pomiar: nazwę pliku, liczbę punktów, nazwę operacji oraz liczbę przebiegów rozgrzewkowych i pomiarowych. Linie 7–12 zawierają średnie czasy wykonania oraz listy czasów cząstkowych dla wariantu listowego, NumPy i Numba. Zapis list czasów pozwala w rozdziale wynikowym analizować nie tylko wartość średnią, lecz także stabilność pomiarów.

Opisane skrypty benchmarkowe kończą część implementacyjną pracy. W rozdziale tym przedstawiono pełny przepływ danych: od wygenerowania syntetycznych plików GPX, przez parsowanie punktów śladu, po wykonanie funkcji analitycznych w trzech wariantach implementacyjnych. Same wartości czasów wykonania nie są omawiane w tym miejscu, ponieważ stanowią przedmiot osobnego rozdziału poświęconego metodyce eksperymentów i analizie wyników.

Rozdział 4

Metodyka eksperymentów i analiza wyników

W niniejszym rozdziale przedstawiono sposób przeprowadzenia eksperymentów wydajnościowych oraz wyniki benchmarków uzyskane dla zaimplementowanych wariantów obliczeniowych. Analizie poddano pięć grup operacji: obliczanie dystansu, obliczanie przewyżżeń, porównywanie trasy zawodnika z trasą referencyjną, segmentowe porównanie prędkości oraz obliczanie dyskretnej odległości Frécheta.

Celem eksperymentów było porównanie czasu wykonania tych samych operacji zrealizowanych w trzech wariantach: bazowym opartym na listach Pythona, wariancie wykorzystującym bibliotekę NumPy oraz wariancie wykorzystującym bibliotekę Numba i kompilację JIT. W rozdziale tym nie analizuje się już implementacji funkcji, lecz ich zachowanie pomiarowe dla rosnącego rozmiaru danych wejściowych.

Wyniki prezentowane są w postaci tabel oraz wykresów. Tabele zawierają średnie czasy wykonania uzyskane w kolejnych benchmarkach, natomiast wykresy mają ułatwiać ocenę trendu wzrostu czasu wykonania oraz różnic między badanymi wariantami.

4.1 Środowisko testowe

Wiarygodna interpretacja benchmarków wymaga określenia środowiska, w którym zostały wykonane pomiary. Czas działania funkcji może zależeć między innymi od procesora, pamięci operacyjnej, systemu operacyjnego, wersji interpretera Pythona oraz środowiska, w którym uruchamiane są skrypty. Najważniejsze parametry sprzętowe i programowe zestawiono w tabeli 4.1.

Tabela 4.1: Podstawowe parametry środowiska testowego

Parametr	Wartość
System operacyjny hosta	Microsoft Windows 10 Pro
Typ komputera	Komputer stacjonarny x64
Procesor	Intel Core i7-5820K CPU @ 3.30 GHz
Liczba rdzeni procesora	6 rdzeni
Pamięć RAM	32 GB
Płyta główna	MSI X99S SLI PLUS (MS-7885)
Środowisko uruchomieniowe	Windows Subsystem for Linux 2
Wersja WSL	2.7.3.0
Domyślna dystrybucja WSL	Ubuntu
Dystrybucja Linux	Ubuntu 24.04.4 LTS (Noble Numbat)
Wersja jądra Linux w WSL	6.6.114.1-microsoft-standard-WSL2
Wersja interpretera Pythona	Python 3.12.3

Skrypty benchmarkowe były uruchamiane w środowisku WSL 2, w dystrybucji Ubuntu. Parametry sprzętowe odnoszą się do komputera hosta, natomiast pomiary wykonywano w środowisku linuksowym udostępnionym przez WSL. Takie rozróżnienie jest istotne, ponieważ system hosta i środowisko wykonania skryptów nie są w tym przypadku tym samym systemem operacyjnym.

4.2 Metodyka pomiarów

Pomiary wykonano w celu porównania czasu działania pięciu grup funkcji analitycznych: obliczania dystansu, obliczania przewyższeń, porównywania trasy zawodnika z trasą referencyjną, segmentowego porównania prędkości oraz obliczania dyskretnej odległości Frécheta. Każda z tych operacji została uruchomiona w trzech wariantach implementacyjnych: listowym, NumPy oraz Numba.

Dane wejściowe zostały przygotowane za pomocą skryptu `generate_gpx_data.py`. Skrypt tworzy syntetyczne pliki GPX o ustalonej liczbie punktów. Dla każdej wartości N powstają dwa pliki: `trasa_N.gpx` oraz `trasa_N_zawodnik.gpx`. Pierwszy z nich pełni rolę trasy referencyjnej, natomiast drugi zawiera niewielkie odchylenia współrzędnych i jest używany w metodach porównujących dwie trasy.

Wygenerowane pliki zachowują strukturę GPX 1.1 i zawierają jeden segment `trkseg`. Współrzędne punktów są zapisane jako szerokość i długość geograficzna w stopniach dziesiętnych, a nie jako współrzędne w układzie metrycznym. Dzięki temu dane wejściowe odpowiadają strukturze plików GPX, natomiast ewentualna transformacja do układu metrycznego jest wykonywana dopiero w funkcjach wymagających obliczeń geometrycznych.

Benchmarki obejmowały wyłącznie czas wykonania funkcji analitycznych na danych

wcześniej odczytanych z plików GPX. Etap parsowania dokumentu XML nie był wliczany do czasu właściwych obliczeń. Takie rozdzielanie pozwala porównywać warianty implementacyjne funkcji, a nie wydajność samego parsera.

Tabela 4.2: Zakres danych i liczba powtórzeń w benchmarkach

Operacja	Zakres danych	Warm-up	Pomiary
Obliczanie dystansu	10 000–1 000 000 punktów	3	10
Obliczanie przewyższeń	10 000–1 000 000 punktów	3	10
Porównywanie tras	pary tras od 1 000 do 10 000 punktów, co 1 000	2	5
Segmentowe porównanie prędkości	pary tras od 10 000 do 1 000 000 punktów	3	10
Dyskretna odległość Fréchet	pary tras od 1 000 do 10 000 punktów, co 1 000	2	5

Dla segmentowego porównania prędkości przyjęto długość segmentu równą 500 m. Metoda ta operuje na dwóch trasach, ale jej główny koszt ma charakter zbliżony do liniowego względem liczby punktów, dlatego możliwe było użycie większych plików wejściowych niż w przypadku porównywania punktów z segmentami oraz metody Fréchet.

Do pomiaru czasu wykorzystano funkcję `time.perf_counter`. Dla wariantu Numba przed właściwym pomiarem wykonywano krótkie wywołanie przygotowawcze, aby oddzielić koszt kompilacji JIT od czasu działania funkcji już skompilowanej. Wyniki zapisywano do plików JSON w katalogu `results`, osobno dla każdej grupy operacji.

4.3 Wyniki benchmarków

Wyniki benchmarków przedstawiono osobno dla każdej grupy operacji. Taki układ pozwala porównać warianty implementacyjne w ramach funkcji o podobnej strukturze obliczeniowej. Operacje wykonywane na pojedynczej trasie, takie jak dystans i przewyższenia, zestawiono oddzielnie od metod porównujących dwie trasy, ponieważ różnią się one kosztem obliczeniowym oraz sposobem organizacji pętli.

Dobór analizowanych funkcji wynikał z ich potencjalnego zastosowania w analizie tras sportowych. Dystans i przewyższenia opisują pojedynczą trasę lub ślad zawodnika. Porównywanie punktów z segmentami oraz dyskretna odległość Fréchet pozwalają oceniać relację geometryczną między śladem zawodnika a trasą referencyjną. Segmentowe porównanie prędkości uzupełnia tę analizę o informację czasową, ponieważ wykorzystuje znaczniki czasu zapisane w pliku GPX.

W każdej podsekcji zaprezentowano tabelę ze średnimi czasami wykonania oraz wykres pokazujący zmianę czasu wraz ze wzrostem liczby punktów. Interpretacja wyników dotyczy

badanej implementacji i przyjętego środowiska pomiarowego.

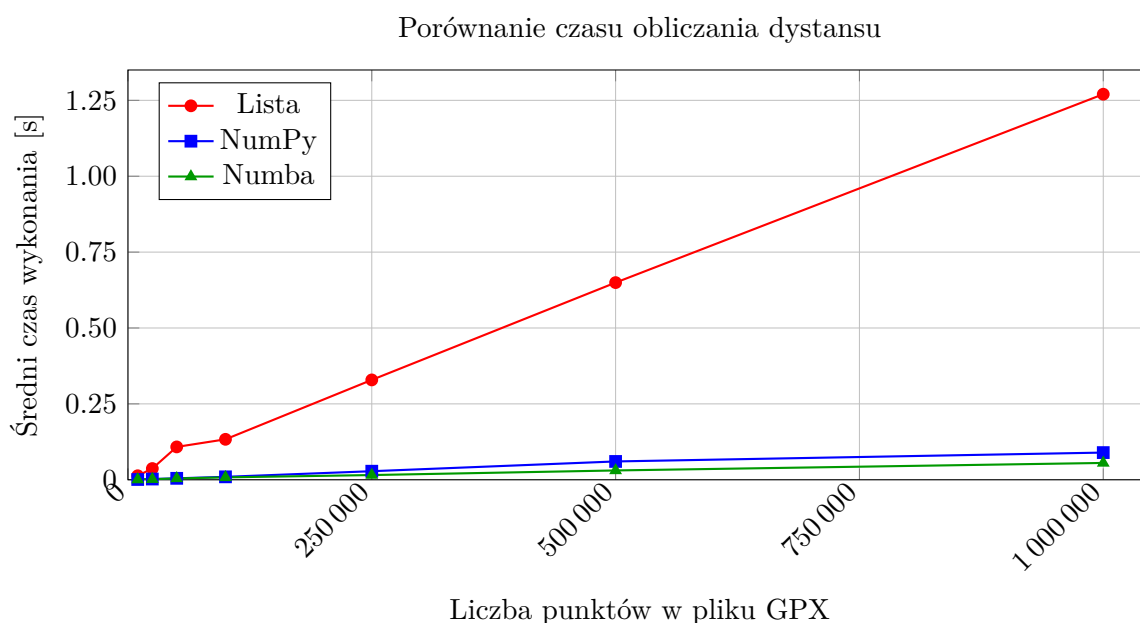
4.3.1 Obliczanie dystansu

Pierwszą analizowaną operacją jest obliczanie całkowitego dystansu trasy na podstawie kolejnych par punktów śladu. Jest to operacja liniowa względem liczby punktów wejściowych, dlatego stanowi dobry punkt odniesienia dla porównania wariantu listowego, NumPy i Numba.

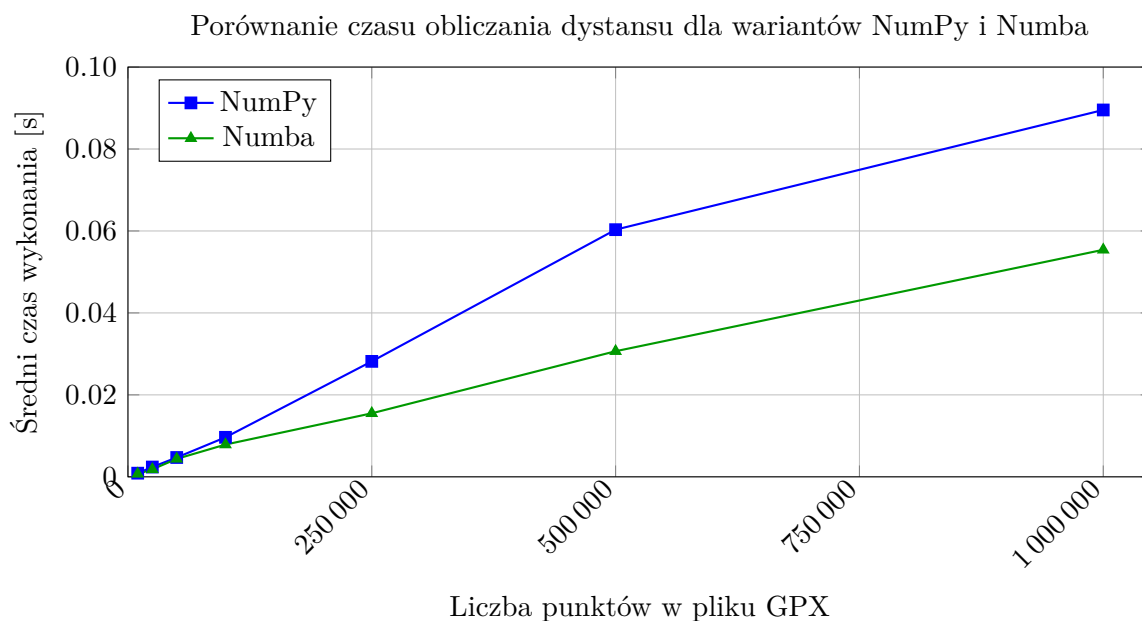
Tabela 4.3 przedstawia średnie czasy wykonania uzyskane dla kolejnych rozmiarów danych wejściowych. W każdym przypadku porównywano te same pliki GPX, a różnice dotyczyły wyłącznie sposobu realizacji obliczeń.

Tabela 4.3: Średnie czasy wykonania benchmarku obliczania dystansu

Liczba punktów	Lista [s]	NumPy [s]	Numba [s]
10 000	0,013216	0,000891	0,000714
25 000	0,036973	0,002391	0,001820
50 000	0,108215	0,004717	0,004384
100 000	0,133220	0,009641	0,007873
250 000	0,328946	0,028175	0,015503
500 000	0,649536	0,060318	0,030652
1 000 000	1,270193	0,089529	0,055405



Wykres 4.1: Porównanie średniego czasu obliczania dystansu dla wariantów listowego, NumPy i Numba.



Wykres 4.2: Porównanie średniego czasu obliczania dystansu dla wariantów NumPy i Numba.

Na podstawie danych z tabeli 4.3 można stwierdzić, że wszystkie trzy warianty wykazują wzrost czasu wykonania wraz ze wzrostem liczby punktów wejściowych, co jest zgodne z liniowym charakterem analizowanej operacji. Jednocześnie różnice między wariantami są wyraźne w całym badanym zakresie danych.

Wariant listowy okazał się najwolniejszy dla wszystkich rozmiarów wejścia. Wariant NumPy był wyraźnie szybszy od wariantu listowego, natomiast najlepsze wyniki w całym zakresie danych uzyskał wariant Numba. Dla najmniejszego testowanego pliku, zawierającego 10 000 punktów, czas wykonania zmniejszył się z około 0.0132 s dla wariantu listowego do około 0.00071 s dla wariantu Numba. Dla największego pliku, zawierającego 1 000 000 punktów, czasy wynosiły odpowiednio około 1.27 s, 0.0895 s oraz 0.0554 s.

Oznacza to, że w przeprowadzonym eksperymencie wariant NumPy był około 14 razy szybszy od wariantu listowego dla największego badanego pliku, natomiast wariant Numba był od niego szybszy około 23 razy. Wynik ten sugeruje, że dla operacji polegającej na wielokrotnym wykonywaniu prostych obliczeń numerycznych na kolejnych parach punktów zarówno operacje tablicowe, jak i kompilacja JIT przynoszą istotną korzyść wydajnościową.

Należy jednak ostrożnie interpretować przyczyny uzyskanego przyspieszenia. Jedną z możliwych przyczyn przewagi wariantu NumPy może być ograniczenie kosztu jawnych pętli Pythona przez zastosowanie operacji tablicowych. Z kolei w przypadku wariantu Numba znaczenie może mieć przeniesienie pętli numerycznej do funkcji kompilowanej. Na podstawie samych benchmarków czasowych nie należy jednak jednoznacznie przypisywać całego przyspieszenia jednej przyczynie. Wynik zależy zarówno od sposobu zapisu pętli, jak i od reprezentacji danych oraz kosztu wywołań funkcji numerycznych.

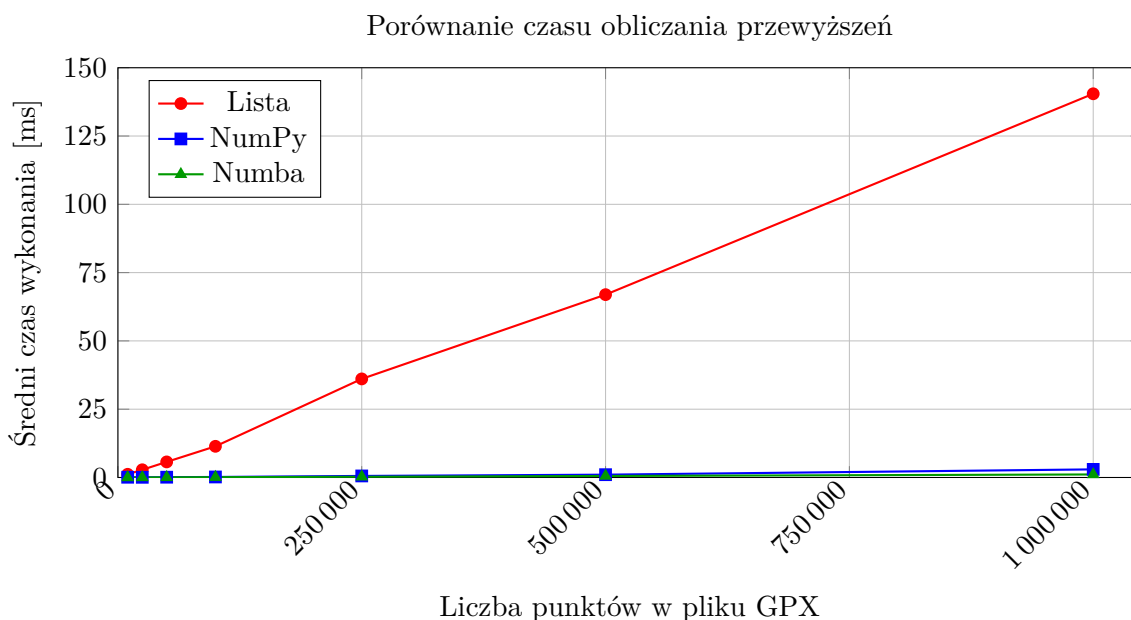
4.3.2 Obliczanie przewyższeń

Drugą analizowaną operacją jest obliczanie sumy podejść i zejść na podstawie kolejnych wartości wysokości. W odróżnieniu od obliczania dystansu operacja ta nie wymaga funkcji trygonometrycznych ani przeliczania współrzędnych geograficznych. Algorytm opiera się na wyznaczeniu różnic wysokości między sąsiednimi punktami oraz oddzielnym zsumowaniu różnic dodatnich i ujemnych.

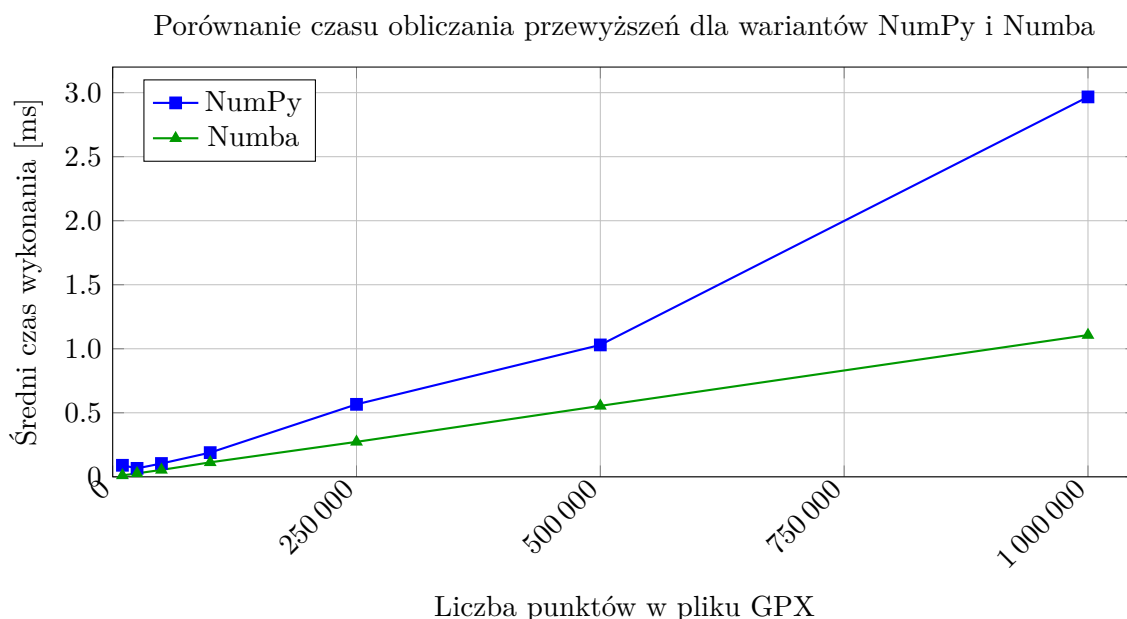
Tabela 4.4 przedstawia średnie czasy wykonania uzyskane dla wariantu listowego, NumPy oraz Numba. Podobnie jak w przypadku dystansu, pomiar wykonano dla plików o liczbie punktów od 10 000 do 1 000 000.

Tabela 4.4: Średnie czasy wykonania benchmarku obliczania przewyższeń

Liczba punktów	Lista [s]	NumPy [s]	Numba [s]
10 000	0,001171	0,000090	0,000011
25 000	0,002825	0,000066	0,000027
50 000	0,005702	0,000104	0,000054
100 000	0,011412	0,000189	0,000113
250 000	0,036085	0,000566	0,000273
500 000	0,066933	0,001030	0,000554
1 000 000	0,140462	0,002967	0,001107



Wykres 4.3: Porównanie średniego czasu obliczania przewyższeń dla wariantów listowego, NumPy i Numba. Czas przedstawiono w milisekundach.



Wykres 4.4: Porównanie średniego czasu obliczania przewyższeń dla wariantów NumPy i Numba. Czas przedstawiono w milisekundach.

Na podstawie tabeli 4.4 można zauważyć, że wariant listowy wykazuje wzrost czasu wykonania wraz ze wzrostem liczby punktów wejściowych. Jest to zgodne z charakterem operacji, ponieważ funkcja przechodzi po kolejnych parach sąsiednich wysokości i dla każdej z nich oblicza różnicę.

Warianty NumPy i Numba osiągnęły znacznie krótsze czasy wykonania niż wariant listowy. Dla największego pliku, zawierającego 1 000 000 punktów, wariant listowy wykonał obliczenia w czasie około 0.1405 s, wariant NumPy w czasie około 0.0030 s, a wariant Numba w czasie około 0.0011 s. Oznacza to, że dla tego przypadku wariant NumPy był około 47 razy szybszy od wariantu listowego, natomiast wariant Numba około 127 razy szybszy od wariantu listowego.

Różnice czasu można wyjaśnić sposobem wykonania obliczeń. Wariant NumPy oblicza tablicę różnic wysokości za pomocą działania na przesuniętych fragmentach tablicy, a następnie stosuje maski logiczne do wyboru wartości dodatnich i ujemnych. Wariant Numba zachowuje prostą pętlę po kolejnych wysokościach, ale wykonuje ją w funkcji skompilowanej. Dla tak prostej operacji numerycznej narzut pętli Pythona w wariancie listowym staje się dobrze widoczny już przy większych zbiorach danych.

Należy jednak ostrożnie interpretować wyniki dla najmniejszych plików. Czasy wykonania wariantów NumPy i Numba są tam bardzo małe, dlatego pojedyncze różnice mogą wynikać z narzutu wywołania funkcji, chwilowego obciążenia systemu lub innych czynników pomiarowych. Z tego powodu większe znaczenie interpretacyjne ma ogólny trend widoczny dla rosnącej liczby punktów niż pojedyncze odchylenia między sąsiednimi rozmiarami danych.

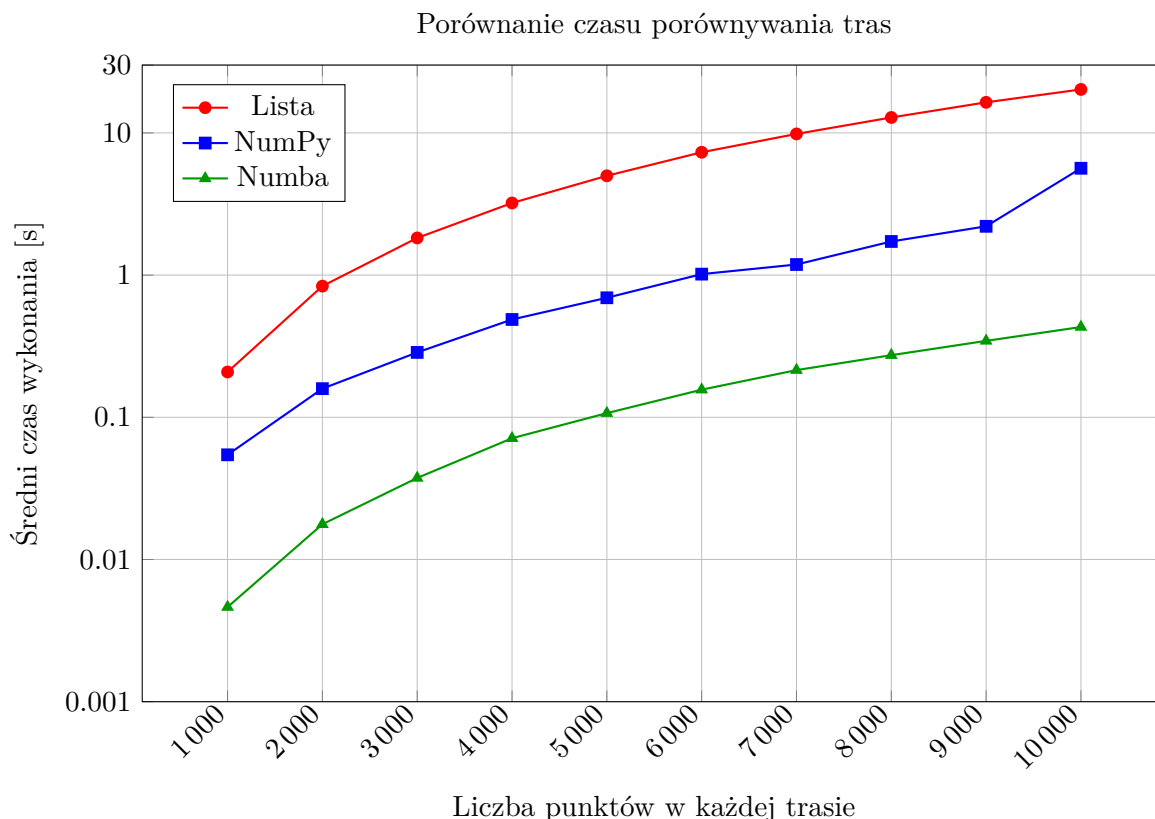
4.3.3 Porównywanie trasy zawodnika z trasą referencyjną

Kolejną analizowaną operacją jest porównywanie trasy zawodnika z trasą referencyjną. W tej metodzie każdy punkt śladu zawodnika jest porównywany z segmentami trasy referencyjnej. Dla par tras o tej samej liczbie punktów n trasa referencyjna zawiera $n - 1$ segmentów, dlatego liczba relacji punkt-segment wynosi $n(n - 1)$. Oznacza to, że koszt obliczeń ma charakter zbliżony do kwadratowego względem liczby punktów.

Tabela 4.5 przedstawia średnie czasy wykonania dla wariantu listowego, NumPy oraz Numba. W benchmarku wykorzystano pary plików `trasa_N.gpx` oraz `trasa_N_zawodnik.gpx`, gdzie liczba punktów zwiększała się od 1000 do 10 000.

Tabela 4.5: Średnie czasy wykonania benchmarku porównywania tras

Liczba punktów	Relacje punkt-segment	Lista [s]	NumPy [s]	Numba [s]
1 000	999 000	0,208341	0,054492	0,004629
2 000	3 998 000	0,836794	0,159032	0,017689
3 000	8 997 000	1,825742	0,285992	0,037458
4 000	15 996 000	3,215168	0,487333	0,071246
5 000	24 995 000	4,992853	0,692888	0,106967
6 000	35 994 000	7,311315	1,016197	0,156377
7 000	48 993 000	9,839682	1,186977	0,214730
8 000	63 992 000	12,840634	1,723951	0,273499
9 000	80 991 000	16,407631	2,203830	0,344766
10 000	99 990 000	20,216993	5,641450	0,431464



Wykres 4.5: Porównanie średniego czasu porównywania tras dla wariantów listowego, NumPy i Numba. Oś pionowa została przedstawiona w skali logarytmicznej ze względu na duże różnice czasu wykonania między wariantami.

Wyniki przedstawione w tabeli 4.5 pokazują znacznie szybszy wzrost czasu wykonania niż w przypadku operacji wykonywanych na pojedynczej trasie. Jest to zgodne ze strukturą metody, ponieważ zwiększenie liczby punktów powoduje wzrost liczby relacji punkt-segment. Dla 1000 punktów liczba sprawdzeń wynosi 999 000, natomiast dla 10 000 punktów wzrasta do 99 990 000.

Wariant listowy uzyskał najwyższe czasy wykonania w całym zakresie danych. Dla pary tras zawierających po 1000 punktów średni czas wyniósł około 0.208 s, natomiast dla 10 000 punktów wzrósł do około 20.217 s. Wariant NumPy był szybszy od wariantu listowego, ale nie w takim stopniu jak przy obliczaniu dystansu lub przewyższeń. Dla 10 000 punktów czas wykonania wariantu NumPy wyniósł około 5.641 s.

Najkrótsze czasy wykonania uzyskał wariant Numba. Dla 1000 punktów średni czas wyniósł około 0.0046 s, a dla 10 000 punktów około 0.4315 s. Oznacza to, że dla największego badanego przypadku wariant Numba był około 47 razy szybszy od wariantu listowego oraz około 13 razy szybszy od wariantu NumPy.

Różnice między wariantami można powiązać ze strukturą zastosowanych funkcji. Wariant NumPy wykonuje część obliczeń tablicowo, ale nadal iteruje po kolejnych punktach śladu zawodnika. Częściowa wektoryzacja dotyczy przede wszystkim obliczania odległości

jednego punktu od wszystkich segmentów trasy referencyjnej. Wariant Numba zachowuje strukturę pętli, lecz przenosi ich wykonanie do funkcji kompilowanej. W przypadku metody wymagającej dużej liczby powtarzalnych obliczeń punkt–segment taka organizacja może być jedną z przyczyn wyraźnego skrócenia czasu wykonania.

Warto zwrócić uwagę, że dla wariantu NumPy przy 10 000 punktów widoczny jest większy wzrost czasu niż między wcześniejszymi rozmiarami danych. Może to wynikać między innymi z kosztu tworzenia tablic pośrednich dla coraz większej liczby segmentów, wpływu zarządzania pamięcią lub chwilowych czynników pomiarowych. Nie należy jednak interpretować tego pojedynczego wzrostu jako samodzielnego dowodu na konkretną przyczynę. Może on wynikać zarówno ze struktury obliczeń, jak i z kosztu tworzenia tablic pośrednich lub z czynników środowiskowych wpływających na pomiar czasu.

4.3.4 Segmentowe porównanie prędkości

Kolejną analizowaną operacją jest segmentowe porównanie prędkości. Funkcja ta różni się od wcześniejszych metod tym, że wykorzystuje nie tylko współrzędne punktów, lecz także znaczniki czasu odczytane z pliku GPX. Dla trasy referencyjnej oraz trasy zawodnika wyznaczone są czasy pokonania kolejnych segmentów dystansowych, a następnie obliczany jest średni ważony stosunek prędkości zawodnika do prędkości referencji.

W benchmarku przyjęto segment o długości 500 m. Ostatni fragment trasy również jest uwzględniany, nawet jeśli jest krótszy od pełnego segmentu. Wynik końcowy jest liczony jako średnia ważona długościami segmentów, dzięki czemu krótszy odcinek końcowy nie ma takiego samego wpływu na wynik jak pełny segment.

Pod względem obliczeniowym metoda ma charakter zbliżony do liniowego względem liczby punktów śladu. Główny koszt wynika z obliczenia dystansu narastającego dla obu tras oraz z wyznaczenia czasów odpowiadających granicom segmentów. W odróżnieniu od metody punkt–segment i dyskretnej odległości Frécheta funkcja nie wymaga porównywania każdego punktu jednej trasy z wieloma punktami lub segmentami drugiej trasy.

Tabela 4.6: Średnie czasy wykonania segmentowego porównania prędkości

Liczba punktów	Lista [s]	NumPy [s]	Numba [s]	Lista/NumPy	Lista/Numba
10 000	0,093579	0,002799	0,002256	33,4	41,5
25 000	0,208338	0,006752	0,006019	30,9	34,6
50 000	0,415450	0,013540	0,012837	30,7	32,4
100 000	0,867939	0,036137	0,025011	24,0	34,7
250 000	1,926521	0,096761	0,055915	19,9	34,5
500 000	3,766445	0,235519	0,115930	16,0	32,5
1 000 000	7,785088	0,517905	0,238240	15,0	32,7

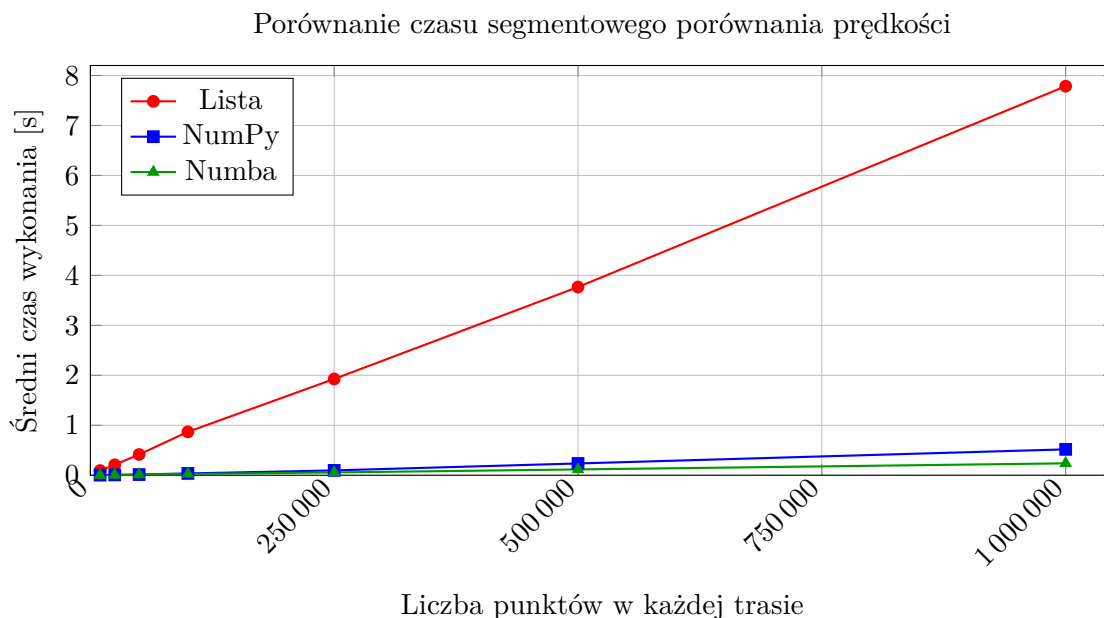
Wyniki przedstawione w tabeli 4.6 pokazują, że czas wykonania segmentowego porówna-

nia prędkości rośnie wraz z liczbą punktów wejściowych. Wariant listowy jest najwolniejszy w całym zakresie danych. Dla największego przypadku testowego, obejmującego 1 000 000 punktów w każdej trasie, średni czas wykonania wyniósł około 7.79 s.

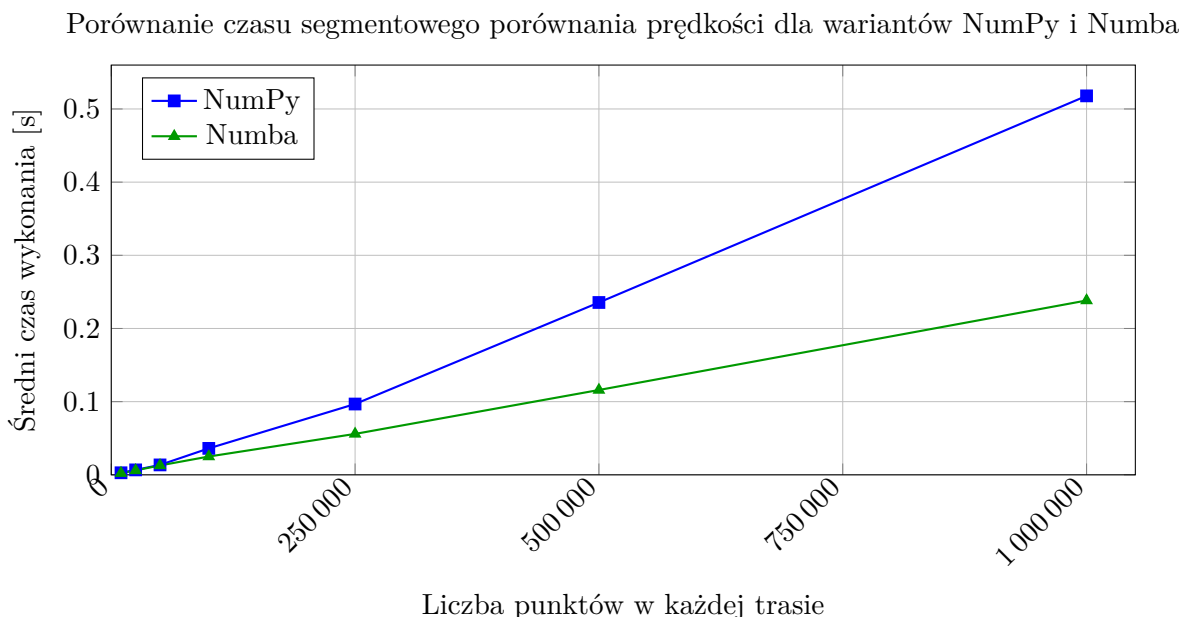
Wariant NumPy wykonał tę samą operację dla największego przypadku w czasie około 0.52 s, natomiast wariant Numba w czasie około 0.24 s. Oznacza to, że dla największego zbioru danych wariant NumPy był około 15 razy szybszy od wariantu listowego, a wariant Numba około 33 razy szybszy. Różnica wynika głównie z tego, że wariant listowy oblicza odległości między kolejnymi punktami w jawnych pętlach Pythona, natomiast warianty NumPy i Numba wykonują zasadniczą część obliczeń na tablicach numerycznych lub w funkcjach kompilowanych.

Warto zauważyć, że przewaga wariantu NumPy względem list jest największa dla mniejszych plików, a następnie stopniowo maleje. Dla 10 000 punktów wariant NumPy był około 33 razy szybszy od listowego, natomiast dla 1 000 000 punktów około 15 razy szybszy. Wariant Numba utrzymał bardziej stabilną przewagę nad listami, wynoszącą w większości przypadków ponad 30 razy. Dla największego zbioru danych Numba była również około 2.2 razy szybsza od wariantu NumPy.

Segmentowe porównanie prędkości uzupełnia wcześniejsze metody, ponieważ wykorzystuje czas zapisany w pliku GPX. Wartości tej funkcji należy jednak interpretować ostrożnie w przypadku danych syntetycznych. Dane te zostały przygotowane przede wszystkim do porównania czasu wykonania wariantów implementacyjnych, a nie do realistycznego modelowania tempa zawodnika.



Wykres 4.6: Porównanie średniego czasu wykonania segmentowego porównania prędkości dla wariantów listowego, NumPy i Numba.



Wykres 4.7: Porównanie średniego czasu wykonania segmentowego porównania prędkości dla wariantów NumPy i Numba.

Rysunek 4.7 pokazuje różnicę między wariantami NumPy i Numba bez dominującego wpływu wariantu listowego na skalę wykresu. Wariant Numba uzyskał krótszy czas wykonania w całym zakresie danych, a przewaga ta była najbardziej widoczna dla największych plików wejściowych.

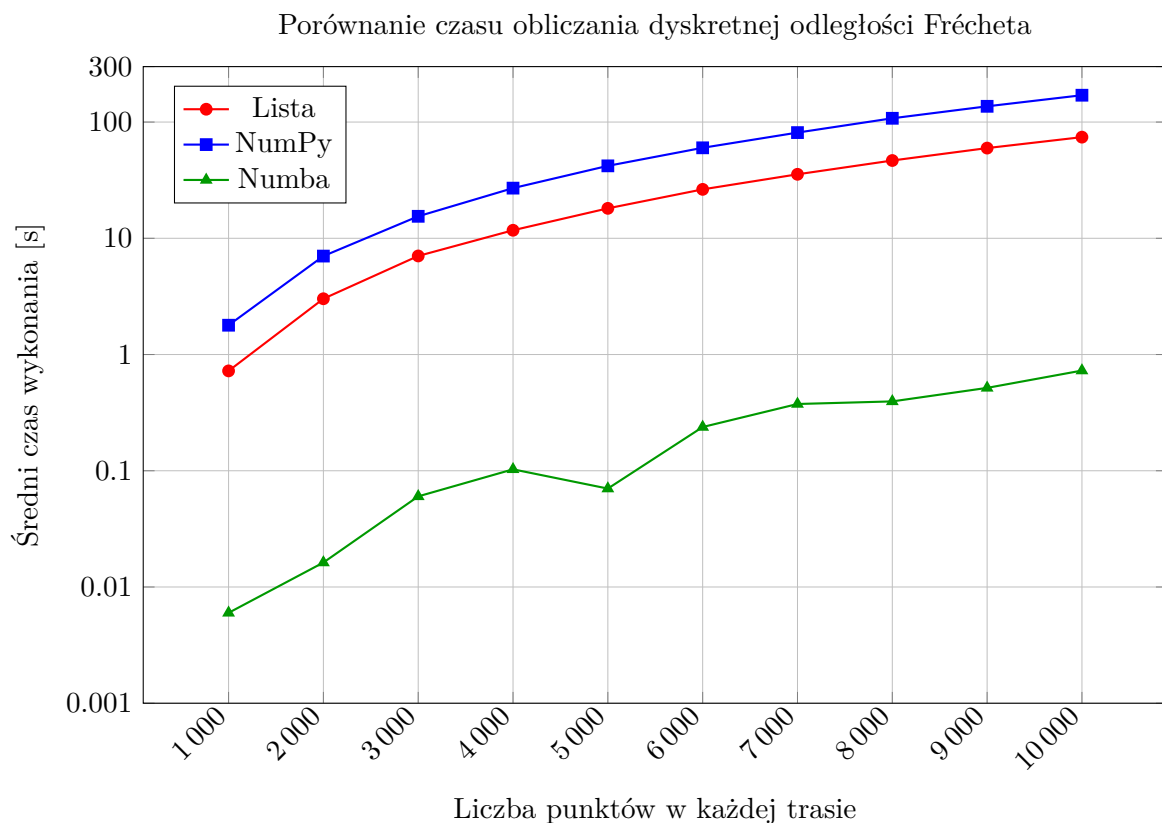
4.3.5 Dyskretna odległość Fréchet

Ostatnią analizowaną operacją jest obliczanie dyskretnej odległości Frécheta między trasą referencyjną a trasą zawodnika. Metoda ta porównuje dwie sekwencje punktów z uwzględnieniem ich kolejności i wymaga utworzenia macierzy programowania dynamicznego. Dla dwóch tras o tej samej liczbie punktów n macierz zawiera n^2 komórek, dlatego zarówno liczba aktualizowanych stanów, jak i zapotrzebowanie na pamięć rosną kwadratowo względem liczby punktów.

Tabela 4.7 przedstawia średnie czasy wykonania dla wariantu listowego, NumPy oraz Numba. Zastosowano ten sam zakres par tras jak w benchmarku porównywania punktów z segmentami, czyli od 1000 do 10 000 punktów.

Tabela 4.7: Średnie czasy wykonania benchmarku dyskretnej odległości Fréchet

Liczba punktów	Komórki DP	Lista [s]	NumPy [s]	Numba [s]
1 000	1 000 000	0,723205	1,787297	0,005993
2 000	4 000 000	3,018974	7,030743	0,016234
3 000	9 000 000	7,048458	15,460332	0,060198
4 000	16 000 000	11,724797	27,014392	0,102872
5 000	25 000 000	18,100327	41,999846	0,070250
6 000	36 000 000	26,357626	60,041088	0,237831
7 000	49 000 000	35,527144	81,173815	0,375307
8 000	64 000 000	46,686739	107,765870	0,395133
9 000	81 000 000	59,718604	136,604679	0,516438
10 000	100 000 000	74,181451	170,175190	0,727838



Wykres 4.8: Porównanie średniego czasu obliczania dyskretnej odległości Fréchet dla wariantów listowego, NumPy i Numba. Oś pionowa została przedstawiona w skali logarytmicznej ze względu na duże różnice czasu wykonania między wariantami.

Wyniki przedstawione w tabeli 4.7 pokazują bardzo szybki wzrost czasu wykonania wraz ze wzrostem liczby punktów. Jest to zgodne ze strukturą algorytmu, ponieważ dla tras o tej samej liczbie punktów liczba komórek macierzy programowania dynamicznego

rośnie kwadratowo. Dla 1000 punktów macierz zawiera 1 000 000 komórek, natomiast dla 10 000 punktów liczba ta wzrasta do 100 000 000. Wariant listowy dla największego przypadku testowego wykonał obliczenia w czasie około 74.18 s. Wariant NumPy był w tej metodzie wolniejszy i dla 10 000 punktów osiągnął średni czas około 170.18 s. Wynik ten można wyjaśnić sposobem użycia NumPy w tej implementacji. Biblioteka służy tu głównie do przechowywania macierzy jako tablicy `ndarray`, natomiast kolejne komórki macierzy są nadal wypełniane iteracyjnie w zagnieżdżonych pętlach zapisanych w Pythonie.

Najkrótsze czasy wykonania uzyskał wariant Numba. Dla 1000 punktów średni czas wyniósł około 0.006 s, natomiast dla 10 000 punktów około 0.728 s. Oznacza to, że dla największego badanego przypadku wariant Numba był około 102 razy szybszy od wariantu listowego oraz około 234 razy szybszy od wariantu NumPy.

Szczególnie istotne jest porównanie wariantu listowego i NumPy. W operacjach takich jak obliczanie dystansu lub przewyższeń NumPy pozwalał zapisać znaczną część obliczeń jako operacje tablicowe. W przypadku dyskretnej odległości Frécheta zależność rekurencyjna wymusza jednak sekwencyjne wypełnianie macierzy. Z tego powodu samo zastosowanie tablicy NumPy nie gwarantuje przyspieszenia. Może wręcz zwiększyć koszt wykonania, jeżeli dostęp do elementów tablicy odbywa się w zagnieżdżonej pętli zapisanej nadal po stronie Pythona.

Wariant Numba lepiej odpowiada strukturze tej metody, ponieważ zachowuje jawne pętle, ale wykonuje je w funkcji skompilowanej. Wyniki benchmarku wskazują, że dla algorytmu programowania dynamicznego, w którym pełna wektoryzacja jest utrudniona, kompilacja numerycznego rdzenia może być znacznie korzystniejsza niż samo użycie tablic NumPy.

Warto zauważyć, że czasy wariantu Numba nie rosną idealnie monotonicznie dla każdego kolejnego rozmiaru danych. Przykładowo wynik dla 5000 punktów jest niższy niż dla 4000 punktów. Może to wynikać z czynników pomiarowych, takich jak chwilowe obciążenie systemu, zarządzanie pamięcią, sposób alokacji macierzy lub wpływ środowiska wykonania. Z tego powodu pojedyncze odchylenia nie powinny być interpretowane jako cecha samego algorytmu. Istotniejszy jest ogólny trend, zgodnie z którym czas wykonania rośnie wraz z liczbą komórek macierzy DP, a wariant Numba pozostaje wyraźnie najszybszy w całym badanym zakresie.

4.4 Wnioski

Wnioski z benchmarków należy odnosić do konkretnych grup operacji, ponieważ badane funkcje mają różną strukturę obliczeniową. Operacje liniowe, takie jak dystans i przewyższenia, wymagają przejścia po jednej sekwencji punktów. Porównywanie tras oraz dyskretna odległość Frécheta operują natomiast na relacjach między dwiema sekwencjami,

co powoduje znacznie szybszy wzrost kosztu obliczeń.

W przypadku obliczania dystansu i przewyższeń warianty NumPy oraz Numba uzyskały wyraźnie krótsze czasy wykonania niż wariant listowy. Wynika to z faktu, że obie operacje można wyrazić jako przetwarzanie sąsiednich elementów jednej sekwencji. Wariant NumPy wykorzystuje w tych przypadkach przesunięte wycinki tablic oraz operacje tablicowe, natomiast wariant Numba zachowuje jawną pętlę, ale wykonuje ją w funkcji kompilowanej. Dla takich operacji ograniczenie narzutu interpretera Pythona może być jedną z istotnych przyczyn uzyskanego przyspieszenia.

Inaczej wygląda sytuacja w przypadku metod porównujących dwie trasy. Porównywanie punktów z segmentami trasy referencyjnej ma większy koszt obliczeniowy niż operacje liniowe, ponieważ każdy punkt śladu zawodnika jest porównywany z wieloma segmentami trasy referencyjnej. W tej metodzie wariant NumPy był szybszy od wariantu listowego, ale jego przewaga była mniejsza niż przy dystansie i przewyższeniach. Jest to zgodne z implementacją, ponieważ wariant NumPy nie eliminuje całej pętli po punktach zawodnika. Wektoryzacji podlega przede wszystkim obliczenie odległości jednego punktu od wszystkich segmentów referencyjnych.

Segmentowe porównanie prędkości ma odmienny charakter od pozostałych metod działających na dwóch trasach. Wykorzystuje zarówno współrzędne punktów, jak i znaczniki czasu, ale jego główny koszt wynika przede wszystkim z przejścia po punktach obu śladów, obliczenia dystansu narastającego oraz wyznaczenia czasów na granicach segmentów. Metoda nie wymaga porównywania każdego punktu z wieloma segmentami ani tworzenia macierzy programowania dynamicznego, dlatego mogła zostać przetestowana na większych plikach wejściowych.

Najbardziej wyraźne ograniczenie wariantu NumPy zaobserwowano przy dyskretnej odległości Frécheta. W tej metodzie NumPy służy głównie do przechowywania macierzy programowania dynamicznego jako tablicy `ndarray`, natomiast sama macierz nadal jest wypełniana w zagnieżdżonych pętlach zapisanych w Pythonie. W przeprowadzonych pomiarach wariant NumPy był wolniejszy od wariantu listowego. Wynik ten pokazuje, że samo użycie tablic NumPy nie jest równoznaczne z przyspieszeniem obliczeń. Korzyść z NumPy zależy od tego, czy struktura algorytmu pozwala przenieść znaczącą część pracy do operacji tablicowych wykonywanych poza jawną pętlą Pythona.

W przeprowadzonych pomiarach wariant Numba uzyskał najkrótsze czasy wykonania we wszystkich analizowanych grupach operacji. Szczególnie duża przewaga wystąpiła w metodach zawierających kosztowne pętle numeryczne, takich jak porównywanie tras oraz dyskretna odległość Frécheta. W tych przypadkach struktura algorytmu jest trudna do pełnej wektoryzacji, ale dobrze nadaje się do kompilacji pętli operujących na liczbach i tablicach. Wyniki te wskazują, że dla analizowanych metod Numba była najkorzystniejszym wariantem wykonawczym, jednak wniosek ten dotyczy konkretnej implementacji, danych

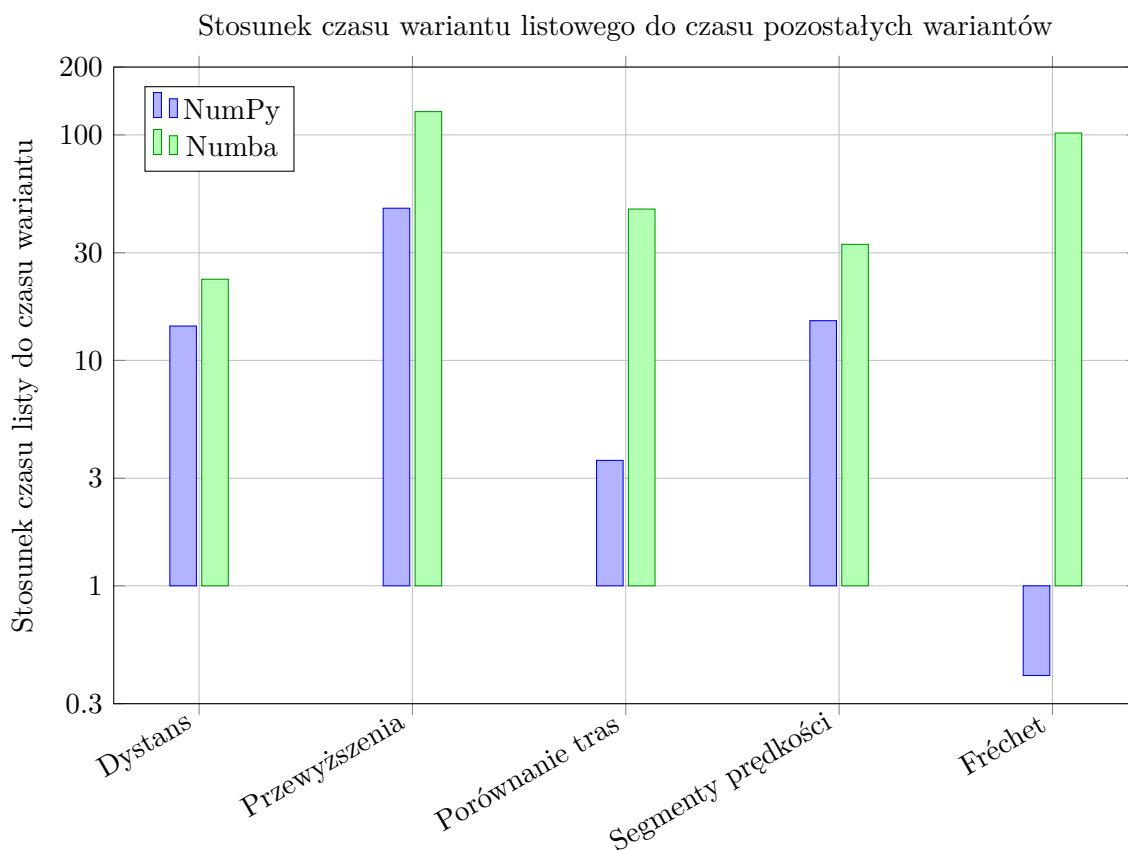
testowych oraz środowiska pomiarowego.

Tabela 4.8: Stosunek czasu wariantu listowego do czasu wariantów NumPy i Numba dla największych badanych danych

Operacja	Lista / NumPy	Lista / Numba
Obliczanie dystansu, 1 000 000 punktów	około 14,2×	około 22,9×
Obliczanie przewyższeń, 1 000 000 punktów	około 47,3×	około 126,9×
Porównywanie tras, 10 000 punktów	około 3,6×	około 46,9×
Segmentowe porównanie prędkości, 1 000 000 punktów	około 15,0×	około 32,7×
Dyskretna odległość Frécheta, 10 000 punktów	około 0,44×	około 101,9×

Wartość większa od 1 oznacza, że dany wariant był szybszy od wariantu listowego. Wartość mniejsza od 1, widoczna dla wariantu NumPy przy dyskretnej odległości Frécheta, oznacza wynik wolniejszy od wariantu listowego.

Tabela 4.8 zbiera najważniejsze różnice między wariantami dla największych danych użytych w poszczególnych benchmarkach. Zestawienie to pokazuje, że stosunek czasu wariantu listowego do pozostałych wariantów nie jest stałą cechą biblioteki lub technologii, lecz zależy od konkretnej operacji. NumPy bardzo dobrze sprawdziło się przy przewyższeniach, gdzie obliczenia można sprowadzić do różnic sąsiednich elementów i masek logicznych. Segmentowe porównanie prędkości również skorzystało z reprezentacji tablicowej, ale największe przyspieszenie w tej metodzie uzyskał wariant Numba. Jednocześnie przy Fréchecie wariant NumPy był wolniejszy od listowego, ponieważ nie usuwał głównej zależności rekurencyjnej ani zagnieżdżonych pętli.



Wykres 4.9: Stosunek czasu wariantu listowego do czasu wariantów NumPy i Numba dla największych danych w każdej grupie operacji. Wartości większe od 1 oznaczają wynik szybszy od wariantu listowego, natomiast wartości mniejsze od 1 oznaczają wynik wolniejszy. Oś pionowa została przedstawiona w skali logarytmicznej.

Interpretując wyniki, należy uwzględnić ograniczenia przyjętej metodyki. Dane wejściowe miały charakter syntetyczny, dzięki czemu możliwe było kontrolowanie liczby punktów, kompletności struktury GPX oraz powtarzalności benchmarków. Nie oznacza to jednak, że odwzorowują one wszystkie cechy rzeczywistych śladów GPS, takich jak przerwy w zapisie, zmienny interwał rejestracji, błędy wysokości czy zakłócenia pomiaru pozycji.

Na wyniki wpływa również sposób wydzielenia mierzonego fragmentu programu. Benchmarki obejmowały czas wykonania funkcji analitycznych na przygotowanych danych, a nie pełny czas przetwarzania pliku od odczytu XML do uzyskania wyniku końcowego. Takie rozdzielanie było potrzebne, ponieważ celem pracy było porównanie wariantów obliczeniowych, a nie wydajności parsera GPX.

Pomiary czasu mogą być zależne od chwilowego obciążenia systemu, zarządzania pamięcią, środowiska WSL oraz mechanizmów działania interpretera Pythona. Ma to szczególne znaczenie dla bardzo krótkich czasów wykonania, gdzie niewielkie zakłócenia mogą wpływać na pojedyncze przebiegi. Z tego powodu interpretacja wyników powinna opierać się przede wszystkim na trendach widocznych dla rosnącej liczby punktów, a nie

na pojedynczych odchyleniach.

W przypadku wariantu Numba trzeba dodatkowo pamiętać o rozdzieleniu kosztu kompilacji JIT od czasu wykonania funkcji już skompilowanej. W benchmarkach zastosowano wywołania przygotowawcze, dlatego prezentowane wyniki opisują przede wszystkim czas działania funkcji po zakończeniu etapu kompilacji.

Najważniejszy wniosek z benchmarków jest taki, że wybór wariantu implementacyjnego powinien wynikać ze struktury algorytmu. NumPy było korzystne wtedy, gdy znaczną część obliczeń można było zapisać jako operacje tablicowe. Numba okazała się szczególnie skuteczna w metodach opartych na kosztownych pętlach numerycznych. Wyniki te dotyczą jednak przygotowanej implementacji, danych syntetycznych i wskazanego środowiska pomiarowego.

Podsumowanie

Celem pracy było zaprojektowanie i zaimplementowanie oprogramowania do analizy tras zapisanych w formacie GPX oraz porównanie wybranych wariantów obliczeniowych pod względem czasu wykonania. W pracy przygotowano parser plików GPX, funkcje obliczające dystans i przewyższenia, metodę porównywania śladu zawodnika z trasą referencyjną, segmentowe porównanie prędkości wykorzystujące znaczniki czasu oraz implementację dyskretnej odległości Frécheta. Każdą z głównych operacji przygotowano w wariancie listowym, NumPy oraz Numba.

Przeprowadzone benchmarki pokazały, że wpływ zastosowanego wariantu implementacyjnego zależy od charakteru danej operacji. W przypadku obliczania dystansu i przewyższeń warianty NumPy oraz Numba uzyskały wyraźnie krótsze czasy wykonania niż wariant oparty na listach Pythona. Wynika to z faktu, że operacje te mają prostą strukturę sekwencyjną i można je stosunkowo dobrze zapisać jako działania na tablicach albo jako pętle numeryczne kompilowane przez Numbę.

W metodach porównujących dwie trasy różnice między wariantami były bardziej zależne od struktury algorytmu. Porównywanie punktów śladu zawodnika z segmentami trasy referencyjnej wymaga dużej liczby obliczeń punkt-segment, dlatego szczególnie korzystny okazał się wariant Numba. W przypadku dyskretnej odległości Frécheta samo użycie tablic NumPy nie wystarczyło do uzyskania przyspieszenia, ponieważ algorytm nadal wymagał iteracyjnego wypełniania macierzy programowania dynamicznego. Wynik ten pokazuje, że NumPy nie powinno być traktowane jako automatyczna gwarancja szybszego działania programu.

Przygotowane rozwiązanie ma charakter biblioteki funkcji analitycznych, a nie kompletnej aplikacji użytkowej. Program pozwala odczytywać dane GPX, wykonywać obliczenia oraz uruchamiać benchmarki, ale nie posiada interfejsu graficznego ani mechanizmów obsługi typowych dla gotowego systemu przeznaczonego dla użytkownika końcowego. Naturalnym kierunkiem rozwoju byłoby zbudowanie aplikacji wykorzystującej przygotowane funkcje, na przykład do wczytywania tras, wskazywania trasy referencyjnej i prezentowania wyników na mapie oraz w tabelach.

Kolejnym możliwym kierunkiem rozwoju byłoby rozszerzenie testów o rzeczywiste dane pochodzące z urządzeń GPS. W pracy wykorzystano dane syntetyczne, ponieważ pozwa-

łały kontrolować liczbę punktów i zachować powtarzalność pomiarów. Dane rzeczywiste mogłyby jednak lepiej pokazać problemy praktyczne, takie jak brakujące wysokości, wiele segmentów śladu, nierówny odstęp czasowy między punktami albo zakłócenia pomiaru pozycji.

Podsumowując, praca pokazała, że wydajność analizy tras GPX zależy nie tylko od samej definicji metody obliczeniowej, lecz także od sposobu reprezentacji danych i mechanizmu wykonania kodu. Najlepsze wyniki uzyskano tam, gdzie struktura algorytmu dobrze pasowała do operacji tablicowych lub do kompilacji pętli numerycznych. Opracowany program może stanowić podstawę do dalszej rozbudowy w kierunku praktycznego narzędzia wspierającego analizę tras sportowych.

Bibliografia

- [1] Ride with GPS, *The Elevation Profile on Web*, tryb dostępu: <https://support.ridewithgps.com/hc/en-us/articles/4419005868315>, (dostęp: 21 maja 2026 r.).
- [2] Ride with GPS, *Route Planning 101*, tryb dostępu: <https://support.ridewithgps.com/hc/en-us/articles/4415462488475>, (dostęp: 21 maja 2026 r.).
- [3] Topografix, *GPX 1.1 Schema Documentation*, tryb dostępu: <https://www.topografix.com/gpx/1/1/>, (dostęp: 2 marca 2026 r.).
- [4] J. L. Hennessy, D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, Cambridge, 2017.
- [5] R. E. Bryant, D. R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., Pearson, Boston, 2016.
- [6] Python Software Foundation, *The Python Language Reference: Data Model*, tryb dostępu: <https://docs.python.org/3/reference/datamodel.html>, (dostęp: 3 marca 2026 r.).
- [7] Python Software Foundation, *Design and History FAQ: How Are Lists Implemented in CPython?*, tryb dostępu: <https://docs.python.org/3/faq/design.html>, (dostęp: 21 maja 2026 r.).
- [8] Python Software Foundation, *xml.etree.ElementTree – The ElementTree XML API*, tryb dostępu: <https://docs.python.org/3/library/xml.etree.elementtree.html>, (dostęp: 21 maja 2026 r.).
- [9] pyproj Developers, *Transformer – pyproj Documentation*, tryb dostępu: <https://pyproj4.github.io/pyproj/stable/api/transformer.html>, (dostęp: 21 maja 2026 r.).
- [10] M. J. Flynn, *Very High-Speed Computing Systems, Proceedings of the IEEE*, 1966, t. 54, nr 12, s. 1901–1909, doi: 10.1109/PROC.1966.5273.

- [11] OpenMP Architecture Review Board, *OpenMP API Specification 5.1 – SIMD Construct*, tryb dostępu: <https://www.openmp.org/spec-html/5.1/openmpsu49.html>, (dostęp: 2 marca 2026 r.).
- [12] G. M. Amdahl, *Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities*, *AFIPS Conference Proceedings*, 1967, t. 30, s. 483–485, doi: 10.1145/1465482.1465560.
- [13] J. Januszewski, *Systemy satelitarne GPS, Galileo i inne*, Wydawnictwo Naukowe PWN, Warszawa, 2010.
- [14] National Geospatial-Intelligence Agency, *Department of Defense World Geodetic System 1984: Its Definition and Relationships with Local Geodetic Systems*, NGA.STND.0036_1.0.0_WGS84, Version 1.0.0, 2014.
- [15] J. Pasławski (red.), *Wprowadzenie do kartografii i topografii*, Nowa Era, Warszawa–Wrocław, 2010.
- [16] EPSG Geodetic Parameter Dataset, *ETRS89 / PL-1992, EPSG:2180*, tryb dostępu: https://epsg.org/crs_2180/ETRF2000-PL-CS92.html, (dostęp: 18 czerwca 2026 r.).
- [17] R. Sanchez, P. E. Egli, K. Jornet, M. Duggan, M. Besomi, *How Fractal Complexity Distorts Distance and Elevation Gain in Trail and Mountain Running: The Case for Course Measurement Standardization*, *Proceedings of the Institution of Mechanical Engineers, Part P: Journal of Sports Engineering and Technology*, 2025, doi: 10.1177/17543371251341660.
- [18] R. W. Sinnott, *Virtues of the Haversine*, *Sky and Telescope*, 1984, t. 68, nr 2, s. 159.
- [19] P. Ranacher, R. Brunauer, W. Trutschnig, S. Van der Spek, S. Reich, *Why GPS Makes Distances Bigger Than They Are*, *International Journal of Geographical Information Science*, 2016, t. 30, nr 2, s. 316–333, doi: 10.1080/13658816.2015.1086924.
- [20] Open Geospatial Consortium, *OpenGIS Implementation Standard for Geographic Information – Simple Feature Access – Part 1: Common Architecture*, OGC 06-103r4, version 1.2.1, 2011, tryb dostępu: <https://docs.ogc.org/is/06-103r4/06-103r4.pdf>, (dostęp: 21 maja 2026 r.).
- [21] C. Ericson, *Real-Time Collision Detection*, Morgan Kaufmann, San Francisco, 2005, rozdz. 5.1.2: *Closest Point on Line Segment to Point*, ISBN 978-1-55860-732-3.
- [22] T. Eiter, H. Mannila, *Computing Discrete Fréchet Distance*, Technical Report CD-TR 94/64, Christian Doppler Laboratory for Expert Systems, Technische Universität

- Wien, 1994, tryb dostępu: <https://www.kr.tuwien.ac.at/staff/eiter/et-archive/files/cdtr9464.pdf>, (dostęp: 17 maja 2026 r.).
- [23] M. Gorelick, I. Ozsvald, *High Performance Python: Practical Performant Programming for Humans*, 2nd ed., O'Reilly Media, Sebastopol, 2020.
- [24] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith i in., *Array Programming with NumPy*, *Nature*, 2020, t. 585, s. 357–362, doi: 10.1038/s41586-020-2649-2.
- [25] NumPy Developers, *The N-dimensional Array*, tryb dostępu: <https://numpy.org/doc/stable/reference/arrays.ndarray.html>, (dostęp: 21 maja 2026 r.).
- [26] NumPy Developers, *Universal Functions (ufunc)*, tryb dostępu: <https://numpy.org/doc/stable/reference/ufuncs.html>, (dostęp: 21 maja 2026 r.).
- [27] NumPy Developers, *Broadcasting*, tryb dostępu: <https://numpy.org/doc/stable/user/basics.broadcasting.html>, (dostęp: 21 maja 2026 r.).
- [28] S. K. Lam, A. Pitrou, S. Seibert, *Numba: A LLVM-Based Python JIT Compiler*, *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, 2015, s. 1–6, doi: 10.1145/2833157.2833162.
- [29] Y. Zheng, *Trajectory Data Mining: An Overview*, *ACM Transactions on Intelligent Systems and Technology*, 2015, t. 6, nr 3, art. 29, doi: 10.1145/2743025.
- [30] D. Pfoser, C. S. Jensen, Y. Theodoridis, *Novel Approaches to the Indexing of Moving Object Trajectories*, *Proceedings of the 26th International Conference on Very Large Data Bases*, Cairo, 2000, s. 395–406.
- [31] J. J. Malone, R. Lovell, M. C. Varley, A. J. Coutts, *Unpacking the Black Box: Applications and Considerations for Using GPS Devices in Sport*, *International Journal of Sports Physiology and Performance*, 2017, t. 12, Suppl. 2, s. S2-18–S2-26, doi: 10.1123/ijsp.2016-0236.
- [32] Numba Developers, *Compiling Python Code with @jit*, tryb dostępu: <https://numba.readthedocs.io/en/stable/user/jit.html>, (dostęp: 21 maja 2026 r.).

Spis listingów

1.1	Fragment przykładowego pliku GPX	11
3.1	Ogólna struktura katalogów projektu	29
3.2	Parser GPX w wariancie listowym	31
3.3	Parser GPX w wariancie NumPy	32
3.4	Obliczanie dystansu i przewyższeń w wariancie listowym	34
3.5	Obliczanie dystansu i przewyższeń w wariancie NumPy	35
3.6	Obliczanie dystansu i przewyższeń w wariancie Numba	37
3.7	Funkcje pomocnicze porównywania tras w wariancie listowym	39
3.8	Agregacja wyników porównania tras w wariancie listowym	40
3.9	Budowa segmentów i odległość punkt–segment w wariancie NumPy	42
3.10	Porównywanie tras w wariancie NumPy	43
3.11	Rdzeń porównywania tras w wariancie Numba	44
3.12	Funkcja zewnętrzna porównywania tras w wariancie Numba	45
3.13	Segmentowe porównanie prędkości w wariancie listowym	47
3.14	Segmentowe porównanie prędkości w wariancie NumPy	49
3.15	Segmentowe porównanie prędkości w wariancie Numba	51
3.16	Dyskretna odległość Frécheta w wariancie listowym	53
3.17	Dyskretna odległość Frécheta w wariancie NumPy	54
3.18	Rdzeń obliczania dyskretniej odległości Frécheta w wariancie Numba . . .	56
3.19	Funkcja zewnętrzna obliczania odległości Frécheta w wariancie Numba .	56
3.20	Funkcja pomocnicza do pomiaru czasu wykonania	58
3.21	Zakres danych i liczba pomiarów dla operacji liniowych	59
3.22	Zakres danych dla benchmarków metod porównujących dwie trasy	60
3.23	Przykład krótkiego wywołania wymuszającego kompilację Numby	61
3.24	Przykładowa struktura rekordu wynikowego benchmarku	61

Spis wykresów

4.1	Porównanie średniego czasu obliczania dystansu dla wariantów listowego, NumPy i Numba.	66
4.2	Porównanie średniego czasu obliczania dystansu dla wariantów NumPy i Numba.	67
4.3	Porównanie średniego czasu obliczania przewyższeń dla wariantów listowego, NumPy i Numba. Czas przedstawiono w milisekundach.	68
4.4	Porównanie średniego czasu obliczania przewyższeń dla wariantów NumPy i Numba. Czas przedstawiono w milisekundach.	69
4.5	Porównanie średniego czasu porównywania tras dla wariantów listowego, NumPy i Numba. Oś pionowa została przedstawiona w skali logarytmicznej ze względu na duże różnice czasu wykonania między wariantami.	71
4.6	Porównanie średniego czasu wykonania segmentowego porównania prędkości dla wariantów listowego, NumPy i Numba.	73
4.7	Porównanie średniego czasu wykonania segmentowego porównania prędkości dla wariantów NumPy i Numba.	74
4.8	Porównanie średniego czasu obliczania dyskretnej odległości Fréchet'a dla wariantów listowego, NumPy i Numba. Oś pionowa została przedstawiona w skali logarytmicznej ze względu na duże różnice czasu wykonania między wariantami.	75
4.9	Stosunek czasu wariantu listowego do czasu wariantów NumPy i Numba dla największych danych w każdej grupie operacji. Wartości większe od 1 oznaczają wynik szybszy od wariantu listowego, natomiast wartości mniejsze od 1 oznaczają wynik wolniejszy. Oś pionowa została przedstawiona w skali logarytmicznej.	79

Spis tabel

4.1	Podstawowe parametry środowiska testowego	64
4.2	Zakres danych i liczba powtórzeń w benchmarkach	65
4.3	Średnie czasy wykonania benchmarku obliczania dystansu	66
4.4	Średnie czasy wykonania benchmarku obliczania przewyższeń	68
4.5	Średnie czasy wykonania benchmarku porównywania tras	70
4.6	Średnie czasy wykonania segmentowego porównania prędkości	72
4.7	Średnie czasy wykonania benchmarku dyskretnej odległości Frécheta . . .	75
4.8	Stosunek czasu wariantu listowego do czasu wariantów NumPy i Numba dla największych badanych danych	78

